



TECNOLOGICO NACIONAL DE MEXICO

INSTITUTO TECNOLÓGICO de Tuxtepec

“ACTIVIDAD”

CURSO INTENSIVO

Materia:

C DATOS

Carrera

Ing. Sistemas Computacionales.

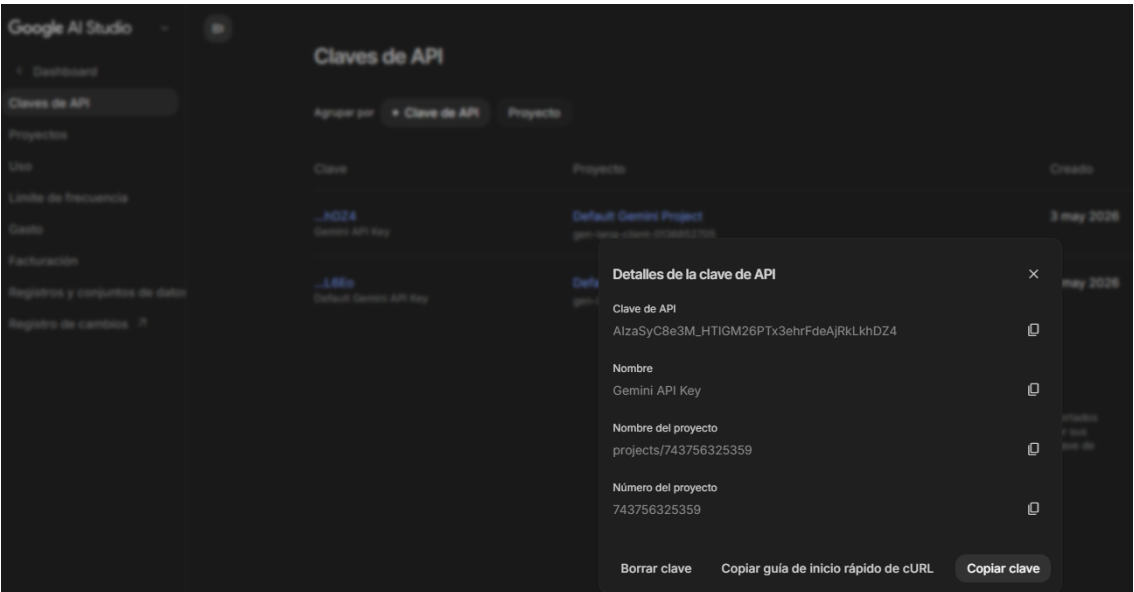
Presentan:

Urieta Ubieta Jesús 22350384

Asesor:

JULIO AGUILAR CARMONA

Tuxtepec, Oax. 04 de Mayo de 2026



Day 1a - From Prompt to Action

Draft guardado

Archivo Edición Vista Corre Escenarios Complementos Ayuda

+ Ejecutar todo Código Sesión de borrador Apagado (ejecuta una celda para empezar)

Section 1: Setup

1.1: Install dependencies

The Kaggle Notebooks environment includes a pre-installed version of the [google-adk](#) library for Python and its required dependencies, so you don't need to install additional packages in this notebook.

To install and use ADK in your own Python development environment outside of this course, you can do so by running:

```
pip install google-adk
```

Section 1: Setup

1.1: Install dependencies

The Kaggle Notebooks environment includes a pre-installed version of the [google-adk](#) library for Python and its required dependencies, so you don't need to install additional packages in this notebook.

To install and use ADK in your own Python development environment outside of this course, you can do so by running:

```
pip install google-adk
```

```
pip install google-adk
```

```
Requirement already satisfied: google-adk in /usr/local/lib/python3.11/dist-packages (1.18.0)
Requirement already satisfied: PyYAML<7.0.0,>=6.0.2 in /usr/local/lib/python3.11/dist-packages (from google-adk) (6.0.3)
Requirement already satisfied: anyio<5.0.0,>=4.9.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (4.11.0)
Requirement already satisfied: authlib<2.0.0,>=1.5.1 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.6.5)
Requirement already satisfied: click<9.0.0,>=8.1.8 in /usr/local/lib/python3.11/dist-packages (from google-adk) (8.3.0)
Requirement already satisfied: fastapi<1.119.0,>=0.115.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.116.1)
Requirement already satisfied: google-api-python-client<3.0.0,>=2.157.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.177.0)
Requirement already satisfied: google-cloud-aiplatform<2.0.0,>=1.125.0 in /usr/local/lib/python3.11/dist-packages (from google-cloud-aiplatform[agent-engines]<2.0.0,>=1.125.0->google-adk) (1.125.0)
Requirement already satisfied: google-cloud-bigtable>=2.32.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.34.0)
Requirement already satisfied: google-cloud-discoveryengine<0.14.0,>=0.13.12 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.13.12)
Requirement already satisfied: google-cloud-secret-manager<3.0.0,>=2.22.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.25.0)
Requirement already satisfied: google-cloud-spanner<4.0.0,>=3.56.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (3.56.0)
Requirement already satisfied: google-cloud-speech<3.0.0,>=2.30.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.34.0)
Requirement already satisfied: google-cloud-storage<4.0.0,>=3.0.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (3.5.0)
Requirement already satisfied: google-generativeai<2.0.0,>=1.45.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.48.0)
Requirement already satisfied: graphviz<1.0.0,>=0.20.2 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.21)
Requirement already satisfied: mpc<2.0.0,>=1.8.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.20.0)
Requirement already satisfied: opentelemetry-api<1.37.0,>=1.37.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.37.0)
Requirement already satisfied: opentelemetry-exporter-gcp-logging<2.0.0,>=1.9.0a0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.11.0a0)
Requirement already satisfied: opentelemetry-exporter-gcp-monitoring<2.0.0,>=1.9.0a0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.11.0a0)
Requirement already satisfied: opentelemetry-exporter-gcp-trace<2.0.0,>=1.9.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.11.0)
Requirement already satisfied: opentelemetry-exporter-otlp-proto-http<=1.36.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.37.0)
Requirement already satisfied: opentelemetry-resource-detector-gcp<2.0.0,>=1.9.0a0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.11.0a0)
Requirement already satisfied: opentelemetry-sdk<1.37.0,>=1.37.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.37.0)
Requirement already satisfied: pydantic<3.0.0,>=2.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.12.4)
Requirement already satisfied: python-dateutil<3.0.0,>=2.9.0.post0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.9.0.post0)
Requirement already satisfied: python-dotenv<2.0.0,>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.2.1)
Requirement already satisfied: requests<3.0.0,>=2.32.4 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.32.5)
Requirement already satisfied: sqlalchemy-spanner>=1.14.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.17.1)
```

Day 1a - From Prompt to Action Draft saved

File Edit View Run Settings Add-ons Help

1.2: Configure

This notebook uses the

Google Cloud Services

Google Cloud SDK

Install Dependencies

Secrets

New secret

LABEL

GOOGLE_API_KEY

VALUE

e3M_HTIGM26PTx3ehrFdeAjRkLkhDZ4

Save

Cancel

1.2: Configure your Gemini API Key

This notebook uses the [Gemini API](#), which requires authentication.

1. Get your API key

If you don't have one already, create an [API key](#) in [Google AI Studio](#).

2. Add the key to Kaggle Secrets

Next, you will need to add your API key to your Kaggle Notebook as a Kaggle User Secret.

1. In the top menu bar of the notebook editor, select **Add-ons** then **Secrets**.
2. Create a new secret with the label `GOOGLE_API_KEY`.
3. Paste your API key into the "Value" field and click "Save".
4. Ensure that the checkbox next to `GOOGLE_API_KEY` is selected so that the secret is attached to the notebook.

3. Authenticate in the notebook

Run the cell below to complete authentication.

+ Code + Markdown

```
[ ]: import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Gemini API key setup complete.")
except Exception as e:
    print(
        f"🚨 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}"
    )
```

```
> import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Gemini API key setup complete.")
except Exception as e:
    print(
        f"🚨 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}"
    )
```

✅ Gemini API key setup complete.

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

+ Code + Markdown

```
[ ]: from google.adk.agents import Agent
from google.adk.models.google_llm import Gemini
from google.adk.runners import InMemoryRunner
from google.adk.tools import google_search
from google.genai import types

print("✅ ADK components imported successfully.")
```

```
[3]: from google.adk.agents import Agent
from google.adk.models.google_llm import Gemini
from google.adk.runners import InMemoryRunner
from google.adk.tools import google_search
from google.genai import types

print("✅ ADK components imported successfully.")
```

✅ ADK components imported successfully.

1.4: Helper functions

We'll define some helper functions. If you are running this outside the Kaggle environment, you don't need to do this.

```
# Define helper functions that will be reused throughout the notebook

from IPython.core.display import display, HTML
from jupyter_server.serverapp import list_running_servers

# Gets the proxied URL in the Kaggle Notebooks environment
def get_adk_proxy_url():
    PROXY_HOST = "https://kbb-production.jupyter-proxy.kaggle.net"
    ADK_PORT = "8000"

    servers = list(list_running_servers())
    if not servers:
        raise Exception("No running Jupyter servers found.")

    baseURL = servers[0]["base_url"]

    try:
        path_parts = baseURL.split("/")
        kernel = path_parts[2]
        token = path_parts[3]
    except IndexError:
        raise Exception(f"Could not parse kernel/token from base URL: {baseURL}")

    url_prefix = f"/k/{kernel}/{token}/proxy/proxy/{ADK_PORT}"
    url = f"{PROXY_HOST}{url_prefix}"

    styled_html = f"""
<div style="padding: 15px; border: 2px solid #f0ad4e; border-radius: 8px; background-color: #fef9f0; margin: 20px 0;">
  <div style="font-family: sans-serif; margin-bottom: 12px; color: #333; font-size: 1.1em;">
    <strong>⚠ IMPORTANT: Action Required</strong>
  </div>
  <div style="font-family: sans-serif; margin-bottom: 15px; color: #333; line-height: 1.5;">
    The ADK web UI is <strong>not running yet</strong>. You must start it in the next cell.
    <ol style="margin-top: 10px; padding-left: 20px;">
      <li style="margin-bottom: 5px;"><strong>Run the next cell</strong> (the one with <code>!adk web ...</code>) to start the ADK web UI.</li>
      <li style="margin-bottom: 5px;"><strong>Wait for that cell to show it is "Running" (it will not "complete").</li>
      <li>Once it's running, <strong>return to this button</strong> and click it to open the UI.</li>
    </ol>
    <em style="font-size: 0.9em; color: #555;">(If you click the button before running the next cell, you will get a 500 error.)</em>
  </div>
  <a href='{url}' target='_blank' style="
    display: inline-block; background-color: #1a73e8; color: white; padding: 10px 20px;
    text-decoration: none; border-radius: 25px; font-family: sans-serif; font-weight: 500;
    box-shadow: 0 2px 5px rgba(0,0,0,0.2); transition: all 0.2s ease;">
    Open ADK Web UI (after running cell below) ↗
  </a>
</div>
"""

display(HTML(styled_html))
```

```
    <em style="font-size: 0.9em; color: #555;">(If you click the button before running the next cell, you will get a 500 error.)</em>
  </div>
  <a href='{url}' target='_blank' style="
    display: inline-block; background-color: #1a73e8; color: white; padding: 10px 20px;
    text-decoration: none; border-radius: 25px; font-family: sans-serif; font-weight: 500;
    box-shadow: 0 2px 5px rgba(0,0,0,0.2); transition: all 0.2s ease;">
    Open ADK Web UI (after running cell below) ↗
  </a>
</div>
"""

display(HTML(styled_html))

return url_prefix

print("✅ Helper functions defined.")

✅ Helper functions defined.
```

1.5: Configure Retry Options

When working with LLMs, you may encounter transient errors like rate limits or temporary service unavailability. Retry options automatically handle these failures by retrying the request with exponential backoff.

+ Code

+ Markdown

```
retry_config=types.HttpRetryOptions(
    attempts=5, # Maximum retry attempts
    exp_base=7, # Delay multiplier
    initial_delay=1, # Initial delay before first retry (in seconds)
    http_status_codes=[429, 500, 503, 504] # Retry on these HTTP errors
)
```

+ Code

+ Markdown

🤖 Section 2: Your first AI Agent with ADK

🧐 2.1 What is an AI Agent?

You've probably used an LLM like Gemini before, where you give it a prompt and it gives you a text response.

Prompt -> LLM -> Text

An AI Agent takes this one step further. An agent can think, take actions, and observe the results of those actions to give you a better answer.

Prompt -> Agent -> Thought -> Action -> Observation -> Final Answer

In this notebook, we'll build an agent that can take the action of searching Google. Let's see the difference!

2.2 Define your agent

Now, let's build our agent. We'll configure an `Agent` by setting its key properties, which tell it what to do and how to operate.

To learn more, check out the documentation related to [agents](#) in ADK.

These are the main properties we'll set:

- **name and description:** A simple name and description to identify our agent.
- **model:** The specific LLM that will power the agent's reasoning. We'll use "gemini-2.5-flash-lite".
- **instruction:** The agent's guiding prompt. This tells the agent what its goal is and how to behave.
- **tools:** A list of [tools](#) that the agent can use. To start, we'll give it the `google_search` tool, which lets it find up-to-date information online.

```
1: root_agent = Agent(
    name="helpful_assistant",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    description="A simple agent that can answer general questions.",
    instruction="You are a helpful assistant. Use Google Search for current info or if unsure.",
    tools=[google_search],
)

print("✅ Root Agent defined.")
```

+ Code + Markdown

```
8]: root_agent = Agent(
    name="helpful_assistant",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    description="A simple agent that can answer general questions.",
    instruction="You are a helpful assistant. Use Google Search for current info or if unsure.",
    tools=[google_search],
)
```

```
[18]: root_agent = Agent(
    name="helpful_assistant",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    description="A simple agent that can answer general questions.",
    instruction="You are a helpful assistant. Use Google Search for current info or if unsure.",
    tools=[google_search],
)

print("✅ Root Agent defined.")
```

✅ Root Agent defined.

2.3 Run your agent

Now it's time to bring your agent to life and send it a query. To do this, you need a `Runner`, which is the central component within ADK that acts as the orchestrator. It manages the conversation, sends our messages to the agent, and handles its responses.

a. Create an `InMemoryRunner` and tell it to use our `root_agent`:

```
[ 3]: runner = InMemoryRunner(agent=root_agent)

print("✅ Runner created.")
```

➤ runner = InMemoryRunner(agent=root_agent)

```
print("✅ Runner created.")
```

✅ Runner created.

+ Code + Markdown

⚡ Note that we are using the Python Runner directly in this notebook. You can also run agents using ADK command-line tools such as `adk run`, `adk web`, or `adk api_server`. To learn more, check out the documentation related to [runtime in ADK](#).

b. Now you can call the `.run_debug()` method to send our prompt and get an answer.

⚡ This method abstracts the process of session creation and maintenance and is used in prototyping. We'll explore "what sessions are and how to create them" on Day 3.

```
[ ]: response = await runner.run_debug(
    "What is Agent Development Kit from Google? What languages is the SDK available in?"
)
```

```
> response = await runner.run_debug(
    "What is Agent Development Kit from Google? What languages is the SDK available in?"
)
```

```
### Created new session: debug_session_id

User > what is Agent Development kit from Google? what languages is the SDK available in?
helpful_assistant > The Google Agent Development Kit (ADK) is an open-source framework designed to simplify the creation, debugging, and deployment of AI agents and multi-agent systems. It applies software development principles to AI agent creation, allowing developers to build everything from personal AI assistants to complex business workflows. The ADK is optimized for Gemini and the Google ecosystem but is designed to be model-agnostic and deployment-agnostic, meaning it can work with various models and be deployed in different environments. It supports multi-agent architectures, enabling specialized agents to collaborate and delegate tasks. The framework also includes a rich tool ecosystem for integrating third-party applications and custom code.

The ADK SDK is available in the following languages:
* Python
* TypeScript
* Go
* Java
```

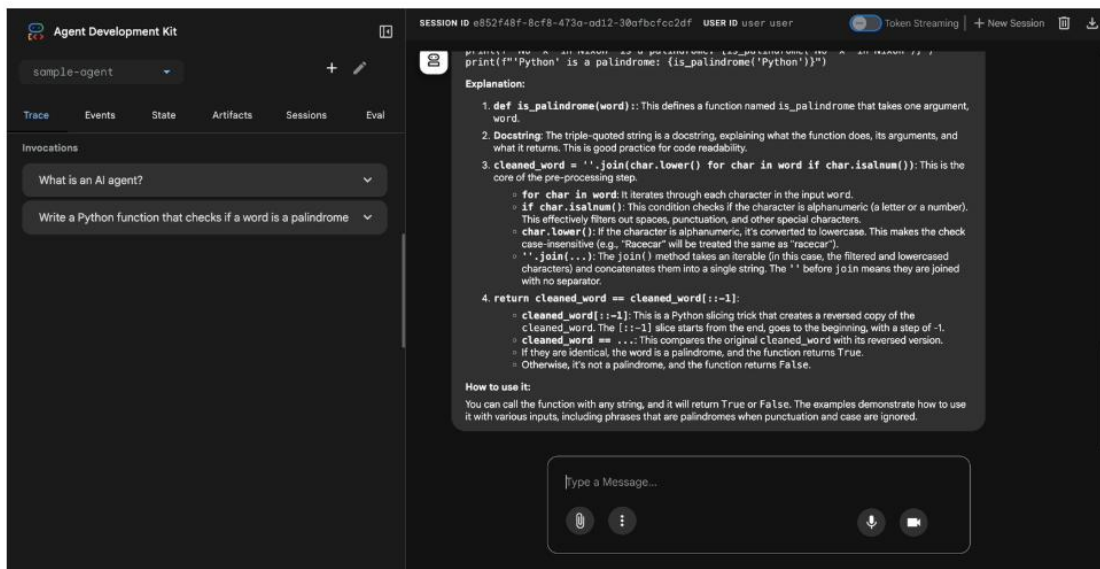
+ Code + Markdown

You can see a summary of ADK and its available languages in the response.

Section 3: Try the ADK Web Interface

Overview

ADK includes a built-in web interface for interactively chatting with, testing, and debugging your agents.



To use the ADK web UI, you'll need to create an agent with Python files using the `adk create` command.

Run the command below to generate a `sample-agent` folder that contains all the necessary files, including `agent.py` for your code, an `.env` file with your API key pre-configured, and an `__init__.py` file:

```
+ Code + Markdown
```

```
[ ]: !adk create sample-agent --model gemini-2.5-flash-lite --api_key $GOOGLE_API_KEY
```

```
> !adk create sample-agent --model gemini-2.5-flash-lite --api_key $GOOGLE_API_KEY
```

```
Agent created in /kaggle/working/sample-agent:
- .env
- __init__.py
- agent.py
```

```
[ 1: url_prefix = get_adk_proxy_url()
```

```
▶ url_prefix = get_adk_proxy_url()
```

⚠ IMPORTANT: Action Required

The ADK web UI is not running yet. You must start it in the next cell.

1. Run the next cell (the one with `!adk web ...`) to start the ADK web UI.
2. Wait for that cell to show it is "Running" (it will not "complete").
3. Once it's running, return to this button and click it to open the UI.

(If you click the button before running the next cell, you will get a 500 error.)

[Open ADK Web UI \(after running cell below\) ↗](#)

[+ Code](#) [+ Markdown](#)

Now we can run ADK web:

```
[ 1: !adk web --url_prefix {url_prefix}
```

```
!adk web --url_prefix {url_prefix}
```

```
/usr/local/lib/python3.11/dist-packages/google/adk/cli/fast_api.py:130: UserWarning: [EXPERIMENTAL] InMemoryCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  credential_service = InMemoryCredentialService()
/usr/local/lib/python3.11/dist-packages/google/adk/auth/credential_service/in_memory_credential_service.py:33: UserWarning: [EXPERIMENTAL] BaseCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  super().__init__()
INFO: Started server process [100]
INFO: Waiting for application startup.
```

```
+-----+
| ADK Web Server started |
|                         |
| For local testing, access at http://127.0.0.1:8000. |
+-----+
```

```
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: 35.191.75.49:0 - "GET / HTTP/1.1" 307 Temporary Redirect
INFO: 35.191.18.115:0 - "GET /dev-ui/ HTTP/1.1" 200 OK
INFO: 35.191.87.111:0 - "GET /dev-ui/styles-EWMP9V3U.css HTTP/1.1" 200 OK
INFO: 35.191.114.21:0 - "GET /dev-ui/polyfills-B6TNHQ6-.js HTTP/1.1" 200 OK
INFO: 35.191.114.21:0 - "GET /dev-ui/chunk-20H2EV66.js HTTP/1.1" 200 OK
```

```
!adk web --url_prefix {url_prefix}
```

```
/usr/local/lib/python3.11/dist-packages/google/adk/cli/fast_api.py:130: UserWarning: [EXPERIMENTAL] InMemoryCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  credential_service = InMemoryCredentialService()
/usr/local/lib/python3.11/dist-packages/google/adk/auth/credential_service/in_memory_credential_service.py:33: UserWarning: [EXPERIMENTAL] BaseCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  super().__init__()
INFO: Started server process [100]
INFO: Waiting for application startup.
```

```
+-----+
| ADK Web Server started |
|                         |
| For local testing, access at http://127.0.0.1:8000. |
+-----+
```

```
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: 35.191.75.49:0 - "GET / HTTP/1.1" 307 Temporary Redirect
INFO: 35.191.18.115:0 - "GET /dev-ui/ HTTP/1.1" 200 OK
INFO: 35.191.87.111:0 - "GET /dev-ui/styles-EWMP9V3U.css HTTP/1.1" 200 OK
INFO: 35.191.114.21:0 - "GET /dev-ui/polyfills-B6TNHQ6-.js HTTP/1.1" 200 OK
INFO: 35.191.114.20:0 - "GET /dev-ui/chunk-20H2EV66.js HTTP/1.1" 200 OK
INFO: 35.191.87.110:0 - "GET /dev-ui/main-0520W253.js HTTP/1.1" 200 OK
INFO: 35.191.87.108:0 - "GET /dev-ui/assets/config/runtime-config.json HTTP/1.1" 200 OK
INFO: 35.191.87.110:0 - "GET /dev-ui/adk_favicon.svg HTTP/1.1" 200 OK
INFO: 35.191.18.115:0 - "GET /list-apps?relative_path=/ HTTP/1.1" 200 OK
INFO: 35.191.114.22:0 - "GET /dev-ui/assets/ADM-512-color.svg HTTP/1.1" 200 OK
INFO: 35.191.18.116:0 - "GET /builder/app/sample-agent?ts=1777833292812 HTTP/1.1" 200 OK
INFO: google_adk.google.adk.cli.adk_web_server:New session created: 984a3732-ee70-49f8-9a75-b2759d47eae0
INFO: 35.191.114.23:0 - "POST /apps/sample-agent/users/user/sessions HTTP/1.1" 200 OK
INFO: 35.191.87.111:0 - "GET /apps/sample-agent/users/user/sessions HTTP/1.1" 200 OK
INFO: 35.191.18.113:0 - "GET /apps/sample-agent/users/user/sessions/984a3732-ee70-49f8-9a75-b2759d47eae0 HTTP/1.1" 200 OK
INFO: 35.191.114.21:0 - "GET /apps/sample-agent/eval_sets HTTP/1.1" 200 OK
INFO: 35.191.87.108:0 - "GET /apps/sample-agent/eval_results HTTP/1.1" 200 OK
INFO: 35.191.18.114:0 - "GET /debug/trace/session/984a3732-ee70-49f8-9a75-b2759d47eae0 HTTP/1.1" 200 OK
INFO: 35.191.114.22:0 - "GET /apps/sample-agent/users/user/sessions HTTP/1.1" 200 OK
```

Asignaciones del Día 1

1. Completar la Unidad 1 – "Introducción a los agentes":

- ✓ Escucha el [episodio resumen del podcast](#) de esta unidad.
- ✓ Para complementar el podcast, lee el [whitepaper](#)
- ✓ ["Introducción a los agentes"](#). Completa estos codelabs en Kaggle:
 - ✓ [Construye](#) tu primer agente usando Gemini y ADK.
 - ✓ [Construye](#) tus primeros sistemas multiagente usando ADK.
 - ✓ Asegúrate [de verificar tu cuenta de Kaggle por teléfono](#) antes de empezar, es necesario para los codelabs. También tenemos una guía de [solución de problemas](#) para los codelabs.□
- Asegúrate de comprobar allí si hay soluciones a problemas comunes.
- ✓ ¿Quieres tener una conversación interactiva? Prueba a añadir los whitepapers a NotebookLM.

The screenshot displays the Google AI Assistant interface. On the left, a sidebar titled 'Kit de Desarrollo de Agentes' (Agent Development Kit) is visible, featuring a search bar and a list of agents: 'agente de muestra' (sample agent), 'Trace', 'Eventos', 'Estado', 'Artefactos', and 'Sesiones'. Below this, a section labeled 'Invocaciones' (Invocations) shows a list of prompts: 'hola', '¿cómo funcionan?', and 'que eres capaz de hacer'. The main chat area on the right shows a conversation with the assistant. The user's prompt is '¡Hola! ¿En qué puedo ayudarte hoy?'. The assistant's response is 'Soy un modelo de lenguaje grande, entrenado por Google.' Below this, a detailed list of capabilities is shown, including: 'Responder tus preguntas', 'Generar texto', 'Traducir idiomas', 'Asistir en la escritura', 'Conversar contigo', 'Ayudar con la programación', and 'Divertirme'. The interface is clean and modern, with a dark theme and a clear distinction between user and assistant messages.

DIA 1B

Day 1b - Agent Architectures a0cb4c

Draft guardado

ArchivoEdiciónVistaCorreEscenariosComplementosAyuda

+<⌂📁▶▶▶Ejecutar todoCódigo

Sesión de borrador (38m)

Section 1: Setup

1.1: Install dependencies

The Kaggle Notebooks environment includes a pre-installed version of the [google-adk](#) library for Python and its required dependencies, so you don't need to install additional packages in this notebook.

To install and use ADK in your own Python development environment outside of this course, you can do so by running:

```
pip install google-adk
```

[2]:

```
pip install google-adk
```

```
Requirement already satisfied: google-adk in /usr/local/lib/python3.11/dist-packages (1.18.0)
Requirement already satisfied: PyYAML<7.0.0,>=6.0.2 in /usr/local/lib/python3.11/dist-packages (from google-adk) (6.0.3)
Requirement already satisfied: anyio<5.0.0,>=4.9.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (4.11.0)
Requirement already satisfied: authlib<2.0.0,>=1.5.1 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.6.5)
Requirement already satisfied: click<9.0.0,>=8.1.8 in /usr/local/lib/python3.11/dist-packages (from google-adk) (8.2.0)
Requirement already satisfied: fastapi<1.119.0,>=0.115.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.116.1)
Requirement already satisfied: google-api-python-client<3.0.0,>=2.157.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.177.0)
Requirement already satisfied: google-cloud-aiplatform<2.0.0,>=1.125.0 in /usr/local/lib/python3.11/dist-packages (from google-cloud-aiplatform[agent-engines]<2.0.0,>=1.125.0->google-adk) (1.125.0)
```

Day 1b - Agent Architectures a0cb4cDraft guardado

ArchivoEdiciónVista¡CorreEscenariosComplementosAyuda

+<=>Ejecutar todoDescuento

1.2: Configure your Gemini API Key

This notebook uses the [Gemini API](#), which requires authentication.

1. Get your API key

If you don't have one already, create an [API key](#) in [Google AI Studio](#).

2. Add the key to Kaggle Secrets

Next, you will need to add your API key to your Kaggle Notebook as a Kaggle User Secret.

1. In the top menu bar of the notebook editor, select `Add-ons` then `Secrets`.
2. Create a new secret with the label `GOOGLE_API_KEY`.
3. Paste your API key into the "Value" field and click "Save".
4. Ensure that the checkbox next to `GOOGLE_API_KEY` is selected so that the secret is attached to the notebook.

3. Authenticate in the notebook

Run the cell below to complete authentication.

+ Code+ Markdown

```
[3]:import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Gemini API key setup complete.")
except Exception as e:
    print(
        f"🚨 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}"
    )

✅ Gemini API key setup complete.
```

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

```
[4]:from google.adk.agents import Agent, SequentialAgent, ParallelAgent, LoopAgent
from google.adk.models.google_llm import Gemini
from google.adk.runners import InMemoryRunner
from google.adk.tools import AgentTool, FunctionTool, google_search
from google.genai import types

print("✅ ADK components imported successfully.")

✅ ADK components imported successfully.
```

Day 1b - Agent Architectures a0cb4cDraft guardado

ArchivoEdiciónVista¡CorreEscenariosComplementosAyuda

+<=>Ejecutar todoDescuento

1.4: Configure Retry Options

When working with LLMs, you may encounter transient errors like rate limits or temporary service unavailability. Retry optio

```
[13]:retry_config=types.HttpRetryOptions(
    attempts=5, # Maximum retry attempts
    exp_base=7, # Delay multiplier
    initial_delay=1,
    http_status_codes=[429, 500, 503, 504], # Retry on these HTTP errors
)
```

2.1 Example: Research & Summarization System

Let's build a system with two specialized agents:

1. **Research Agent** - Searches for information using Google Search
2. **Summarizer Agent** - Creates concise summaries from research findings

```
# Research Agent: Its job is to use the google_search tool and present findings.
research_agent = Agent(
    name="ResearchAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    instruction="""You are a specialized research agent. Your only job is to use the
google_search tool to find 2-3 pieces of relevant information on the given topic and present the findings with citations.""",
    tools=[google_search],
    output_key="research_findings", # The result of this agent will be stored in the session state with this key.
)

print("✅ research_agent created.")
```

✅ research_agent created.

+ Code

+ Markdown

```
# Summarizer Agent: Its job is to summarize the text it receives.
summarizer_agent = Agent(
    name="SummarizerAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    # The instruction is modified to request a bulleted list for a clear output format.
    instruction="""Read the provided research findings: {research_findings}
Create a concise summary as a bulleted list with 3-5 key points.""",
    output_key="final_summary",
)

print("✅ summarizer_agent created.")
```

✅ summarizer_agent created.

```
print("✅ summarizer_agent created.")
```

✅ summarizer_agent created.

+ Code

+ Markdown

Refer to the ADK documentation for more information on guiding agents with clear and specific instructions.

Then we bring the agents together under a root agent, or coordinator:

```
# Root Coordinator: Orchestrates the workflow by calling the sub-agents as tools.
root_agent = Agent(
    name="ResearchCoordinator",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    # This instruction tells the root agent HOW to use its tools (which are the other agents).
    instruction="""You are a research coordinator. Your goal is to answer the user's query by orchestrating a workflow.
1. First, you MUST call the 'ResearchAgent' tool to find relevant information on the topic provided by the user.
2. Next, after receiving the research findings, you MUST call the 'SummarizerAgent' tool to create a concise summary.
3. Finally, present the final summary clearly to the user as your response.""",
    # We wrap the sub-agents in 'AgentTool' to make them callable tools for the root agent.
    tools=[AgentTool(research_agent), AgentTool(summarizer_agent)],
)

print("✅ root_agent created.")
```

✅ root_agent created.

Here we're using `AgentTool` to wrap the sub-agents to make them callable tools for the root agent. We'll explore `AgentTool` in-detail on Day 2.

Let's run the agent and ask it about a topic:

```
runner = InMemoryRunner(agent=root_agent)
response = await runner.run_debug(
    "What are the latest advancements in quantum computing and what do they mean for AI?"
)
```

Created new session: debug_session_id

User \ what are the latest advancements in quantum computing and what do they mean for AI?

Section 3: Sequential Workflows - The Assembly Line

The Problem: Unpredictable Order

The previous multi-agent system worked, but it relied on a **detailed instruction prompt** to force the LLM to run steps in order. This can be unreliable. A complex LI

The Solution: A Fixed Pipeline

When you need tasks to happen in a **guaranteed, specific order**, you can use a `SequentialAgent`. This agent acts like an assembly line, running each sub-agent predictable and reliable workflow.

Use Sequential when: Order matters, you need a linear pipeline, or each step builds on the previous one.

To learn more, check out the documentation related to [sequential agents in ADK](#).

Architecture: Blog Post Creation Pipeline



+ Code

+ Markdown

3.1 Example: Blog Post Creation with Sequential Agents

Let's build a system with three specialized agents:

1. **Outline Agent** - Creates a blog outline for a given topic
2. **Writer Agent** - Writes a blog post
3. **Editor Agent** - Edits a blog post draft for clarity and structure

```
# Outline Agent: Creates the initial blog post outline.
outline_agent = Agent(
    name="OutlineAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    instruction="""Create a blog outline for the given topic with:
1. A catchy headline
2. An introduction hook
3. 3-5 main sections with 2-3 bullet points for each
4. A concluding thought""",
    output_key="blog_outline", # The result of this agent will be stored in the session state with this key.
)

print("✅ outline_agent created.")
```

✅ outline_agent created.

```
# Writer Agent: Writes the full blog post based on the outline from the previous agent.
writer_agent = Agent(
    name="WriterAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    # The `{blog_outline}` placeholder automatically injects the state value from the previous agent's output.
    instruction="""Following this outline strictly: {blog_outline}
    Write a brief, 200 to 300-word blog post with an engaging and informative tone.""",
    output_key="blog_draft", # The result of this agent will be stored with this key.
)

print("✅ writer_agent created.")
```

✅ writer_agent created.

```
# Editor Agent: Edits and polishes the draft from the writer agent.
editor_agent = Agent(
    name="EditorAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    # This agent receives the `{blog_draft}` from the writer agent's output.
    instruction="""Edit this draft: {blog_draft}
    Your task is to polish the text by fixing any grammatical errors, improving the flow and sentence structure, and enhancing overall clarity.""",
    output_key="final_blog", # This is the final output of the entire pipeline.
)

print("✅ editor_agent created.")
```

✅ editor_agent created.

Then we bring the agents together under a sequential agent, which runs the agents in the order that they are listed:

```
root_agent = SequentialAgent(
    name="BlogPipeline",
    sub_agents=[outline_agent, writer_agent, editor_agent],
)

print("✅ Sequential Agent created.")
```

✅ Sequential Agent created.

4.1 Example: Parallel Multi-Topic Research

Let's build a system with four agents:

- 1. **Tech Researcher** - Researches AI/ML news and trends
- 2. **Health Researcher** - Researches recent medical news and trends
- 3. **Finance Researcher** - Researches finance and fintech news and trends
- 4. **Aggregator Agent** - Combines all research findings into a single summary

```
# Tech Researcher: Focuses on AI and ML trends.
tech_researcher = Agent(
    name="TechResearcher",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    instruction="""Research the latest AI/ML trends. Include 3 key developments,
the main companies involved, and the potential impact. Keep the report very concise (100 words).""",
    tools=[google_search],
    output_key="tech_research", # The result of this agent will be stored in the session state with this key.
)

print("✅ tech_researcher created.")
```

✅ tech_researcher created.

```
# Health Researcher: Focuses on medical breakthroughs.
health_researcher = Agent(
    name="HealthResearcher",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    instruction="""Research recent medical breakthroughs. Include 3 significant advances,
their practical applications, and estimated timelines. Keep the report concise (100 words).""",
    tools=[google_search],
    output_key="health_research", # The result will be stored with this key.
)

print("✅ health_researcher created.")
```

✅ health_researcher created.

+ Code + Markdown

```
# Finance Researcher: Focuses on fintech trends.
finance_researcher = Agent(
    name="FinanceResearcher",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    instruction="""Research current fintech trends. Include 3 key trends,
their market implications, and the future outlook. Keep the report concise (100 words).""",
    tools=[google_search],
    output_key="finance_research", # The result will be stored with this key.
)

print("✅ finance_researcher created.")
```

✅ finance_researcher created.

```
# The AggregatorAgent runs *after* the parallel step to synthesize the results.
aggregator_agent = Agent(
    name="AggregatorAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    # It uses placeholders to inject the outputs from the parallel agents, which are now in the session state.
    instruction="""Combine these three research findings into a single executive summary:

**Technology Trends:**
{tech_research}

**Health Breakthroughs:**
{health_research}

**Finance Innovations:**
{finance_research}

Your summary should highlight common themes, surprising connections, and the most important key takeaways from all three reports. The final summary should be around 200 words.""",
    output_key="executive_summary", # This will be the final output of the entire system.
)

print("✅ aggregator_agent created.")
```

✅ aggregator_agent created.

```

    **Health Breakthroughs:**
    {health_research}

    **Finance Innovations:**
    {finance_research}

    Your summary should highlight common themes, surprising connections, and the most important key takeaways from all three reports. The final s
    output_key="executive_summary", # This will be the final output of the entire system.
    )

print("✅ aggregator_agent created.")

✅ aggregator_agent created.
```

👉 Then we bring the agents together under a parallel agent, which is itself nested inside of a sequential agent.

This design ensures that the research agents run first in parallel, then once all of their research is complete, the aggregator agent brings together all of the research findings into a single report:

```

# The ParallelAgent runs all its sub-agents simultaneously.
parallel_research_team = ParallelAgent(
    name="ParallelResearchTeam",
    sub_agents=[tech_researcher, health_researcher, finance_researcher],
)

# This SequentialAgent defines the high-level workflow: run the parallel team first, then run the aggregator.
root_agent = SequentialAgent(
    name="ResearchSystem",
    sub_agents=[parallel_research_team, aggregator_agent],
)

print("✅ Parallel and Sequential Agents created.")

✅ Parallel and Sequential Agents created.
```

Let's run the agent and give it a prompt to research the given topics:

+ Code

+ Markdown

```

runner = InMemoryRunner(agent=root_agent)
response = await runner.run_debug(
    "Run the daily executive briefing on Tech, Health, and Finance"
)
```

Created new session: debug_session_id

User > Run the daily executive briefing on Tech, Health, and Finance

5.1 Example: Iterative Story Refinement

Let's build a system with two agents:

1. **Writer Agent** - Writes a draft of a short story
2. **Critic Agent** - Reviews and critiques the short story to suggest improvements

```

# This agent runs ONCE at the beginning to create the first draft.
initial_writer_agent = Agent(
    name="InitialWriterAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    instruction="""Based on the user's prompt, write the first draft of a short story (around 100-150 words).
    Output only the story text, with no introduction or explanation.""",
    output_key="current_story", # Stores the first draft in the state.
)

print("✅ initial_writer_agent created.")

✅ initial_writer_agent created.
```

```

# This agent's only job is to provide feedback or the approval signal. It has no tools.
critic_agent = Agent(
    name="CriticAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    instruction="""You are a constructive story critic. Review the story provided below.
    Story: {current_story}

    Evaluate the story's plot, characters, and pacing.
    - If the story is well-written and complete, you MUST respond with the exact phrase: "APPROVED"
    - Otherwise, provide 2-3 specific, actionable suggestions for improvement.""",
    output_key="critique", # Stores the feedback in the state.
)

print("✅ critic_agent created.")

✅ critic agent created.
```

```
Story Draft: {current_story}
Critique: {critique}

Your task is to analyze the critique.
- IF the critique is EXACTLY "APPROVED", you MUST call the 'exit_loop' function and nothing else.
- OTHERWISE, rewrite the story draft to fully incorporate the feedback from the critique.
output_key="current_story", # It overwrites the story with the new, refined version.
tools=[
    FunctionTool(exit_loop)
], # The tool is now correctly initialized with the function reference.
)

print("✅ refiner_agent created.")
```

✅ refiner_agent created.

Then we bring the agents together under a loop agent, which is itself nested inside of a sequential agent.

This design ensures that the system first produces an initial story draft, then the refinement loop runs up to the specified number of `max_iterations`:

```
# The LoopAgent contains the agents that will run repeatedly: Critic -> Refiner.
story_refinement_loop = LoopAgent(
    name="StoryRefinementLoop",
    sub_agents=[critic_agent, refiner_agent],
    max_iterations=2, # Prevents infinite loops
)

# The root agent is a SequentialAgent that defines the overall workflow: Initial Write -> Refinement Loop.
root_agent = SequentialAgent(
    name="StoryPipeline",
    sub_agents=[initial_writer_agent, story_refinement_loop],
)

print("✅ Loop and Sequential Agents created.")
```

✅ Loop and Sequential Agents created.

+ Code + Markdown

Let's run the agent and give it a topic to write a short story about:

```
runner = InMemoryRunner(agent=root_agent)
response = await runner.run_debug(
    "Write a short story about a lighthouse keeper who discovers a mysterious, glowing map"
)

### Created new session: debug_session_id
```

✅ Congratulations! You're Now an Agent Orchestrator

In this notebook, you made the leap from a single agent to a **multi-agent system**.

You saw **why** a team of specialists is easier to build and debug than one "do-it-all" agent. Most importantly, you learned how to be the **director** of that team.

You used `SequentialAgent`, `ParallelAgent`, and `LoopAgent` to create deterministic workflows, and you even used an LLM as a "manager" to make dynamic decisions. You also mastered the "plumbing" by using `output_key` to pass state between agents and make them collaborative.

Note: No submission required!

This notebook is for your hands-on practice and learning only. You **do not** need to submit it anywhere to complete the course.

Dia 2

Day 2a - Agent Tools d07195

Draft saved

File Edit View Run Settings Add-ons Help

+ - 🔍 📄 📁 ▶ ▶▶ Run All Code -

1.1: Install dependencies

The Kaggle Notebooks environment includes a pre-installed version of the [google-adk](#) library for Python and its required dependencies.

To install and use ADK in your own Python development environment outside of this course, you can do so by running:

```
pip install google-adk
```

+ Code + Markdown

```
pip install google-adk

Requirement already satisfied: google-adk in /usr/local/lib/python3.11/dist-packages (1.18.0)
Requirement already satisfied: pyyaml<7.0.0,>=6.0.2 in /usr/local/lib/python3.11/dist-packages (from google-adk) (6.0.3)
Requirement already satisfied: anyio<5.0.0,>=4.9.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (4.11.0)
Requirement already satisfied: authlib<2.0.0,>=1.5.1 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.6.5)
Requirement already satisfied: click<9.0.0,>=8.1.8 in /usr/local/lib/python3.11/dist-packages (from google-adk) (8.3.0)
Requirement already satisfied: fastapi<1.119.0,>=0.115.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.116.1)
Requirement already satisfied: google-api-python-client<3.0.0,>=2.157.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.177.0)
Requirement already satisfied: google-cloud-aiplatform<2.0.0,>=1.125.0 in /usr/local/lib/python3.11/dist-packages (from google-cloud-aiplatform[agent-engines]<2.0.0,>=1.125.0->google-adk) (1.125.0)
Requirement already satisfied: google-cloud-bigtable>=2.32.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.34.0)
Requirement already satisfied: google-cloud-discoveryengine<0.14.0,>=0.13.12 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.13.12)
Requirement already satisfied: google-cloud-secret-manager<3.0.0,>=2.22.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.25.0)
Requirement already satisfied: google-cloud-spanner<4.0.0,>=3.56.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (3.56.0)
Requirement already satisfied: google-cloud-speech<3.0.0,>=2.30.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.34.0)
```

Day 2a - Agent Tools d07195

Draft saved

File Edit View Run Settings Add-ons Help

+ - 🔍 📄 📁 ▶ ▶▶ Run All Code -

Successfully installed cachetools-5.5.2 protobuf-5.29.6
Note: you may need to restart the kernel to use updated packages.

+ Code + Markdown

1.2: Configure your Gemini API Key

This notebook uses the [Gemini API](#), which requires an API key.

1. Get your API key

If you don't have one already, create an API key in [Google AI Studio](#).

2. Add the key to Kaggle Secrets

Next, you will need to add your API key to your Kaggle Notebook as a Kaggle User Secret.

1. In the top menu bar of the notebook editor, select **Add-ons** then **Secrets**.
2. Create a new secret with the label `GOOGLE_API_KEY`.
3. Paste your API key into the "Value" field and click "Save".
4. Ensure that the checkbox next to `GOOGLE_API_KEY` is selected so that the secret is attached to the notebook.

3. Authenticate in the notebook

Run the cell below to access the `GOOGLE_API_KEY` you just saved and set it as an environment variable for the notebook to use:

```
[1]: import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Setup and authentication complete.")
except Exception as e:
    print(
        f"🚨 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}"
    )
```

```
[2]: import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Setup and authentication complete.")
except Exception as e:
    print(
        f"🚨 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}"
    )

✅ Setup and authentication complete.
```

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

```
6 from google.genai import types

from google.adk.agents import LLaAgent
from google.adk.models.google_llm import Gemini
from google.adk.runners import InMemoryRunner
from google.adk.sessions import InMemorySessionService
from google.adk.tools import google_search, AgentTool, ToolContext
from google.adk.code_executors import BuiltInCodeExecutor

print("✅ ADK components imported successfully.")
```

+ Code + Markdown

```
7 from google.genai import types

from google.adk.agents import LLaAgent
from google.adk.models.google_llm import Gemini
from google.adk.runners import InMemoryRunner
from google.adk.sessions import InMemorySessionService
from google.adk.tools import google_search, AgentTool, ToolContext
from google.adk.code_executors import BuiltInCodeExecutor

print("✅ ADK components imported successfully.")
```

✅ ADK components imported successfully.

1.4: Helper functions

Helper function that prints the generated Python code and results from the code execution tool:

+ Code + Markdown

```
def show_python_code_and_result(response):
    for i in range(len(response)):
        # Check if the response contains a valid function call result from the code executor
        if (
            (response[i].content.parts)
            and (response[i].content.parts[0])
            and (response[i].content.parts[0].function_response)
            and (response[i].content.parts[0].function_response.response)
        ):
            response_code = response[i].content.parts[0].function_response.response
            if "result" in response_code and response_code["result"] != "":
                if "tool_code" in response_code["result"]:
                    print(
                        "Generated Python Code >> ",
                        response_code["result"].replace("tool_code", ""),
                    )
                else:
                    print("Generated Python Response >> ", response_code["result"])

print("✅ Helper functions defined.")
```

```
def show_python_code_and_result(response):
    for i in range(len(response)):
        # Check if the response contains a valid function call result from the code executor
        if (
            (response[i].content.parts)
            and (response[i].content.parts[0])
            and (response[i].content.parts[0].function_response)
            and (response[i].content.parts[0].function_response.response)
        ):
            response_code = response[i].content.parts[0].function_response.response
            if "result" in response_code and response_code["result"] != "":
                if "tool_code" in response_code["result"]:
                    print(
                        "Generated Python Code >> ",
                        response_code["result"].replace("tool_code", ""),
                    )
                else:
                    print("Generated Python Response >> ", response_code["result"])

print("✅ Helper functions defined.")
```

✅ Helper functions defined.

Day 2a - Agent Tools b8269d Draft guardado

Archivo Edición Vista Corre Escenarios Complementos Ayuda

+ - 🔗 📄 📁 ▶️ ⏏ Cancelar Run Código -

🤖 2.2: How to define a Tool? 1

Any Python function can become an agent tool by following these simple guidelines:

- 1. Create a Python function
- 2. Follow the best practices listed below
- 3. Add your function to the agent's `tools=[]` list and ADK handles the rest automatically.

🔧 ADK Best Practices in Action

Notice how our tools follow ADK best practices:

- 1. **Dictionary Returns:** Tools return `{ "status": "success", "data": ... }` or `{ "status": "error", "error_message": ... }`
- 2. **Clear Docstrings:** LLMs use docstrings to understand when and how to use tools
- 3. **Type Hints:** Enable ADK to generate proper schemas (`str`, `dict`, etc.)
- 4. **Error Handling:** Structured error responses help LLMs handle failures gracefully

These patterns make your tools reliable and easy for LLMs to use correctly.

👉 Let's see this in action with our first tool:

+ Code + Markdown

```
# Pay attention to the docstring, type hints, and return value.
def get_fee_for_payment_method(method: str) -> dict:
    """Looks up the transaction fee percentage for a given payment method.

    This tool simulates looking up a company's internal fee structure based on
    the name of the payment method provided by the user.

    Args:
        method: The name of the payment method. It should be descriptive,
            e.g., 'platinum credit card' or 'bank transfer'.

    Returns:
        Dictionary with status and fee information.
        Success: {'status': 'success', 'fee_percentage': 0.02}
        Error: {'status': 'error', 'error_message': 'Payment method not found'}
    """
    # This simulates looking up a company's internal fee structure.
    fee_database = {
        "platinum credit card": 0.02, # 2%
        "gold debit card": 0.035, # 3.5%
        "bank transfer": 0.01, # 1%
    }

    fee = fee_database.get(method.lower())
    if fee is not None:
        return {"status": "success", "fee_percentage": fee}
    else:
        return {
            "status": "error",
            "error_message": f"Payment method '{method}' not found",
        }

print("✅ Fee lookup function created")
print(f"🧪 Test: {get_fee_for_payment_method('platinum credit card')}")

# Fee lookup function created
# Test: {'status': 'success', 'fee_percentage': 0.02}
```

+

▼



Ejecutar todo

Código ▼

⇒

```
def get_exchange_rate(base_currency: str, target_currency: str) -> dict:
    """Looks up and returns the exchange rate between two currencies.

    Args:
        base_currency: The ISO 4217 currency code of the currency you
            are converting from (e.g., "USD").
        target_currency: The ISO 4217 currency code of the currency you
            are converting to (e.g., "EUR").

    Returns:
        Dictionary with status and rate information.
        Success: {'status': "success", "rate": 0.93}
        Error: {'status': "error", "error_message": "Unsupported currency pair"}
    """

    # Static data simulating a live exchange rate API
    # In production, this would call something like: requests.get("api.exchangerates.com")
    rate_database = {
        "usd": {
            "eur": 0.93, # Euro
            "jpy": 157.50, # Japanese Yen
            "inr": 83.58, # Indian Rupee
        }
    }

    # Input validation and processing
    base = base_currency.lower()
    target = target_currency.lower()

    # Return structured result with status
    rate = rate_database.get(base, {}).get(target)
    if rate is not None:
        return {"status": "success", "rate": rate}
    else:
        return {
            "status": "error",
            "error_message": f"Unsupported currency pair: {base_currency}/{target_currency}",
        }

print("✅ Exchange rate function created")
print(f"🧪 Test: {get_exchange_rate('USD', 'EUR')}")
```

✅ Exchange rate function created

🧪 Test: {'status': 'success', 'rate': 0.93}

Day 2a - Agent Tools b8269d

Draft guardado

ArchivoEdiciónVistaíCorreEscenariosComplementosAyuda

+

▼



Ejecutar todo

Código ▼

⇒

```
# Currency agent with custom function tools
currency_agent = LLMAgent(
    name="currency_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    instruction="""You are a smart currency conversion assistant.

    For currency conversion requests:
    1. Use 'get_fee_for_payment_method()' to find transaction fees
    2. Use 'get_exchange_rate()' to get currency conversion rates
    3. Check the "status" field in each tool's response for errors
    4. Calculate the final amount after fees based on the output from 'get_fee_for_payment_method' and 'get_exchange_rate' methods and provide a clear breakdown
    5. First, state the final converted amount.
       Then, explain how you got that result by showing the intermediate amounts. Your explanation must include: the fee percentage and its
       value in the original currency, the amount remaining after the fee, and the exchange rate used for the final conversion.

    If any tool returns status "error", explain the issue to the user clearly.
    """
    tools=[get_fee_for_payment_method, get_exchange_rate],
)

print("✅ Currency agent created with custom function tools")
print("\n Available tools:")
print(" • get_fee_for_payment_method - Looks up company fee structure")
print(" • get_exchange_rate - Gets current exchange rates")
```

✅ Currency agent created with custom function tools

🔍 Available tools:

• get_fee_for_payment_method - Looks up company fee structure

• get_exchange_rate - Gets current exchange rates

+ Code

+ Markdown

⇒

```
# Test the currency agent
currency_runner = InMemoryRunner(agent=currency_agent)
_ = await currency_runner.run_debug(
    "I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?"
)

### Created new session: debug_session_id

User > I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?
```

Day 2a - Agent Tools

File Edit View Run Settings Add-ons Help

+ - ✂ 📄 📁 ▶ ▶▶ Run All Code - Draft Session (2m) 003 004 005 🔌 ↺ ⋮

- get_exchange_rate - Gets current exchange rates

▶ # Test the currency agent
currency_runner = InMemoryRunner(agent=currency_agent)
_ = await currency_runner.run_debug(
 "I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?"
)

Created new session: debug_session_id

User > I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?

WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call', 'function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call', 'function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

currency_agent > You will receive 455.70 Euros.

Here's how that's calculated:

1. The transaction fee for using a Platinum Credit Card is 2.00% (0.02).
2. The fee amount is 10.00 USD (500 USD * 0.02).
3. The amount remaining after the fee is 490.00 USD (500 USD - 10.00 USD).
4. The exchange rate from USD to EUR is 0.93.
5. The final amount in EUR is 455.70 EUR (490.00 USD * 0.93).

+ Code + Markdown

Day 2a - Agent Tools b8269d

Draft guardado

Archivo Edición Vista Corre Escenarios Complementos Ayuda

+ - ✂ 📄 📁 ▶ ▶▶ Ejecutar todo Descuento -

Let's see this in action:

+ Code + Markdown

```
[17]: enhanced_currency_agent = LLMAgent(  
    name="enhanced_currency_agent",  
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),  
    # Updated instruction  
    instructions="You are a smart currency conversion assistant. You must strictly follow these steps and use the available tools.  
  
    For any currency conversion request:  
  
    1. Get Transaction Fee: Use the get_fee_for_payment_method() tool to determine the transaction fee.  
    2. Get Exchange Rate: Use the get_exchange_rate() tool to get the currency conversion rate.  
    3. Error Check: After each tool call, you must check the 'status' field in the response. If the status is 'error', you must stop.  
    4. Calculate Final Amount (CRITICAL): You are strictly prohibited from performing any arithmetic calculations yourself. You must code will use the fee information from step 1 and the exchange rate from step 2.  
    5. Provide Detailed Breakdown: In your summary, you must:  
        * State the final converted amount.  
        * Explain how the result was calculated, including:  
            * The fee percentage and the fee amount in the original currency.  
            * The amount remaining after deducting the fee.  
            * The exchange rate applied.  
    ...  
    tools=[  
        get_fee_for_payment_method,  
        get_exchange_rate,  
        AgentTool(agent=calculation_agent), # Using another agent as a tool!  
    ],  
)  
  
print("✅ Enhanced currency agent created")  
print("🔗 New capability: Delegates calculations to specialist agent")  
print("🔧 Tool types used:")  
print("  • Function Tools (fee, rates)")  
print("  • Agent Tool (calculation specialist)")
```

✅ Enhanced currency agent created
🔗 New capability: Delegates calculations to specialist agent
🔧 Tool types used:
• Function Tools (fee, rates)
• Agent Tool (calculation specialist)

▶ # Test the currency agent
currency_runner = InMemoryRunner(agent=currency_agent)
_ = await currency_runner.run_debug(
 "I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?"
)

Created new session: debug_session_id

User > I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?

WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call', 'function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call', 'function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

currency_agent > You will receive 455.70 Euros.

Here's how that's calculated:

1. The transaction fee for using a Platinum Credit Card is 2.00% (0.02).
2. The fee amount is 10.00 USD (500 USD * 0.02).
3. The amount remaining after the fee is 490.00 USD (500 USD - 10.00 USD).
4. The exchange rate from USD to EUR is 0.93.

[illegible][illegible]

Day 2b - Agent Tools Best Practices

Standard MCP Protocol

```
graph TD
    A[Standard MCP Protocol] --> B[GitHub Server]
    A --> C[Slack MCP]
    A --> D[Maps MCP]
    A --> E[...]
```

2.2: Using MCP with Your Agent

The workflow is simple:

1. Choose an MCP Server and tool
2. Create the MCP Toolset (configure connection)
3. Add it to your agent
4. Run and test the agent

Step 1: Choose MCP Server

For this demo, we'll use the **Everything MCP Server** - an open package (`@models/contextprotocol/server-everything`) designed for testing MCP integrations.

It provides a `getTinyImage` tool that returns a simple test image (16x16 pixels, base64-encoded). **Find more servers:** [models/contextprotocol/servers/examples](#)

NOTE: This is a demo server to learn MCP. In production, you'll use servers for Google Maps, Slack, Discord, etc.

Step 2: Create the MCP Toolset

The `McpToolset` is used to integrate an SDK Agent with an MCP Server.

What the code does:

- Uses `npm` (Node package runner) to run the MCP server
- Connects to `@models/contextprotocol/server-everything`
- Filters to only use the `getTinyImage` tool (the server has others, but we only need this one)

```
[1] # MCP integration with Everything Server
mcp_image_server = McpToolset(
    connection_params=StdioConnectionParams(
        server_params=StdioServerParameters(
            command="npm", # Run MCP server via npm
            args=[
                "-y", # Argument for npm to auto-confirm install
                "models/contextprotocol/server-everything",
            ],
            tool_filter="getTinyImage",
        ),
        timeout=30,
    )
)

print("MCP Tool created")
```

MCP tool created

Compartir Guardar la versión

Fragmento de código

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret('GOOGLE_API_KEY')
```

Copiar a la carpeta

GOOGLE_API_KEY

+ Añadir secreto Fragmento de copia

Day 2b - Agent Tools Best Practices

Step 3: Add MCP tool to agent

Let's add the `mcp_image_server` to the agent's tool array and update the agent's instructions to handle requests to generate tiny images.

```
[21] # Create image agent with MCP integration
image_agent = LlmAgent(
    model=Gemini(model='gemini-1.5-flash-lite', retry_options=retry_config),
    name='image_agent',
    instructions='Use the MCP tool to generate images for user queries',
    tools=[mcp_image_server],
)
```

```
[22] # Create image agent with MCP integration
image_agent = LlmAgent(
    model=Gemini(model='gemini-1.5-flash-lite', retry_options=retry_config),
    name='image_agent',
    instructions='Use the MCP tool to generate images for user queries',
    tools=[mcp_image_server],
)
```

Create the runner:

```
[23] from google.adx.runners import InMemoryRunner
runner = InMemoryRunner(agent=image_agent)
```

```
[24] from google.adx.runners import InMemoryRunner
runner = InMemoryRunner(agent=image_agent)
```

Step 4: Test the agent

Ask the agent to generate an image. Watch it use the MCP tool:

```
[25] response = await runner.run_async('Provide a sample tiny image', verbose=True)
```

Created new session: debug_session_id

User > Provide a sample tiny image

WARNING:google.genai.types.lmdspkg: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

image_agent > (Calling tool: get-tiny-image())

image_agent > [Tool result: {'content': [{'type': 'text', 'text': 'Here's the image you requested!'}, {'type': 'image', 'data': ...}]]

image_agent > Here's the image you requested:

Compartir Guardar la versión

Fragmento de código

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret('GOOGLE_API_KEY')
```

Copiar a la carpeta

GOOGLE_API_KEY

+ Añadir secreto Fragmento de copia

Step 4: Test the agent

Ask the agent to generate an image. Watch it use the MCP tool:

```
[28]: response = await runner.run_debug("Provide a sample tiny image", verbose=True)

## Created new session: debug_session_id

User > Provide a sample tiny image
WARNING:google_genai.types:Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
image_agent > [Calling tool: get-tiny-image({})]
image_agent > [Tool result: {'content': [{'type': 'text', 'text': "Here's the image you requested:"}, {'type': 'image', 'data': ...}]
image_agent > Here's the image you requested:
The image above is the MCP logo.
```

Display the image:

The server returns base64-encoded image data. Let's decode and display it:

+ Code + Markdown

```
[55]: from IPython.display import display, Image as IPImage
import base64

for event in response:
    if event.content and event.content.parts:
        for part in event.content.parts:
            if hasattr(part, "function_response") and part.function_response:
                for item in part.function_response.response.get("content", []):
                    if item.get("type") == "image":
                        display(IPImage(data=base64.b64decode(item["data"])))
```



Day 2b - Agent Tools Best Practices

Draft guardado

Archivo Edición Vista Corre Escenarios Complementos Ayuda

+ - ✂ 📄 ▶ 🛑 Cancelar Run Código -

● Sesión de borrador Carrera3 (re) 1/2 1/3 1/4 1/5 1/6 1/7 1/8 1/9 1/10 1/11 1/12 1/13 1/14 1/15 1/16 1/17 1/18 1/19 1/20 1/21 1/22 1/23 1/24 1/25 1/26 1/27 1/28 1/29 1/30 1/31 1/32 1/33 1/34 1/35 1/36 1/37 1/38 1/39 1/40 1/41 1/42 1/43 1/44 1/45 1/46 1/47 1/48 1/49 1/50 1/51 1/52 1/53 1/54 1/55 1/56 1/57 1/58 1/59 1/60 1/61 1/62 1/63 1/64 1/65 1/66 1/67 1/68 1/69 1/70 1/71 1/72 1/73 1/74 1/75 1/76 1/77 1/78 1/79 1/80 1/81 1/82 1/83 1/84 1/85 1/86 1/87 1/88 1/89 1/90 1/91 1/92 1/93 1/94 1/95 1/96 1/97 1/98 1/99 1/100 1/101 1/102 1/103 1/104 1/105 1/106 1/107 1/108 1/109 1/110 1/111 1/112 1/113 1/114 1/115 1/116 1/117 1/118 1/119 1/120 1/121 1/122 1/123 1/124 1/125 1/126 1/127 1/128 1/129 1/130 1/131 1/132 1/133 1/134 1/135 1/136 1/137 1/138 1/139 1/140 1/141 1/142 1/143 1/144 1/145 1/146 1/147 1/148 1/149 1/150 1/151 1/152 1/153 1/154 1/155 1/156 1/157 1/158 1/159 1/160 1/161 1/162 1/163 1/164 1/165 1/166 1/167 1/168 1/169 1/170 1/171 1/172 1/173 1/174 1/175 1/176 1/177 1/178 1/179 1/180 1/181 1/182 1/183 1/184 1/185 1/186 1/187 1/188 1/189 1/190 1/191 1/192 1/193 1/194 1/195 1/196 1/197 1/198 1/199 1/200 1/201 1/202 1/203 1/204 1/205 1/206 1/207 1/208 1/209 1/210 1/211 1/212 1/213 1/214 1/215 1/216 1/217 1/218 1/219 1/220 1/221 1/222 1/223 1/224 1/225 1/226 1/227 1/228 1/229 1/230 1/231 1/232 1/233 1/234 1/235 1/236 1/237 1/238 1/239 1/240 1/241 1/242 1/243 1/244 1/245 1/246 1/247 1/248 1/249 1/250 1/251 1/252 1/253 1/254 1/255 1/256 1/257 1/258 1/259 1/260 1/261 1/262 1/263 1/264 1/265 1/266 1/267 1/268 1/269 1/270 1/271 1/272 1/273 1/274 1/275 1/276 1/277 1/278 1/279 1/280 1/281 1/282 1/283 1/284 1/285 1/286 1/287 1/288 1/289 1/290 1/291 1/292 1/293 1/294 1/295 1/296 1/297 1/298 1/299 1/300 1/301 1/302 1/303 1/304 1/305 1/306 1/307 1/308 1/309 1/310 1/311 1/312 1/313 1/314 1/315 1/316 1/317 1/318 1/319 1/320 1/321 1/322 1/323 1/324 1/325 1/326 1/327 1/328 1/329 1/330 1/331 1/332 1/333 1/334 1/335 1/336 1/337 1/338 1/339 1/340 1/341 1/342 1/343 1/344 1/345 1/346 1/347 1/348 1/349 1/350 1/351 1/352 1/353 1/354 1/355 1/356 1/357 1/358 1/359 1/360 1/361 1/362 1/363 1/364 1/365 1/366 1/367 1/368 1/369 1/370 1/371 1/372 1/373 1/374 1/375 1/376 1/377 1/378 1/379 1/380 1/381 1/382 1/383 1/384 1/385 1/386 1/387 1/388 1/389 1/390 1/391 1/392 1/393 1/394 1/395 1/396 1/397 1/398 1/399 1/400 1/401 1/402 1/403 1/404 1/405 1/406 1/407 1/408 1/409 1/410 1/411 1/412 1/413 1/414 1/415 1/416 1/417 1/418 1/419 1/420 1/421 1/422 1/423 1/424 1/425 1/426 1/427 1/428 1/429 1/430 1/431 1/432 1/433 1/434 1/435 1/436 1/437 1/438 1/439 1/440 1/441 1/442 1/443 1/444 1/445 1/446 1/447 1/448 1/449 1/450 1/451 1/452 1/453 1/454 1/455 1/456 1/457 1/458 1/459 1/460 1/461 1/462 1/463 1/464 1/465 1/466 1/467 1/468 1/469 1/470 1/471 1/472 1/473 1/474 1/475 1/476 1/477 1/478 1/479 1/480 1/481 1/482 1/483 1/484 1/485 1/486 1/487 1/488 1/489 1/490 1/491 1/492 1/493 1/494 1/495 1/496 1/497 1/498 1/499 1/500 1/501 1/502 1/503 1/504 1/505 1/506 1/507 1/508 1/509 1/510 1/511 1/512 1/513 1/514 1/515 1/516 1/517 1/518 1/519 1/520 1/521 1/522 1/523 1/524 1/525 1/526 1/527 1/528 1/529 1/530 1/531 1/532 1/533 1/534 1/535 1/536 1/537 1/538 1/539 1/540 1/541 1/542 1/543 1/544 1/545 1/546 1/547 1/548 1/549 1/550 1/551 1/552 1/553 1/554 1/555 1/556 1/557 1/558 1/559 1/560 1/561 1/562 1/563 1/564 1/565 1/566 1/567 1/568 1/569 1/570 1/571 1/572 1/573 1/574 1/575 1/576 1/577 1/578 1/579 1/580 1/581 1/582 1/583 1/584 1/585 1/586 1/587 1/588 1/589 1/590 1/591 1/592 1/593 1/594 1/595 1/596 1/597 1/598 1/599 1/600 1/601 1/602 1/603 1/604 1/605 1/606 1/607 1/608 1/609 1/610 1/611 1/612 1/613 1/614 1/615 1/616 1/617 1/618 1/619 1/620 1/621 1/622 1/623 1/624 1/625 1/626 1/627 1/628 1/629 1/630 1/631 1/632 1/633 1/634 1/635 1/636 1/637 1/638 1/639 1/640 1/641 1/642 1/643 1/644 1/645 1/646 1/647 1/648 1/649 1/650 1/651 1/652 1/653 1/654 1/655 1/656 1/657 1/658 1/659 1/660 1/661 1/662 1/663 1/664 1/665 1/666 1/667 1/668 1/669 1/670 1/671 1/672 1/673 1/674 1/675 1/676 1/677 1/678 1/679 1/680 1/681 1/682 1/683 1/684 1/685 1/686 1/687 1/688 1/689 1/690 1/691 1/692 1/693 1/694 1/695 1/696 1/697 1/698 1/699 1/700 1/701 1/702 1/703 1/704 1/705 1/706 1/707 1/708 1/709 1/710 1/711 1/712 1/713 1/714 1/715 1/716 1/717 1/718 1/719 1/720 1/721 1/722 1/723 1/724 1/725 1/726 1/727 1/728 1/729 1/730 1/731 1/732 1/733 1/734 1/735 1/736 1/737 1/738 1/739 1/740 1/741 1/742 1/743 1/744 1/745 1/746 1/747 1/748 1/749 1/750 1/751 1/752 1/753 1/754 1/755 1/756 1/757 1/758 1/759 1/760 1/761 1/762 1/763 1/764 1/765 1/766 1/767 1/768 1/769 1/770 1/771 1/772 1/773 1/774 1/775 1/776 1/777 1/778 1/779 1/780 1/781 1/782 1/783 1/784 1/785 1/786 1/787 1/788 1/789 1/790 1/791 1/792 1/793 1/794 1/795 1/796 1/797 1/798 1/799 1/800 1/801 1/802 1/803 1/804 1/805 1/806 1/807 1/808 1/809 1/810 1/811 1/812 1/813 1/814 1/815 1/816 1/817 1/818 1/819 1/820 1/821 1/822 1/823 1/824 1/825 1/826 1/827 1/828 1/829 1/830 1/831 1/832 1/833 1/834 1/835 1/836 1/837 1/838 1/839 1/840 1/841 1/842 1/843 1/844 1/845 1/846 1/847 1/848 1/849 1/850 1/851 1/852 1/853 1/854 1/855 1/856 1/857 1/858 1/859 1/860 1/861 1/862 1/863 1/864 1/865 1/866 1/867 1/868 1/869 1/870 1/871 1/872 1/873 1/874 1/875 1/876 1/877 1/878 1/879 1/880 1/881 1/882 1/883 1/884 1/885 1/886 1/887 1/888 1/889 1/890 1/891 1/892 1/893 1/894 1/895 1/896 1/897 1/898 1/899 1/900 1/901 1/902 1/903 1/904 1/905 1/906 1/907 1/908 1/909 1/910 1/911 1/912 1/913 1/914 1/915 1/916 1/917 1/918 1/919 1/920 1/921 1/922 1/923 1/924 1/925 1/926 1/927 1/928 1/929 1/930 1/931 1/932 1/933 1/934 1/935 1/936 1/937 1/938 1/939 1/940 1/941 1/942 1/943 1/944 1/945 1/946 1/947 1/948 1/949 1/950 1/951 1/952 1/953 1/954 1/955 1/956 1/957 1/958 1/959 1/960 1/961 1/962 1/963 1/964 1/965 1/966 1/967 1/968 1/969 1/970 1/971 1/972 1/973 1/974 1/975 1/976 1/977 1/978 1/979 1/980 1/981 1/982 1/983 1/984 1/985 1/986 1/987 1/988 1/989 1/990 1/991 1/992 1/993 1/994 1/995 1/996 1/997 1/998 1/999 1/1000 1/1001 1/1002 1/1003 1/1004 1/1005 1/1006 1/1007 1/1008 1/1009 1/1010 1/1011 1/1012 1/1013 1/1014 1/1015 1/1016 1/1017 1/1018 1/1019 1/1020 1/1021 1/1022 1/1023 1/1024 1/1025 1/1026 1/1027 1/1028 1/1029 1/1030 1/1031 1/1032 1/1033 1/1034 1/1035 1/1036 1/1037 1/1038 1/1039 1/1040 1/1041 1/1042 1/1043 1/1044 1/1045 1/1046 1/1047 1/1048 1/1049 1/1050 1/1051 1/1052 1/1053 1/1054 1/1055 1/1056 1/1057 1/1058 1/1059 1/1060 1/1061 1/1062 1/1063 1/1064 1/1065 1/1066 1/1067 1/1068 1/1069 1/1070 1/1071 1/1072 1/1073 1/1074 1/1075 1/1076 1/1077 1/1078 1/1079 1/1080 1/1081 1/1082 1/1083 1/1084 1/1085 1/1086 1/1087 1/1088 1/1089 1/1090 1/1091 1/1092 1/1093 1/1094 1/1095 1/1096 1/1097 1/1098 1/1099 1/1100 1/1101 1/1102 1/1103 1/1104 1/1105 1/1106 1/1107 1/1108 1/1109 1/1110 1/1111 1/1112 1/1113 1/1114 1/1115 1/1116 1/1117 1/1118 1/1119 1/1120 1/1121 1/1122 1/1123 1/1124 1/1125 1/1126 1/1127 1/1128 1/1129 1/1130 1/1131 1/1132 1/1133 1/1134 1/1135 1/1136 1/1137 1/1138 1/1139 1/1140 1/1141 1/1142 1/1143 1/1144 1/1145 1/1146 1/1147 1/1148 1/1149 1/1150 1/1151 1/1152 1/1153 1/1154 1/1155 1/1156 1/1157 1/1158 1/1159 1/1160 1/1161 1/1162 1/1163 1/1164 1/1165 1/1166 1/1167 1/1168 1/1169 1/1170 1/1171 1/1172 1/1173 1/1174 1/1175 1/1176 1/1177 1/1178 1/1179 1/1180 1/1181 1/1182 1/1183 1/1184 1/1185 1/1186 1/1187 1/1188 1/1189 1/1190 1/1191 1/1192 1/1193 1/1194 1/1195 1/1196 1/1197 1/1198 1/1199 1/1200 1/1201 1/1202 1/1203 1/1204 1/1205 1/1206 1/1207 1/1208 1/1209 1/1210 1/1211 1/1212 1/1213 1/1214 1/1215 1/1216 1/1217 1/1218 1/1219 1/1220 1/1221 1/1222 1/1223 1/1224 1/1225 1/1226 1/1227 1/1228 1/1229 1/1230 1/1231 1/1232 1/1233 1/1234 1/1235 1/1236 1/1237 1/1238 1/1239 1/1240 1/1241 1/1242 1/1243 1/1244 1/1245 1/1246 1/1247 1/1248 1/1249 1/1250 1/1251 1/1252 1/1253 1/1254 1/1255 1/1256 1/1257 1/1258 1/1259 1/1260 1/1261 1/1262 1/1263 1/1264 1/1265 1/1266 1/1267 1/1268 1/1269 1/1270 1/1271 1/1272 1/1273 1/1274 1/1275 1/1276 1/1277 1/1278 1/1279 1/1280 1/1281 1/1282 1/1283 1/1284 1/1285 1/1286 1/1287 1/1288 1/1289 1/1290 1/1291 1/1292 1/1293 1/1294 1/1295 1/1296 1/1297 1/1298 1/1299 1/1300 1/1301 1/1302 1/1303 1/1304 1/1305 1/1306 1/1307 1/1308 1/1309 1/1310 1/1311 1/1312 1/1313 1/1314 1/1315 1/1316 1/1317 1/1318 1/1319 1/1320 1/1321 1/1322 1/1323 1/1324 1/1325 1/1326 1/1327 1/1328 1/1329 1/1330 1/1331 1/1332 1/1333 1/1334 1/1335 1/1336 1/1337 1/1338 1/1339 1/1340 1/1341 1/1342 1/1343 1/1344 1/1345 1/1346 1/1347 1/1348 1/1349 1/1350 1/1351 1/1352 1/1353 1/1354 1/1355 1/1356 1/1357 1/1358 1/1359 1/1360 1/1361 1/1362 1/1363 1/1364 1/1365 1/1366 1/1367 1/1368 1/1369 1/1370 1/1371 1/1372 1/1373 1/1374 1/1375 1/1376 1/1377 1/1378 1/1379 1/1380 1/1381 1/1382 1/1383 1/1384 1/1385 1/1386 1/1387 1/1388 1/1389 1/1390 1/1391 1/1392 1/1393 1/1394 1/1395 1/1396 1/1397 1/1398 1/1399 1/1400 1/1401 1/1402 1/1403 1/1404 1/1405 1/1406 1/1407 1/1408 1/1409 1/1410 1/1411 1/1412 1/1413 1/1414 1/1415 1/1416 1/1417 1/1418 1/1419 1/1420 1/1421 1/1422 1/1423 1/1424 1/1425 1/1426 1/1427 1/1428 1/1429 1/1430 1/1431 1/1432 1/1433 1/1434 1/1435 1/1436 1/1437 1/1438 1/1439 1/1440 1/1441 1/1442 1/1443 1/1444 1/1445 1/1446 1/1447 1/1448 1/1449 1/1450 1/1451 1/1452 1/1453 1/1454 1/1455 1/1456 1/1457 1/1458 1/1459 1/1460 1/1461 1/1462 1/1463 1/1464 1/1465 1/1466 1/1467 1/1468 1/1469 1/1470 1/1471 1/1472 1/1473 1/1474 1/1475 1/1476 1/1477 1/1478 1/1479 1/1480 1/1481 1/1482 1/1483 1/1484 1/1485 1/1486 1/1487 1/1488 1/1489 1/1490 1/1491 1/1492 1/1493 1/1494 1/1495 1/1496 1/1497 1/1498 1/1499 1/1500 1/1501 1/1502 1/1503 1/1504 1/1505 1/1506 1/1507 1/1508 1/1509 1/1510 1/1511 1/1512 1/1513 1/1514 1/1515 1/1516 1/1517 1/1518 1/1519 1/1520 1/1521 1/1522 1/1523 1/1524 1/1525 1/1526 1/1527 1/1528 1/1529 1/1530 1/1531 1/1532 1/1533 1/1534 1/1535 1/1536 1/1537 1/1538 1/1539 1/1540 1/1541 1/1542 1/1543 1/1544 1/1545 1/1546 1/1547 1/1548 1/1549 1/1550 1/1551 1/1552 1/1553 1/1554 1/1555 1/1556 1/1557 1/1558 1/1559 1/1560 1/1561 1/1562 1/1563 1/1564 1/1565 1/1566 1/1567 1/1568 1/1569 1/1570 1/1571 1/1572 1/1573 1/1574 1/1575 1/1576 1/1577 1/1578 1/1579 1/1580 1/1581 1/1582 1/1583 1/1584 1/1585 1/1586 1/1587 1/1588 1/1589 1/1590 1/1591 1/1592 1/1593 1/1594 1/1595 1/1596 1/1597 1/1598 1/1599 1/1600 1/1601 1/1602 1/1603 1/1604 1/1605 1/1606 1/1607 1/1608 1/1609 1/1610 1/1611 1/1612 1/1613 1/1614 1/1615 1/1616 1/1617 1/1618 1/1619 1/1620 1/1621 1/1622 1/1623 1/1624 1/1625 1/1626 1/1627 1/1628 1/1629 1/1630 1/1631 1/1632 1/1633 1/1634 1/1635 1/1636 1/1637 1/1638 1/1639 1/1640 1/1641 1/1642 1/1643 1/1644 1/1645 1/1646 1/1647 1/1648 1/1649 1/1650 1/1651 1/1652 1/1653 1/1654 1/1655 1/1656 1/1657 1/1658 1/1659 1/1660 1/1661 1/1662 1/1663 1/1664 1/1665 1/1666 1/1667 1/1668 1/1669 1/1670 1/1671 1/1672 1/1673 1/1674 1/1675 1/1676 1/1677 1/1678 1/1679 1/1680 1/1681 1/1682 1/1683 1/1684 1/1685 1/1686 1/1687 1/1688 1/1689 1/1690 1/1691 1/1692 1/1693 1/1694 1/1695 1/1696 1/1697 1/1698 1/1699 1/1700 1/1701 1/1702 1/1703 1/1704 1/1705 1/1706 1/1707 1/1708 1/1709 1/1710 1/1711 1/1712 1/1713 1/1714 1/1715 1/1716 1/1717 1/1718 1/1719 1/1720 1/1721 1/1722 1/1723 1/1724 1/1725 1/1726 1/1727 1/1728 1/1729 1/1730 1/1731 1/1732 1/1733 1/1734 1/1735 1/1736 1/1737 1/1738 1/1739 1/1740 1/1741 1/1742 1/1743 1/1744 1/1745 1/1746 1/1747 1/1748 1/1749 1/1750 1/1751 1/1752 1/1753 1/1754 1/1755 1/1756 1/1757 1/1758 1/1759 1/1760 1/1761 1/1762 1/1763 1/1764 1/1765 1/1766 1/1767 1/1768 1/1769 1/1770 1/1771 1/1772 1/1773 1/1774 1/1775 1/1776 1/1777 1/1778 1/1779 1/1780 1/1781 1/1782 1/1783 1/1784 1/1785 1/1786 1/1787 1/1788 1/1789 1/1790 1/1791 1/1792 1/1793 1/1794 1/1795 1/1796 1/1797 1/1798 1/1799 1/1800 1/1801 1/1802 1/1803 1/1804 1/1805 1/1806 1/1807 1/1808 1/1809 1/1810 1/1811 1/1812 1/1813 1/1814 1/1815 1/1

Step 1: Create the agent

Add the tool to the Agent. The tool decides internally when to request approval based on the order size.

```

[1]: # Create shipping agent with pauseable tool
shipping_agent = LLMAgent(
    name="shipping_agent",
    model=Gemini(model="gemini-1.5-flash-lite", retry_options=retry_config,
    instructions="You are a shipping coordinator assistant.

When users request to ship containers:
1. Use the place_shipping_order tool with the number of containers and destination
2. If the order status is 'pending', inform the user that approval is required
3. After receiving the final result, provide a clear summary including:
    - Order status (approved/rejected)
    - Order ID (if available)
    - Number of containers and destination
4. Keep responses concise but informative
...
    tools=FunctionTool([func_place_shipping_order]),
)

print("🚢 Shipping Agent created!")

```

✅ Shipping Agent created!

Step 2: Wrap in resumable App

The problem: A regular `LLMAgent` is stateless - each call is independent with no memory of previous interactions. If a tool requests approval, the agent can't remember what it was doing.

The solution: Wrap your agent in an `App` with `resumability` enabled. The `App` adds a persistence layer that saves and restores state.

What gets saved when a tool pauses:

- All conversation messages so far
- Which tool was called (`place_shipping_order`)
- Tool parameters (10 containers, Rotterdam)
- Where exactly it paused (waiting for approval)

When you resume, the App loads this saved state so the agent continues exactly where it left off - as if no time passed.

```

100 | # Wrap the agent in a resumable app - THIS IS THE KEY FOR LONG-RUNNING OPERATIONS!
      | shipping_app = App(
      |     name="shipping_coordinator",
      |     root_agent=shipping_agent,
      |     resumability_config=ResumabilityConfig(is_resumable=True),
      | )
      |
      | print("✅ Resumable app created!")

```

Draft guardado

Step 3: Create Session and Runner with the App

Pass `app=shipping_app` instead of `agent=...` so the runner knows about resumability.

```
[59]: session_service = InMemorySessionService()

# Create runner with the resumable app
shipping_runner = Runner(
    app=shipping_app, # Pass the app instead of the agent
    session_service=session_service,
)

print("✅ Runner created!")
```

✅ Runner created!

[illegible]

Day 2b - Agent Tools Best Practices Draft guardado

Archivo Edición Vista Corre Escenarios Complementos Ayuda

+ - Cancelar Run Código

4.3: Helper Functions to Process Events

These handle the event iteration logic for you.

`check_for_approval()` - Detects if the agent paused

- Loops through all events and looks for the special `adk_request_confirmation` event
- Returns `approval_id` (identifies this specific request) and `invocation_id` (identifies which execution to resume)
- Returns `None` if no pause detected

```
[79]: def check_for_approval(events):  
    """Check if events contain an approval request.  
  
    Returns:  
    dict with approval details or None  
    """  
    for event in events:  
        if event.content and event.content.parts:  
            for part in event.content.parts:  
                if (part.function_call  
                    and part.function_call.name == "adk_request_confirmation"):  
                    return {  
                        "approval_id": part.function_call.id,  
                        "invocation_id": event.invocation_id,  
                    }  
    return None
```

`print_agent_response()` - Displays agent text

- Simple helper to extract and print text from events

```
+ Code + Markdown  
[80]: def print_agent_response(events):  
    """Print agent's text responses from events."""  
    for event in events:  
        if event.content and event.content.parts:  
            for part in event.content.parts:  
                if part.text:  
                    print(f"Agent > {part.text}")
```

`create_approval_response()` - formats the human decision

- Takes the approval info and boolean decision (True/False) from the human
- Creates a `FunctionResponse` that ADK understands
- Wraps it in a `Content` object to send back to the agent

```
[18]:
```

Compartir Guardar la versión 0

Secretos

Fragmento de código

```
from kaggle_secrets import UserSecrets  
user_secrets = UserSecretsClient()  
secret_value_0 = user_secrets.get_secret_value_0
```

Copiar a la carpeta

GOOGLE_API_KEY

Añadir secreto Fragmento de copia

29°C Soleado 12:33 p.m. 04/05/2024

Day 2b - Agent Tools Best Practices

Draft guardado

Archivo Edición Vista Corre Escenarios Complementos Ayuda

+ - Cancelar Run Código

```
[18]: def create_approval_response(approval_info, approved):  
    """Create approval response message."""  
    confirmation_response = types.FunctionResponse(  
        id=approval_info["approval_id"],  
        name="adk_request_confirmation",  
        response={"confirmed": approved},  
    )  
    return types.Content(  
        role="user", parts=[types.Part(function_response=confirmation_response)]  
    )  
  
print("✅ Helper functions defined")
```

✅ Helper functions defined

Day 2b - Agent Tools Best Practices

4.4: The Workflow Function - Let's tie it all together!

The `run_shipping_workflow()` function orchestrates the entire approval flow. Look for the code explanation in the cell below.

```
graph TD
    UserQuery[User Query] --> Step1[STEP 1: Call Agent: run_async()]
    Step1 --> Tool[place_shipping_order Tool]
    Tool --> Step2[STEP 2: check_for_approval()]
    Step2 -- "loop through the events" --> Step2
    Step2 -- "'adk_request_confirmation' event found?" --> PathA[PATH A: Approval needed]
    Step2 -- "None" --> PathB[PATH B: Approval not needed]
    PathA --> Pause[Pause Agent execution]
    Pause --> GetDecision[Get human decision]
    GetDecision --> CallAgentAgain[Call Agent AGAIN run_async()]
    CallAgentAgain -- "with SAME 'invocation_id'" --> AgentResumes[Agent resumes and completes]
    PathB --> OrderPlaced[Order placed, print_agent_response() No approval needed.]
```

LRO Workflow

```
print(f"\n{'-'*60}")
print(f"User > {query}\n")

# Generate unique session ID
session_id = f"order_{uuid.uuid4().hex[:8]}"

# Create session
await session_service.create_session(
    app_name="shipping_coordinator", user_id="test_user", session_id=session_id
)

query_content = types.Content(role="user", parts=[types.Part(text=query)])
events = []

# -----
# STEP 1: Send initial request to the Agent. If num_containers > 5, the Agent returns the special 'adk_request_confirmation' event
async for event in shipping_runner.run_async(
    user_id="test_user", session_id=session_id, new_message=query_content
):
    events.append(event)

# -----
# STEP 2: Loop through all the events generated and check if 'adk_request_confirmation' is present.
approval_info = check_for_approval(events)

# -----
# STEP 3: If the event is present, it's a large order - HANDLE APPROVAL WORKFLOW
if approval_info:
    print(f"Pausing for approval...")
    print(f"Human Decision: {'APPROVE' if auto_approve else 'REJECT'}\n")

    # PATH A: Resume the agent by calling run_async() again with the approval decision
    async for event in shipping_runner.run_async(
        user_id="test_user",
        session_id=session_id,
        new_message=create_approval_response(
            approval_info, auto_approve
        ), # Send human decision here
        invocation_id=approval_info["invocation_id"], # Critical: same invocation_id tells ADK to RESUME
    ):
        if event.content and event.content.parts:
            for part in event.content.parts:
                if part.text:
                    print(f"Agent > {part.text}")

    # -----
else:
    # PATH B: If the 'adk_request_confirmation' is not present - no approval needed - order completed immediately.
    print_agent_response(events)

print(f"\n{'-'*60}")

print("Workflow function ready")
```

4.5: Demo: Testing the Workflow

Now, let's run our demos. Notice how much cleaner and easier to read they are. All the complex logic for pausing and resuming is now hidden away in our `run_workflow` helper function, allowing us to focus on the tasks we want the agent to perform.

Note: You may see warnings like `Warning: there are non-text parts in the response: ['function_call']` - this is normal and can be ignored. It just means the agent is calling tools in addition to generating text.

```
In [21]: # Demo 1: It's a small order. Agent receives auto-approved status from tool
await run_shipping_workflow("Ship 3 containers to Singapore")

# Demo 2: Workflow simulates human decision: APPROVE
await run_shipping_workflow("Ship 10 containers to Rotterdam", auto_approve=True)

# Demo 3: Workflow simulates human decision: REJECT
await run_shipping_workflow("Ship 8 containers to Los Angeles", auto_approve=False)
```

=====

User > Ship 3 containers to Singapore

WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

Agent > Shipping order approved! 3 containers will be sent to Singapore. The order ID is ORD-3-AUTO.

=====

User > Ship 10 containers to Rotterdam

WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

/usr/local/lib/python3.11/dist-packages/google/adk/tools/tool_context.py:92: UserWarning: [EXPERIMENTAL] ToolConfirmation: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.

ToolConfirmation(

/usr/local/lib/python3.11/dist-packages/google/adk/agents/invoke_context.py:298: UserWarning: [EXPERIMENTAL] BaseAgentState: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.

self.agent_states[event.author] = BaseAgentState()

⏸ Pausing for approval...

👤 Human Decision: APPROVE

Agent > Shipping order approved. Order ID: ORD-10-HUMAN. 10 containers will be shipped to Rotterdam.

=====

User > Ship 8 containers to Los Angeles

WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

⏸ Pausing for approval...

👤 Human Decision: REJECT

Agent > Your shipping order for 8 containers to Los Angeles has been rejected.



Día 2 (Herramientas de Agente e Interoperabilidad con el Protocolo de Contexto de Modelo (MCP))

Bienvenidos al Día 2.

Este documento técnico se centra en herramientas externas, funciones que permiten a un agente realizar acciones o recuperar datos en tiempo real más allá de su conjunto de entrenamiento e introduce las mejores prácticas para diseñar herramientas eficaces. Aprenderás sobre MCP, destacando sus componentes arquitectónicos, la capa de comunicación, los riesgos y las brechas de preparación empresarial.

En los codelabs, crearás herramientas personalizadas para tus agentes convirtiendo tus propias funciones en Python en acciones que tu agente pueda realizar. También usarás MCP e implementarás operaciones de larga duración donde un agente puede pausar las llamadas a herramientas mientras espera la aprobación humana, antes de reanudar.

Asignaciones del día 2:

Completar la Unidad 2 - "Herramientas de Agente e Interoperabilidad con el Protocolo de Contexto del Modelo (MCP)":

- ✓ Escucha el episodio resumen del podcast de esta unidad.
- ✓ Para complementar el podcast, lee el [whitepaper "Agent Tools & Interoperability with Model Context Protocol \(MCP\)"](#).
- ✓ Completa estos codelabs en Kaggle:
 - ✓ Explora nuevas formas de añadir herramientas que amplíen lo que tus agentes pueden hacer.
 - ✓ Explora las mejores prácticas para herramientas, incluyendo el uso de MCP y operaciones de larga duración.

[Opcional]: Mira la retransmisión en directo [grabada de YouTube](#). Kanchana Patlolla y Anant Nawalgaria estuvieron acompañados por la autora de Codelab, Laxmi Harikumar, junto con los invitados de Google Edward Grefenstette, Mike Styer y Oriol Vinyals, así como el ponente externo Alex Wissner-Gross de [Reified](#), para debatir las asignaciones y compartir sus ideas.

DIA 3A

https://www.kaggle.com/code/jesdurnetabola/day-3a-agent-sessions/edit

Day 3a - Agent Sessions

Draft guardado

Archivo Edición Vista Corre Escenarios Complementos Ayuda

+ Cancelar Run Código Sesión de borrador Camera54m

Section 1: Setup

1.1: Install dependencies

The Kaggle Notebooks environment includes a pre-installed version of the [google-adk](#) library for Python and its required dependencies, so you don't need to install additional packages in this notebook.

To install and use ADK in your own Python development environment outside of this course, you can do so by running:

```
pip install google-adk
```

[2]:
pip install google-adk

Requirement already satisfied: google-adk in /usr/local/lib/python3.11/dist-packages (1.15.0)
Requirement already satisfied: PyMySQL<7.0.0,>=6.0.2 in /usr/local/lib/python3.11/dist-packages (from google-adk) (6.0.3)
Requirement already satisfied: anyio<5.0.0,>=4.9.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (4.11.0)
Requirement already satisfied: authlib<2.0.0,>=1.5.1 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.6.5)
Requirement already satisfied: click<9.0.0,>=8.1.1 in /usr/local/lib/python3.11/dist-packages (from google-adk) (8.3.0)
Requirement already satisfied: fastapi<1.115.0,>=0.115.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.116.1)
Requirement already satisfied: google-api-python-client<3.0.0,>=2.157.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.177.0)
Requirement already satisfied: google-cloud-aiplatform<2.0.0,>=1.125.0 in /usr/local/lib/python3.11/dist-packages (from google-cloud-aiplatform[agent-engines]<2.0.0,>=1.125.0->google-adk) (1.125.0)
Requirement already satisfied: google-cloud-bigtable<2.32.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.34.0)
Requirement already satisfied: google-cloud-discoveryengine<0.14.0,>=0.13.12 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.13.12)
Requirement already satisfied: google-cloud-secret-manager<3.0.0,>=2.22.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.25.0)
Requirement already satisfied: google-cloud-spanner<4.0.0,>=3.56.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (3.56.0)
Requirement already satisfied: google-cloud-speech<3.0.0,>=2.30.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.34.0)
Requirement already satisfied: google-cloud-storage<4.0.0,>=3.0.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (3.5.0)
Requirement already satisfied: google-gensim<2.0.0,>=1.45.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.45.0)
Requirement already satisfied: graphviz<1.0.0,>=0.20.2 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.21)
Requirement already satisfied: mcp<2.0.0,>=1.8.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.20.0)
Requirement already satisfied: opentelemetry-api<1.37.0,>=1.37.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.37.0)
Requirement already satisfied: opentelemetry-exporter-gcp-logging<2.0.0,>=1.9.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.11.0)
Requirement already satisfied: opentelemetry-exporter-gcp-monitoring<2.0.0,>=1.9.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.11.0)
Requirement already satisfied: opentelemetry-exporter-gcp-trace<2.0.0,>=1.9.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.11.0)
Requirement already satisfied: opentelemetry-exporter-otlp-proto-http<1.36.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.37.0)
Requirement already satisfied: opentelemetry-resource-detector-gcp<2.0.0,>=1.9.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.11.0)
Requirement already satisfied: opentelemetry-sdk<1.37.0,>=1.37.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.37.0)
Requirement already satisfied: pydantic<3.0.0,>=2.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.12.4)
Requirement already satisfied: python-dateutil<3.0.0,>=2.9.0.post0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.9.0.post0)
Requirement already satisfied: python-dotenv<2.0.0,>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.2.1)
Requirement already satisfied: requests<3.0.0,>=2.32.4 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.32.5)
Requirement already satisfied: sqlalchemy-spanner<1.14.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.17.1)
Requirement already satisfied: sqlalchemy<3.0.0,>=2.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.0.41)
Requirement already satisfied: starlette<1.0.0,>=0.46.2 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.47.2)
Requirement already satisfied: tenacity<10.0.0,>=9.0.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (9.1.2)
Requirement already satisfied: typing-extensions<5,>=4.5 in /usr/local/lib/python3.11/dist-packages (from google-adk) (4.15.0)
Requirement already satisfied: tlzocal<6.0,>=5.3 in /usr/local/lib/python3.11/dist-packages (from google-adk) (5.3.1)
Requirement already satisfied: uvicorn<1.0.0,>=0.34.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.35.0)
Requirement already satisfied: watchdog<7.0.0,>=6.0.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (6.0.0)
Requirement already satisfied: websockets<16.0.0,>=15.0.1 in /usr/local/lib/python3.11/dist-packages (from google-adk) (15.0.1)
Requirement already satisfied: idna<=2.8 in /usr/local/lib/python3.11/dist-packages (from anyio<5.0.0,>=4.9.0->google-adk) (3.11)
Requirement already satisfied: sniffio<=1.1 in /usr/local/lib/python3.11/dist-packages (from anyio<5.0.0,>=4.9.0->google-adk) (1.3.1)
Requirement already satisfied: cryptography in /usr/local/lib/python3.11/dist-packages (from authlib<2.0.0,>=1.5.1->google-adk) (46.0.3)
Requirement already satisfied: http112<1.0.0,>=0.19.0 in /usr/local/lib/python3.11/dist-packages (from google-api-python-client<3.0.0,>=2.157.0->google-adk) (0.22.0)
Requirement already satisfied: google-auth<2.34.0,>=2.25.0 in /usr/local/lib/python3.11/dist-packages (from google-api-python-client<3.0.0,>=2.157.0->google-adk) (2.38.0)
Requirement already satisfied: google-auth-http112<1.0.0,>=0.2.0 in /usr/local/lib/python3.11/dist-packages (from google-api-python-client<3.0.0,>=2.157.0->google-adk) (0.2.0)
Requirement already satisfied: google-api-core<1.2.0,*>=1.2.2,*>=2.3.0 in /usr/local/lib/python3.11/dist-packages (from google-api-python-client<3.0.0,>=2.157.0->google-adk) (2.28.1)
Requirement already satisfied: wrttemplate<5,>=3.0.1 in /usr/local/lib/python3.11/dist-packages (from google-api-python-client<3.0.0,>=2.157.0->google-adk) (4.2.0)
Requirement already satisfied: proto-plus<2.0.0,>=1.22.3 in /usr/local/lib/python3.11/dist-packages (from google-cloud-aiplatform<2.0.0,>=1.125.0->google-cloud-aiplatform[agent-engines]<2.0.0,>=1.125.0->google-adk) (1.26.1)
Requirement already satisfied: protobuf<4.21.0,>=4.21.1 in /usr/local/lib/python3.11/dist-packages (from google-cloud-aiplatform<2.0.0,>=1.125.0->google-cloud-aiplatform[agent-engines]<2.0.0,>=1.125.0->google-adk) (5.29.6)
Requirement already satisfied: packaging<=14.3 in /usr/local/lib/python3.11/dist-packages (from google-cloud-aiplatform<2.0.0,>=1.125.0->google-cloud-aiplatform[agent-engines]<2.0.0,>=1.125.0->google-adk) (25.0)
Requirement already satisfied: google-cloud-bigrquery<3.20.0,>=4.0.0 in /usr/local/lib/python3.11/dist-packages (from google-cloud-aiplatform<2.0.0,>=1.125.0->google-cloud-aiplatform[agent-engines]<2.0.0,>=1.125.0->google-adk) (3.35.1)
Requirement already satisfied: google-cloud-resource-manager<3.0.0,>=1.3.3 in /usr/local/lib/python3.11/dist-packages (from google-cloud-aiplatform<2.0.0,>=1.125.0->google-cloud-aiplatform[agent-engines]<2.0.0,>=1.125.0->google-adk) (1.14.2)

1.2: Configure your Gemini API Key

This notebook uses the [Gemini API](#), which requires authentication.

1. Get your API key

If you don't have one already, create an [API key in Google AI Studio](#).

2. Add the key to Kaggle Secrets

Next, you will need to add your API key to your Kaggle Notebook as a Kaggle User Secret.

1. In the top menu bar of the notebook editor, select 'Add-ons' then 'Secrets'.
2. Create a new secret with the label `GOOGLE_API_KEY`.
3. Paste your API key into the "Value" field and click "Save".
4. Ensure that the checkbox next to `GOOGLE_API_KEY` is selected so that the secret is attached to the notebook.

3. Authenticate in the notebook

Run the cell below to complete authentication.

+ Code + Markdown

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Gemini API key setup complete.")
except Exception as e:
    print(
        f"🚨 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}"
    )
```

✅ Gemini API key setup complete.

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

```
from typing import Any, Dict

from google.adk.agents import Agent, LlmAgent
from google.adk.apps.app import App, EventsCompactionConfig
from google.adk.models.google_llm import Gemini
from google.adk.sessions import DatabaseSessionService
from google.adk.sessions import InMemorySessionService
from google.adk.runners import Runner
from google.adk.tools.tool_context import ToolContext
from google.genai import types

print("✅ ADK components imported successfully.")
```

✅ ADK components imported successfully.

1.4: Helper functions

Helper function that manages a complete conversation session, handling session creation/retrieval, query processing, and response streaming. It supports both single queries and multiple queries in sequence.

Example:

```
>>> await run_session(runner, "What is the capital of France?", "geography-session")
>>> await run_session(runner, ["Hello!", "What's my name?"], "user-intro-session")
```

+ Code

+ Markdown

[5]:

```
# Define helper functions that will be reused throughout the notebook
async def run_session(
    runner_instance: Runner,
    user_queries: list[str] | str = None,
    session_name: str = "default",
):
    print(f"\n ### Session: {session_name}")

    # Get app name from the Runner
    app_name = runner_instance.app_name

    # Attempt to create a new session or retrieve an existing one
    try:
        session = await session_service.create_session(
            app_name=app_name, user_id=USER_ID, session_id=session_name
        )
    except:
        session = await session_service.get_session(
            app_name=app_name, user_id=USER_ID, session_id=session_name
        )

    # Process queries if provided
    if user_queries:
        # Convert single query to list for uniform processing
        if type(user_queries) == str:
            user_queries = [user_queries]

        # Process each query in the list sequentially
        for query in user_queries:
            print(f"\nUser > {query}")

            # Convert the query string to the ADK Content format
            query = types.Content(role="user", parts=[types.Part(text=query)])
```

```

app_name = runner_instance.app_name

# Attempt to create a new session or retrieve an existing one
try:
    session = await session_service.create_session(
        app_name=app_name, user_id=USER_ID, session_id=session_name
    )
except:
    session = await session_service.get_session(
        app_name=app_name, user_id=USER_ID, session_id=session_name
    )

# Process queries if provided
if user_queries:
    # Convert single query to list for uniform processing
    if type(user_queries) == str:
        user_queries = [user_queries]

    # Process each query in the list sequentially
    for query in user_queries:
        print(f"\nUser > {query}")

        # Convert the query string to the ADK Content format
        query = types.Content(role="user", parts=[types.Part(text=query)])

        # Stream the agent's response asynchronously
        async for event in runner_instance.run_async(
            user_id=USER_ID, session_id=session.id, new_message=query
        ):
            # Check if the event contains valid content
            if event.content and event.content.parts:
                # Filter out empty or "None" responses before printing
                if (
                    event.content.parts[0].text != "None"
                    and event.content.parts[0].text
                ):
                    print(f"{MODEL_NAME} > ", event.content.parts[0].text)
            else:
                print("No queries!")

print("✅ Helper functions defined.")

```

✅ Helper functions defined.

2.4 Implementing Our First Stateful Agent

Let's build our first stateful agent, that can remember and have constructive conversations.

ADK offers different types of sessions suitable for different needs. As a start, we'll start with a simple Session Management option (`InMemorySessionService`):

```
[7]: APP_NAME = "default" # Application
USER_ID = "default" # User
SESSION = "default" # Session

MODEL_NAME = "gemini-2.5-flash-lite"

# Step 1: Create the LLM Agent
root_agent = Agent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="text_chat_bot",
    description="A text chatbot", # Description of the agent's purpose
)

# Step 2: Set up Session Management
# InMemorySessionService stores conversations in RAM (temporary)
session_service = InMemorySessionService()

# Step 3: Create the Runner
runner = Runner(agent=root_agent, app_name=APP_NAME, session_service=session_service)

print("✅ Stateful agent initialized!")
print(f"   - Application: {APP_NAME}")
print(f"   - User: {USER_ID}")
print(f"   - Using: {session_service.__class__.__name__}")
```

```
✅ Stateful agent initialized!
- Application: default
- User: default
Using: InMemorySessionService
```

2.5 Testing Our Stateful Agent

Now let's see the magic of sessions in action!

```
In [7]: # Run a conversation with two queries in the same session
# Notice: Both queries are part of the SAME session, so context is maintained
await run_session(
    runner,
    [
        "Hi, I am Sam! What is the capital of United States?",
        "Hello! What is my name?", # This time, the agent should remember!
    ],
    "stateful-agentic-session",
)
```

```
### Session: stateful-agentic-session
```

```
User > Hi, I am Sam! What is the capital of United States?
gemini-2.5-flash-lite > Hello Sam! The capital of the United States is Washington, D.C.
```

```
User > Hello! What is my name?
gemini-2.5-flash-lite > Your name is Sam.
```

🎉 **Success!** The agent remembered your name because both queries were part of the same session. The Runner automatically maintained the conversation history.

But there's a catch: `InMemorySessionService` is temporary. **Once the application stops, all conversation history is lost.**

● (Optional) 2.6 Testing Agent's forgetfulness

To verify that the agent forgets the conversation, **restart the kernel**. Then, **run ALL the previous cells in this notebook EXCEPT the `run_session` in 2.5.**

Now run the cell below. You'll see that the agent doesn't remember anything from the previous conversation.

```
In [6]: # Run this cell after restarting the kernel. All this history will be gone...
await run_session(
    runner,
    [
        "What did I ask you about earlier?", "And remind me, what's my name?"
    ],
    "stateful-agentic-session",
) # Note, we are using same session name
```

```
### Session: stateful-agentic-session
```

```
User > What did I ask you about earlier?
gemini-2.5-flash-lite > I cannot recall previous conversations. I am a text chatbot and do not have memory of past interactions.
```

```
User > And remind me, what's my name?
gemini-2.5-flash-lite > I do not have access to your personal information, including your name. I am a text chatbot and do not have memory of past interactions.
```

3.2 Implementing Persistent Sessions

Let's upgrade to `DatabaseSessionService` using SQLite. This gives us persistence without needing a separate database server for this demo.

Let's create a `chatbot_agent` capable of having a conversation with the user.

```
In [8]: # Step 1: Create the same agent (notice we use LlmAgent this time)
chatbot_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="text_chat_bot",
    description="A text chatbot with persistent memory",
)

# Step 2: Switch to DatabaseSessionService
# SQLite database will be created automatically
db_url = "sqlite:///my_agent_data.db" # Local SQLite file
session_service = DatabaseSessionService(db_url=db_url)

# Step 3: Create a new runner with persistent storage
runner = Runner(agent=chatbot_agent, app_name=APP_NAME, session_service=session_service)

print("✅ Upgraded to persistent sessions!")
print(f"  - Database: my_agent_data.db")
print(f"  - Sessions will survive restarts!")

✅ Upgraded to persistent sessions!
  - Database: my_agent_data.db
  - Sessions will survive restarts!
```

3.3 Test Run 1: Verifying Persistence

In this first test run, we'll start a new conversation with the session ID `test-db-session-01`. We will first introduce our name as 'Sam' and then ask a question. In the second turn, we will ask the agent for o

Since we are using `DatabaseSessionService`, the agent should remember the name.

After the conversation, we'll inspect the `my_agent_data.db` SQLite database directly to see how the conversation `events` (the user queries and model responses) are stored.

```
In [9]: await run_session(
    runner,
    ["Hi, I am Sam! What is the capital of the United States?", "Hello! What is my name?"],
    "test-db-session-01",
)

### Session: test-db-session-01

User > Hi, I am Sam! What is the capital of the United States?
gemini-2.5-flash-lite > Hi Sam! The capital of the United States is Washington, D.C.

User > Hello! What is my name?
gemini-2.5-flash-lite > Your name is Sam.
```

3.5 Let's verify that the session data is isolated

As mentioned earlier, a session is private conversation between an Agent and a User (i.e., two sessions do not share information). Let's run our `run_session` with a different session name `test-db-session-02` to confirm this.

```
In [10]: await run_session(
          runner, [{"Hello! What is my name?"}, "test-db-session-02"]
        ) # Note, we are using new session name

### Session: test-db-session-02

User > Hello! What is my name?
gemini-2.5-flash-lite > I do not have access to your personal information, including your name. As a large language model, I am designed to be a helpful and harmless AI assistant. I do not store or recall personal details about users.
```

3.6 How are the events stored in the Database?

Since we are using a sqlite DB to store information, let's have a quick peek to see how information is stored.

```
In [11]: import sqlite3

def check_data_in_db():
    with sqlite3.connect("my_agent_data.db") as connection:
        cursor = connection.cursor()
        result = cursor.execute(
            "select app_name, session_id, author, content from events"
        )
        print([_[0] for _ in result.description])
        for each in result.fetchall():
            print(each)

check_data_in_db()

['app_name', 'session_id', 'author', 'content']
('default', 'test-db-session-01', 'user', '{"parts": [{"text": "Hi, I am Sam! What is the capital of the United States?"}], "role": "user"}')
('default', 'test-db-session-01', 'text_chat_bot', '{"parts": [{"text": "Hi Sam! The capital of the United States is Washington, D.C."}], "role": "model"}')
('default', 'test-db-session-01', 'user', '{"parts": [{"text": "Hello! What is my name?"}], "role": "user"}')
('default', 'test-db-session-01', 'text_chat_bot', '{"parts": [{"text": "Your name is Sam."}], "role": "model"}')
('default', 'test-db-session-02', 'user', '{"parts": [{"text": "Hello! What is my name?"}], "role": "user"}')
('default', 'test-db-session-02', 'text_chat_bot', '{"parts": [{"text": "I do not have access to your personal information, including your name. As a large language model, I am designed to be a helpful and harmless AI assistant. I do not store or recall personal details about users."}], "role": "model"}')
```

Section 4: Context Compaction

4.1 Create an App for the agent

To enable this feature, let's use the same `chatbot_agent` we created in Section 3.2.

The first step is to create an object called `App`. We'll give it a name and pass in our `chatbot_agent`.

We'll also create a new config to do the Context Compaction. This `EventsCompactionConfig` defines two key variables:

- **compaction_interval**: Asks the Runner to compact the history after every `n` conversations
- **overlap_size**: Defines the number of previous conversations to retain for overlap

We'll then provide this app to the Runner.

```
In [12]: # Re-define our app with Events Compaction enabled
research_app_compacting = App(
    name="research_app_compacting",
    root_agent=chatbot_agent,
    # This is the new part!
    events_compaction_config=EventsCompactionConfig(
        compaction_interval=3, # Trigger compaction every 3 invocations
        overlap_size=1, # Keep 1 previous turn for context
    ),
)

db_url = "sqlite:///my_agent_data.db" # Local SQLite file
session_service = DatabaseSessionService(db_url=db_url)

# Create a new runner for our upgraded app
research_runner_compacting = Runner(
    app=research_app_compacting, session_service=session_service
)

print("✅ Research App upgraded with Events Compaction!")

✅ Research App upgraded with Events Compaction!

/tmp/ipykernel_103/3773147741.py:6: UserWarning: [EXPERIMENTAL] EventsCompactionConfig: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  events_compaction_config=EventsCompactionConfig(
```

4.2 Running the Demo

4.2 Running the Demo

Now, let's have a conversation that is long enough to trigger the compaction. When you run the cell below, the output will look like a normal conversation. However, because we configured our `App`, a compaction process will run silently in the background after the 3rd invocation.

In the next step, we'll prove that it happened.

```
In [13]: # Turn 1
await run_session(
    research_runner_compacting,
    "What is the latest news about AI in healthcare?",
    "compaction_demo",
)

# Turn 2
await run_session(
    research_runner_compacting,
    "Are there any new developments in drug discovery?",
    "compaction_demo",
)

# Turn 3 - Compaction should trigger after this turn!
await run_session(
    research_runner_compacting,
    "Tell me more about the second development you found.",
    "compaction_demo",
)

# Turn 4
await run_session(
    research_runner_compacting,
    "Who are the main companies involved in that?",
    "compaction_demo",
)
```

Session: compaction_demo

User > What is the latest news about AI in healthcare?
gemini-2.5-flash-lite > The field of AI in healthcare is incredibly dynamic, with new developments emerging constantly. To give you the most relevant and up-to-date information, I'll focus on some of the key areas where AI is making significant strides right now:

****1. Diagnostics and Imaging:****

- **Enhanced Image Analysis:**** AI algorithms, particularly deep learning models, are becoming increasingly sophisticated at analyzing medical images like X-rays, CT scans, MRIs, and pathology slides. They can detect subtle patterns indicative of diseases like cancer, diabetic retinopathy, and neurological disorders with remarkable accuracy, often at earlier stages than human eyes alone.
- **Early Disease Detection:**** Beyond imaging, AI is being used to analyze various data points (patient history, lab results, genetic information) to predict the risk of certain diseases, enabling earlier interventions.
- **Personalized Treatment Recommendations:**** By analyzing a patient's unique data and comparing it to vast datasets of similar cases, AI can help clinicians recommend the most effective and personalized treatment plans.

****2. Drug Discovery and Development:****

- **Accelerated Drug Discovery:**** AI is revolutionizing the drug discovery process by identifying potential drug candidates much faster. It can sift through massive databases of molecules and predict their efficacy and potential side effects, significantly shortening the time and cost associated with bringing new drugs to market.
- **Repurposing Existing Drugs:**** AI can identify existing medications that might be effective for new conditions, offering a quicker path to treatment for various diseases.
- **Clinical Trial Optimization:**** AI is helping to design more efficient clinical trials, identify suitable patient cohorts, and predict trial outcomes.

****3. Operational Efficiency and Workflow Optimization:****

- **Administrative Task Automation:**** AI-powered tools are automating mundane administrative tasks like scheduling appointments, managing patient records, and processing insurance claims, freeing up healthcare professionals to focus on patient care.
- **Predictive Staffing:**** AI can forecast patient volumes and optimize staffing levels in hospitals and clinics, ensuring adequate resources are available when and where they are needed.
- **Streamlined Patient Flow:**** AI can help manage patient flow within hospitals, reducing wait times and improving the overall patient experience.

****4. Personalized Medicine and Genomics:****

4.3 Verifying Compaction in the Session History

The conversation above looks normal, but the history has been changed behind the scenes. How can we prove it?

We can inspect the `events` list from our session. The compaction process **doesn't delete old events; it replaces them with a single, new Event that contains the summary**. Let's find it.

```
In [14]: # Get the final session state
final_session = await session_service.get_session(
    app_name=research_runner_compacting.app_name,
    user_id=USER_ID,
    session_id="compaction_demo",
)

print("--- Searching for Compaction Summary Event ---")
found_summary = False
for event in final_session.events:
    # Compaction events have a 'compaction' attribute
    if event.actions and event.actions.compaction:
        print("\n✅ SUCCESS! Found the Compaction Event:")
        print(f"  Author: {event.author}")
        print(f"  {event.compacted_information}")
        found_summary = True
        break

if not found_summary:
    print(
        "\n❌ No compaction event found. Try increasing the number of turns in the demo."
    )
```

--- Searching for Compaction Summary Event ---

✅ SUCCESS! Found the Compaction Event:
Author: user

Compacted information: model_version=None content=None grounding_metadata=None partial=None turn_complete=None finish_reason=None error_code=None error_message=None interrupted=None custom_metadata=None usage_metadata=None liv
e_session_resumption_update=None input_transcription=None output_transcription=None avg_logprobs=None logprobs_result=None cache_metadata=None citation_metadata=None invocation_id='894aa428-1786-4537-9044-0c858ff86662' author
='user' actions=EventActions(skip_summarization=None, state_delta={}, artifact_delta={}, transfer_to_agent=None, escalate=None, requested_auth_configs={}, requested_tool_confirmations={}, compaction={'start_timestamp': 17630584
69.903667, 'end_timestamp': 1763058477.639977, 'compacted_content': {'parts': [{'function_call': None, 'code_execution_result': None, 'executable_code': None, 'file_data': None, 'function_response': None, 'inline_data': None,
'text': 'The user inquired about the latest news in AI in healthcare. The AI agent provided a comprehensive overview covering diagnostics, drug discovery, operational efficiency, personalized medicine, and mental health, while
also highlighting challenges and ethical considerations.\n\nFollowing up on drug discovery, the user specifically asked about new developments in that area. The AI agent elaborated on several key advancements:\n\n* **Generati
ve AI For Molecule Design:** AI creating novel molecules optimized for specific properties.\n* **AI for Target Identification and Validation:** AI analyzing complex biological data to find and confirm disease targets.\n* **AI
in Clinical Trial Design and Optimization:** AI predicting patient response and optimizing trials.\n* **AI for Drug Repurposing:** AI finding new uses for existing drugs.\n* **Advancements in Protein Folding:** AI models
like AlphaFold aiding in understanding drug-target interactions.\n* **AI in Identifying and Mitigating Drug Toxicity:** AI predicting adverse effects early.\n\nThe user then requested more information on the "AI for Target Id
entification and Validation" development. The AI agent explained the concept of a drug target, the traditional challenges in identifying them, and how AI is revolutionizing this process through:\n\n* **Analyzing Omics Data:**
Identifying disease-associated genes and proteins.\n* **Integrating Multi-Modal Data:** Connecting diverse data sources to reveal complex relationships.\n* **Predicting Target Druggability:** Assessing the likelihood of a t
arget being effectively modulated by a drug.\n* **Validating Targets:** Designing experiments and simulating outcomes more efficiently.\n\nKey companies involved in this area were mentioned, including Recursion Pharmaceutical
s, BenevolentAI, Relay Therapeutics, and Insilico Medicine. The AI agent emphasized that this advancement significantly reduces time and cost, increases success rates, and uncovers novel disease biology.\n\n**Unresolved Questio
ns/Tasks:**\n\nThe conversation has been purely informational so far. The user has not requested any specific actions or posed further questions.', 'thought': None, 'thought_signature': None, 'video_metadata': None}], 'rol
e': 'model'}), end_of_agent=None, agent_state=None, rewind_before_invocation_id=None) long_running_tool_ids=set() branch=None id='e0a8db1f-e1a7-4dec-9e75-384658f7e35e' timestamp=1763058485.430933

🟡 Section 5: Working with Session State

5.1 Creating custom tools for Session state management

Let's explore how to manually manage session state through custom tools. In this example, we'll identify a **transferable characteristic**, like a user's name and their country, and create tools to capture and save it.

Why This Example?

The username is a perfect example of information that:

- Is introduced once but referenced multiple times
- Should persist throughout a conversation
- Represents a user-specific characteristic that enhances personalization

Here, for demo purposes, we'll create two tools that can store and retrieve user name and country from the Session State. **Note that all tools have access to the `ToolContext` object.** You don't have to create separate tools for each; you can want to share.

```
In [15]: # Define scope levels for state keys (following best practices)
USER_NAME_SCOPE_LEVELS = ("temp", "user", "app")

# This demonstrates how tools can write to session state using tool_context.
# The 'user:' prefix indicates this is user-specific data.
def save_userinfo(
    tool_context: ToolContext, user_name: str, country: str
) -> Dict[str, Any]:
    """
    Tool to record and save user name and country in session state.

    Args:
        user_name: The username to store in session state
        country: The name of the user's country
    """
    # Write to session state using the 'user:' prefix for user data
    tool_context.state["user:name"] = user_name
    tool_context.state["user:country"] = country

    return {"status": "success"}

# This demonstrates how tools can read from session state.
def retrieve_userinfo(tool_context: ToolContext) -> Dict[str, Any]:
    """
    Tool to retrieve user name and country from session state.

    # Read from session state
    user_name = tool_context.state.get("user:name", "Username not found")
    country = tool_context.state.get("user:country", "Country not found")

    return {"status": "success", "user_name": user_name, "country": country}

print("✅ Tools created.")
```

✅ Tools created.

5.2 Creating an Agent with Session State Tools

Now let's create a new agent that has access to our session state management tools:

```
] # Configuration
APP_NAME = "default"
USER_ID = "default"
MODEL_NAME = "gemini-2.5-flash-lite"

# Create an agent with session state tools
root_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="text_chat_bot",
    description="""A text chatbot.
Tools for managing user context:
* To record username and country when provided use `save_userinfo` tool.
* To fetch username and country when required use `retrieve_userinfo` tool.
""",
    tools=[save_userinfo, retrieve_userinfo], # Provide the tools to the agent
)

# Set up session service and runner
session_service = InMemorySessionService()
runner = Runner(agent=root_agent, session_service=session_service, app_name="default")

print("✅ Agent with session state tools initialized!")
```

✅ Agent with session state tools initialized!

5.3 Testing Session State in Action

Let's test how the agent uses session state to remember information across conversation turns:

```
] # Test conversation demonstrating session state
await run_session(
    runner,
    [
        "Hi there, how are you doing today? What is my name?", # Agent shouldn't know the name yet
        "My name is Sam. I'm from Poland.", # Provide name - agent should save it
        "What is my name? Which country am I from?", # Agent should recall from session state
    ],
    "state-demo-session",
)
```

Session: state-demo-session

User > Hi there, how are you doing today? What is my name?

gemini-2.5-flash-lite > Hello! I'm doing great, thank you for asking. I'm sorry, but I don't have access to your name. My memory is limited to our current conversation. If you'd like to tell me your name, I can remember this chat.

User > My name is Sam. I'm from Poland.

WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model e.

gemini-2.5-flash-lite > It is a pleasure to meet you, Sam! I have saved that you are from Poland.

User > What is my name? Which country am I from?

WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model e.

gemini-2.5-flash-lite > Your name is Sam and you are from Poland.

5.4 Inspecting Session State

Let's directly inspect the session state to see what's stored:

```
: # Retrieve the session and inspect its state
session = await session_service.get_session(
    app_name=APP_NAME, user_id=USER_ID, session_id="state-demo-session"
)

print("Session State Contents:")
print(session.state)
print("\n🔍 Notice the 'user:name' and 'user:country' keys storing our data!")
```

Session State Contents:
{'user:name': 'Sam', 'user:country': 'Poland'}

🔍 Notice the 'user:name' and 'user:country' keys storing our data!

5.5 Session State Isolation

As we've already seen, an important characteristic of session state is that it's isolated per session. Let's demonstrate this by starting a new session:

```
: # Start a completely new session - the agent won't know our name
await run_session(
    runner,
    ["Hi there, how are you doing today? What is my name?"],
    "new-isolated-session",
)

# Expected: The agent won't know the name because this is a different session
```

Session: new-isolated-session

User > Hi there, how are you doing today? What is my name?

gemini-2.5-flash-lite > Hello! I'm doing great, thank you for asking. I'd love to tell you your name, but I don't have access to that information. I can save your name and country

5.6 Cross-Session State Sharing

While sessions are isolated by default, you might notice something interesting. Let's check the state of our new session (`new-isolated-session`):

```
: # Check the state of the new session
session = await session_service.get_session(
    app_name=APP_NAME, user_id=USER_ID, session_id="new-isolated-session"
)

print("New Session State:")
print(session.state)

# Note: Depending on implementation, you might see shared state here.
# This is where the distinction between session-specific and user-specific state becomes important.
```

New Session State:
{'user:name': 'Sam', 'user:country': 'Poland'}

🧹 Cleanup

```
]# # Clean up any existing database to start fresh (if Notebook is restarted)
import os

if os.path.exists("my_agent_data.db"):
    os.remove("my_agent_data.db")
print("✅ Cleaned up old database files")
```

✅ Cleaned up old database files



Something went wrong and this page crashed!

If the issue persists, it's likely a problem on our side.

Failed to execute 'removeChild' on 'Node': The node to be removed is not a child of this node.

NotFoundError: Failed to execute 'removeChild' on 'Node': The node to be removed is not a child of this node.
at sp (https://www.kaggle.com/static/assets/vendor.js?v=16e57cdf2074cbfa:241:659501)
at sd (https://www.kaggle.com/static/assets/vendor.js?v=16e57cdf2074cbfa:241:659185)
at sp (https://www.kaggle.com/static/assets/vendor.js?v=16e57cdf2074cbfa:241:660156)
at sf (https://www.kaggle.com/static/assets/vendor.js?v=16e57cdf2074cbfa:241:660790)
at sh (https://www.kaggle.com/static/assets/vendor.js?v=16e57cdf2074cbfa:241:661326)
at sf (https://www.kaggle.com/static/assets/vendor.js?v=16e57cdf2074cbfa:241:660940)
at sh (https://www.kaggle.com/static/assets/vendor.js?v=16e57cdf2074cbfa:241:661053)
at sf (https://www.kaggle.com/static/assets/vendor.js?v=16e57cdf2074cbfa:241:660940)
at sh (https://www.kaggle.com/static/assets/vendor.js?v=16e57cdf2074cbfa:241:661326)
at sf (https://www.kaggle.com/static/assets/vendor.js?v=16e57cdf2074cbfa:241:660940)

Refresh

3B

Day 3b - Agent Memory

Draft guardado

Archivo Edición Vista Corre Escenarios Complementos Ayuda

+ - ✂ 📄 📁 ▶ ☐ Cancelar Run Código ▾

● Sesión

🔧 Section 1: Setup

1.1: Install dependencies

The Kaggle Notebooks environment includes a pre-installed version of the [google-adk](#) library for Python and its required dependencies, so you don't need to install additional packages in this notebook.

To install and use ADK in your own Python development environment outside of this course, you can do so by running:

```
pip install google-adk
```

+ Code + Markdown

[2]:

```
pip install google-adk
```

```
Requirement already satisfied: google-adk in /usr/local/lib/python3.11/dist-packages (1.18.0)
Requirement already satisfied: PyYAML<7.0.0,>=6.0.2 in /usr/local/lib/python3.11/dist-packages (from google-adk) (6.0.3)
Requirement already satisfied: anyio<5.0.0,>=4.9.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (4.11.0)
Requirement already satisfied: authlib<2.0.0,>=1.5.1 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.6.5)
Requirement already satisfied: click<9.0.0,>=8.1.8 in /usr/local/lib/python3.11/dist-packages (from google-adk) (8.3.0)
Requirement already satisfied: fastapi<1.119.0,>=0.115.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.116.1)
Requirement already satisfied: google-api-python-client<3.0.0,>=2.157.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.177.0)
Requirement already satisfied: google-cloud-aiplatform<2.0.0,>=1.125.0 in /usr/local/lib/python3.11/dist-packages (from google-cloud-aiplatform[agent-engines]<2.0.0,>=1.125)
Requirement already satisfied: google-cloud-bigtable>=2.32.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.34.0)
Requirement already satisfied: google-cloud-discoveryengine<0.14.0,>=0.13.12 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.13.12)
Requirement already satisfied: google-cloud-secret-manager<3.0.0,>=2.22.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.25.0)
Requirement already satisfied: google-cloud-spanner<4.0.0,>=3.56.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (3.56.0)
Requirement already satisfied: google-cloud-speech<3.0.0,>=2.30.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.34.0)
Requirement already satisfied: google-cloud-storage<4.0.0,>=3.0.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (3.5.0)
Requirement already satisfied: google-generativeai<2.0.0,>=1.45.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.48.0)
Requirement already satisfied: graphviz<1.0.0,>=0.20.2 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.21)
Requirement already satisfied: mcp<2.0.0,>=1.8.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.20.0)
```

1.2: Configure your Gemini API Key

This notebook uses the [Gemini API](#), which requires authentication.

1. Get your API key

If you don't have one already, create an [API key in Google AI Studio](#).

2. Add the key to Kaggle Secrets

Next, you will need to add your API key to your Kaggle Notebook as a Kaggle User Secret.

1. In the top menu bar of the notebook editor, select `Add-ons` then `Secrets`.
2. Create a new secret with the label `GOOGLE_API_KEY`.
3. Paste your API key into the "Value" field and click "Save".
4. Ensure that the checkbox next to `GOOGLE_API_KEY` is selected so that the secret is attached to the notebook.

3. Authenticate in the notebook

Run the cell below to complete authentication.

`+ Code` `+ Markdown`

[3]:

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Gemini API key setup complete.")
except Exception as e:
    print(
        f"🚨 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}"
    )
```

✅ Gemini API key setup complete.

    Cancel Run `Código` ▾

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

```
from google.adk.agents import LlmAgent
from google.adk.models.google_llm import Gemini
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.adk.memory import InMemoryMemoryService
from google.adk.tools import load_memory, preload_memory
from google.genai import types

print("✅ ADK components imported successfully.")
```

✅ ADK components imported successfully.

`+ Code` `+ Markdown`

1.4: Helper functions

This helper function manages a complete conversation session, handling session creation/retrieval, query processing, and response streaming.

```
] : async def run_session(
    runner_instance: Runner, user_queries: list[str] | str, session_id: str = "default"
):
    """Helper function to run queries in a session and display responses."""
    print(f"\n### Session: {session_id}")

    # Create or retrieve session
    try:
        session = await session_service.create_session(
            app_name=APP_NAME, user_id=USER_ID, session_id=session_id
        )
    except:
        session = await session_service.get_session(
            app_name=APP_NAME, user_id=USER_ID, session_id=session_id
        )

    # Convert single query to list
    if isinstance(user_queries, str):
        user_queries = [user_queries]

    # Process each query
    for query in user_queries:
        print(f"\nUser > {query}")
        query_content = types.Content(role="user", parts=[types.Part(text=query)])

        # Stream agent response
        async for event in runner_instance.run_async(
            user_id=USER_ID, session_id=session.id, new_message=query_content
        ):
            if event.is_final_response() and event.content and event.content.parts:
                text = event.content.parts[0].text
                if text and text != "None":
                    print(f"Model: > {text}")

    print("✅ Helper functions defined.")
```

✅ Helper functions defined.

Section 3: Initialize MemoryService

3.1 Initialize Memory

ADK provides multiple `MemoryService` implementations through the `BaseMemoryService` interface:

- `InMemoryMemoryService` - Built-in service for prototyping and testing (keyword matching, no persistence)
- `VertexAiMemoryBankService` - Managed cloud service with LLM-powered consolidation and semantic search
- **Custom implementations** - You can build your own using databases, though managed services are recommended

For this notebook, we'll use `InMemoryMemoryService` to learn the core mechanics. The same methods work identically with production-ready services like Vertex AI Memory Bank.

+ Code + Markdown

```
10]: memory_service = (  
    InMemoryMemoryService()  
) # ADK's built-in Memory Service for development and testing
```

3.2 Add Memory to Agent

Next, create a simple agent to answer user queries.

```
11]: # Define constants used throughout the notebook  
APP_NAME = "MemoryDemoApp"  
USER_ID = "demo_user"  
  
# Create agent  
user_agent = LlmAgent(  
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),  
    name="MemoryDemoAgent",  
    instruction="Answer user questions in simple words.",  
)  
  
print("✅ Agent created")
```

✅ Agent created

Create Runner

Now provide both Session and Memory services to the `Runner`.

Key configuration:

The `Runner` requires both services to enable memory functionality:

- `session_service` → Manages conversation threads and events
- `memory_service` → Provides long-term knowledge storage

Both services work together: Sessions capture conversations, Memory stores knowledge for retrieval across sessions.

```
# Create Session Service  
session_service = InMemorySessionService() # Handles conversations  
  
# Create runner with BOTH services  
runner = Runner(  
    agent=user_agent,  
    app_name="MemoryDemoApp",  
    session_service=session_service,  
    memory_service=memory_service, # Memory service is now available!  
)  
  
print("✅ Agent and Runner created with memory support!")
```

✅ Agent and Runner created with memory support!

Section 4: Ingest Session Data into Memory

Why should you transfer Session data to Memory?

Now that memory is initialized, you need to populate it with knowledge. When you initialize a `MemoryService`, it starts completely empty. All your conversations are stored in `Sessions`, which makes this information available for long-term recall, you explicitly transfer it to memory using `add_session_to_memory()`.

Here's where managed memory services like Vertex AI Memory Bank shine. **During transfer, they perform intelligent consolidation - extracting key facts while discarding conversational noise.** This consolidation, which is sufficient for learning the mechanics.

Before we can transfer anything, we need data. Let's have a conversation with our agent to populate the session. This conversation will be stored in the `SessionService` just like you learned

```
[43]: # User tells agent about their favorite color
      await run_session(
        runner,
        "My favorite color is blue-green. Can you write a Haiku about it?",
        "conversation-01", # Session ID
      )
```

Session: conversation-01

User > My favorite color is blue-green. Can you write a Haiku about it?
Model: > A cool, calming hue,
Ocean waves and forest deep,
Nature's gentle blend.

```
session = await session_service.get_session(
    app_name=APP_NAME, user_id=USER_ID, session_id="conversation-01"
)

# Let's see what's in the session
print("📄 Session contains:")
for event in session.events:
    text = (
        event.content.parts[0].text[:60]
        if event.content and event.content.parts
        else "(empty)"
    )
    print(f" {event.content.role}: {text}...")
```

📄 Session contains:
user: My favorite color is blue-green. Can you write a Haiku about...
model: A cool, calming hue,
Ocean waves and forest deep,
Nature's g...

+ Code

+ Markdown

Perfect! The session contains our conversation. Now we're ready to transfer it to memory. Call `add_session_to_memory()` and pass the session object. This ingests the conversation

```
# This is the key method!
await memory_service.add_session_to_memory(session)

print("✅ Session added to memory!")
```

✅ Session added to memory!



5.2 Add Load Memory Tool to Agent

Let's start by implementing the reactive pattern. We'll recreate the agent from Section 3, this time adding the `load_memory` tool to its toolkit. Since this is a built-in ADK tool, you simply include it in the tools array without any custom implementation

```
[46]: # Create agent
user_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="MemoryDemoAgent",
    instruction="Answer user questions in simple words. Use load_memory tool if you need to recall past conversations.",
    tools=[
        load_memory
    ], # Agent now has access to Memory and can search it whenever it decides to!
)

print("✅ Agent with load_memory tool created.")
```

✅ Agent with load_memory tool created.

5.3 Update the Runner and Test

Let's now update the Runner to use our new `user_agent` that has the `load_memory` tool. And we'll ask the Agent about the favorite color which we had stored previously in another session.

👉 Since sessions don't share conversation history, the only way the agent can answer correctly is by using the `load_memory` tool to retrieve the information from long-term memory that we manually stored.

+ Code + Markdown

```
[50]: # Create a new runner with the updated agent
runner = Runner(
    agent=user_agent,
    app_name=APP_NAME,
    session_service=session_service,
    memory_service=memory_service,
)

await run_session(runner, "What is my favorite color?", "color-test")
```

Session: color-test

User > What is my favorite color?

5.4 Complete Manual Workflow Test

Let's see the complete workflow in action. We'll have a conversation about a birthday, manually save it to memory, then test retrieval in a new session. This demonstrates the full cycle: **ingest** → **store** →

```
[4]: await run_session(runner, "My birthday is on March 15th.", "birthday-session-01")
```

Session: birthday-session-01

User > My birthday is on March 15th.

Model: > Ok, I will remember that.

Now manually save this session to memory. This is the crucial step that transfers the conversation from short-term session storage to long-term memory storage.

```
[5]: # Manually save the session to memory
birthday_session = await session_service.get_session(
    app_name=APP_NAME, user_id=USER_ID, session_id="birthday-session-01"
)

await memory_service.add_session_to_memory(birthday_session)

print("✅ Birthday session saved to memory!")
```

✅ Birthday session saved to memory!

Here's the crucial test: we'll start a completely new session with a different session ID and ask the agent to recall the birthday.

```
[6]: # Test retrieval in a NEW session
await run_session(
    runner, "When is my birthday?", "birthday-session-02" # Different session ID
)
```

Session: birthday-session-02

User > When is my birthday?

WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates nse.

Model: > Your birthday is on March 15th.

Test it:

1. Ask "What is my favorite color?" in a new session
2. Ask "Tell me a joke" - notice that `preload_memory` still searches memory even though it's unnecessary
3. Which pattern is better for different use cases?

```
user_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="MemoryDemoAgent",
    instruction="Answer user questions in simple words. Use load_memory tool if you need to recall past conversations.",
    tools=[
        preload_memory
    ],
)
runner = Runner(
    agent=user_agent,
    app_name=APP_NAME,
    session_service=session_service,
    memory_service=memory_service,
)

await run_session(runner, "Tell me a Joke", "joke")
```

Session: joke

User > Tell me a Joke
Model: > Why don't scientists trust atoms?

because they make up everything!

5.5 Manual Memory Search

Beyond agent tools, you can also search memories directly in your code. This is useful for:

- Debugging memory contents
- Building analytics dashboards
- Creating custom memory management UIs

The `search_memory()` method takes a text query and returns a `SearchMemoryResponse` with matching memories.

```
# Search for color preferences
search_response = await memory_service.search_memory(
    app_name=APP_NAME, user_id=USER_ID, query="What is the user's favorite color?"
)

print("🔍 Search Results:")
print(f" Found {len(search_response.memories)} relevant memories")
print()

for memory in search_response.memories:
    if memory.content and memory.content.parts:
        text = memory.content.parts[0].text[:80]
        print(f" [{memory.author}]: {text}...")
```

🔍 Search Results:
Found 2 relevant memories

[user]: My favorite color is blue-green. Can you write a Haiku about it?...
[user]: My birthday is on March 15th....

🔥 Your Turn: Test Different Queries

Try these searches to understand how keyword matching works with `InMemoryMemoryService`:

1. "what color does the user like"
2. "haiku"
3. "age"
4. "preferred hue"

Notice which queries return results and which don't. What pattern do you observe?

💡 **Key Insight:** Memory search is grounded in reality - agents can't hallucinate memories that don't exist.

```
session_service = InMemorySessionService()
runner = Runner(
    agent=user_agent,
    app_name="MemoryDemoApp",
    session_service=session_service,
    memory_service=memory_service,
)

await run_session(runner, "preferred hue")
```

Session: default

User > preferred hue
Model: > My preferred hue is a calming deep hue, like the ocean and forest combined. It brings a sense of peace.

6.2 Automatic Memory Storage with Callbacks

For automatic memory storage, we'll use `after_agent_callback`. This function triggers every time the agent finishes a turn, then calls `add_session_to_memory()` to persist the conversation.

But here's the challenge: how does our callback function actually access the memory service and current session? That's where `callback_context` comes in.

When you define a callback function, ADK automatically passes a special parameter called `callback_context` to it. The `callback_context` provides access to the Memory Service and other

How we'll use it: In our callback, we'll access the memory service and current session to automatically save conversation data after each turn.

🔔 **Important:** You don't create this context - ADK creates it and passes it to your callback automatically when the callback runs.

```
async def auto_save_to_memory(callback_context):
    """Automatically save session to memory after each agent turn."""
    await callback_context.invocation_context.memory_service.add_session_to_memory(
        callback_context.invocation_context.session
    )
```

```
print("✅ Callback created.")
```

✅ Callback created.

6.3 Create an Agent: Callback and PreLoad Memory Tool

Now create an agent that combines:

- **Automatic storage:** `after_agent_callback` saves conversations
- **Automatic retrieval:** `preload_memory` loads memories

This creates a fully automated memory system with zero manual intervention.

```
# Agent with automatic memory saving
auto_memory_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="AutoMemoryAgent",
    instruction="Answer user questions.",
    tools=[preload_memory],
    after_agent_callback=auto_save_to_memory, # Saves after each turn!
)

print("✅ Agent created with automatic memory saving!")
```

✅ Agent created with automatic memory saving!

6.4 Create a Runner and Test The Agent

Time to test! Create a Runner with the auto-memory agent, connecting the session and memory services.

```
# Create a runner for the auto-save agent
# This connects our automated agent to the session and memory services
auto_runner = Runner(
    agent=auto_memory_agent, # Use the agent with callback + preload_memory
    app_name=APP_NAME,
    session_service=session_service, # Same services from Section 3
    memory_service=memory_service,
)

print("✅ Runner created.")
```

✅ Runner created.

```
# Test 1: Tell the agent about a gift (first conversation)
# The callback will automatically save this to memory when the turn completes
await run_session(
    auto_runner,
    "I gifted a new toy to my nephew on his 1st birthday!",
    "auto-save-test",
)

# Test 2: Ask about the gift in a NEW session (second conversation)
# The agent should retrieve the memory using preload_memory and answer correctly
await run_session(
    auto_runner,
    "What did I gift my nephew?",
    "auto-save-test-2", # Different session ID - proves memory works across sessions!
)
```

Session: auto-save-test

```
User > I gifted a new toy to my nephew on his 1st birthday!
Model: > That's wonderful! A 1st birthday is such a special milestone. I hope your nephew enjoys his new toy!
```

Session: auto-save-test-2

```
User > What did I gift my nephew?
Model: > You gifted your nephew a new toy.
```



Día 3 (Ingeniería de Contexto: Sesiones y Memoria)

Bienvenidos al Día 3.

Este whitepaper explora la ingeniería del contexto como la práctica de ensamblar y gestionar dinámicamente la información dentro de la ventana contextual de un agente para crear experiencias AI con estado y personalizadas. Define Sesiones como el contenedor para la historia de una conversación inmediata y única, y Memoria como el mecanismo de persistencia a largo plazo.

En los codelabs, aprenderás a hacer que los agentes tengan estado gestionando el historial de conversaciones mediante ingeniería de contexto en ADK y memoria de trabajo dentro de una sesión, permitiendo que tu agente recuerde el contexto y mantenga conversaciones coherentes de varios turnos. En el segundo cuaderno, le darás a tu agente memoria a largo plazo que persiste en diferentes sesiones.

Día 3 Trabajo

Completar la Unidad 3 - "Ingeniería del Contexto: Sesiones y Memoria":

- ☒ Escucha el episodio resumen del podcast de esta unidad.
- ☒ Para complementar el podcast, lee el documento técnico "Ingeniería del contexto: Sesiones y memoria".
- ☒ Completa estos codelabs en Kaggle:
 - ☒ Construye agentes con estado y realiza ingeniería de contexto.
 - ☒ Explora cómo usar la memoria con tu agente.

[Opcional]: Mira la retransmisión en directo grabada de YouTube. Kanchana Patlolla y Anant Nawalgaria estuvieron acompañadas por el autor de Codelab, Kristopher Overholt, junto con invitados de Google: Steven Johnson, Kimberly Milam y Julia Wiesinger, y el ponente externo Jay Alammar de Cohere.

DIA 4

Day 4a - Agent Observability

Draft guardado

Archivo Edición Vista Corre Escenarios Complementos Ayuda

+ - 🔍 📄 📁 ▶️ ❌ Cancelar Run Código ▾

Section 1: Setup

1.1: Install dependencies

The Kaggle Notebooks environment includes a pre-installed version of the [google-adk](#) library for Python and its required dependencies, so you don't need to install additional packages in this notebook.

To install and use ADK in your own Python development environment outside of this course, you can do so by running:

```
pip install google-adk
```

+ Code + Markdown

[2]:

```
pip install google-adk
```

```
Requirement already satisfied: google-adk in /usr/local/lib/python3.11/dist-packages (1.18.0)
Requirement already satisfied: PyYAML<7.0.0,>=6.0.2 in /usr/local/lib/python3.11/dist-packages (from google-adk) (6.0.3)
Requirement already satisfied: anyio<5.0.0,>=4.9.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (4.11.0)
Requirement already satisfied: authlib<2.0.0,>=1.5.1 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.6.5)
Requirement already satisfied: click<9.0.0,>=8.1.8 in /usr/local/lib/python3.11/dist-packages (from google-adk) (8.3.0)
Requirement already satisfied: fastapi<1.119.0,>=0.115.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.116.1)
Requirement already satisfied: google-api-python-client<3.0.0,>=2.157.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.177.0)
Requirement already satisfied: google-cloud-aiplatform<2.0.0,>=1.125.0 in /usr/local/lib/python3.11/dist-packages (from google-cloud-aiplatform[agent-engines]<2.0)
Requirement already satisfied: google-cloud-bigtable>=2.32.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.34.0)
Requirement already satisfied: google-cloud-discoveryengine<0.14.0,>=0.13.12 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.13.12)
Requirement already satisfied: google-cloud-secret-manager<3.0.0,>=2.22.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.25.0)
Requirement already satisfied: google-cloud-spanner<4.0.0,>=3.56.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (3.56.0)
Requirement already satisfied: google-cloud-speech<3.0.0,>=2.30.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.34.0)
Requirement already satisfied: google-cloud-storage<4.0.0,>=3.0.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (3.5.0)
Requirement already satisfied: google-genai<2.0.0,>=1.45.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.48.0)
Requirement already satisfied: graphviz<1.0.0,>=0.20.2 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.21)
Requirement already satisfied: mcp<2.0.0,>=1.8.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.20.0)
Requirement already satisfied: opentelemetry-api<=1.37.0,>=1.37.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.37.0)
Requirement already satisfied: opentelemetry-exporter-gcp-logging<2.0.0,>=1.9.0a0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.11.0a0)
Requirement already satisfied: opentelemetry-exporter-gcp-monitoring<2.0.0,>=1.9.0a0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.11.0a0)
Requirement already satisfied: opentelemetry-exporter-otlp<=1.37.0,>=1.37.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.37.0)
```

🔑 1.2: Configure your Gemini API Key

This notebook uses the [Gemini API](#), which requires an API key.

1. Get your API key

If you don't have one already, create an [API key in Google AI Studio](#).

2. Add the key to Kaggle Secrets

Next, you will need to add your API key to your Kaggle Notebook as a Kaggle User Secret.

1. In the top menu bar of the notebook editor, select `Add-ons` then `Secrets`.
2. Create a new secret with the label `GOOGLE_API_KEY`.
3. Paste your API key into the "Value" field and click "Save".
4. Ensure that the checkbox next to `GOOGLE_API_KEY` is selected so that the secret is attached to the notebook.

3. Authenticate in the notebook

Run the cell below to access the `GOOGLE_API_KEY` you just saved and set it as an environment variable for the notebook to use:

+ Code

+ Markdown

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Setup and authentication complete.")
except Exception as e:
    print(
        f"🔑 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}"
    )
```

✅ Setup and authentication complete.

🔧 1.3: Set up logging and cleanup old files

Let's configure logging for our debugging session. The following cell makes sure we also capture other log levels, like `DEBUG`.

```
import logging
import os

# Clean up any previous logs
for log_file in ["logger.log", "web.log", "tunnel.log"]:
    if os.path.exists(log_file):
        os.remove(log_file)
        print(f"🧹 Cleaned up {log_file}")

# Configure logging with DEBUG log level.
logging.basicConfig(
    filename="logger.log",
    level=logging.DEBUG,
    format="%(filename)s:%(lineno)s %(levelname)s:%(message)s",
)

print("✅ Logging configured")
```

✅ Logging configured

1.4: Set up proxy and tunneling

We'll use a proxy to access the ADK web UI from within the Kaggle Notebooks environment. If you are running this outside the Kaggle environment, you don't need to do this.

```
from IPython.core.display import display, HTML
from jupyter_server.serverapp import list_running_servers

# Gets the proxied URL in the Kaggle Notebooks environment
def get_adk_proxy_url():
    PROXY_HOST = "https://kkg-production.jupyter-proxy.kaggle.net"
    ADK_PORT = "8000"

    servers = list(list_running_servers())
    if not servers:
        raise Exception("No running Jupyter servers found.")

    baseURL = servers[0]["base_url"]

    try:
        path_parts = baseURL.split("/")
        kernel = path_parts[2]
        token = path_parts[3]
    except IndexError:
        raise Exception(f"Could not parse kernel/token from base URL: {baseURL}")

    url_prefix = f"/k/{kernel}/{token}/proxy/proxy/{ADK_PORT}"
    url = f"{PROXY_HOST}{url_prefix}"

    styled_html = f"""
<div style="padding: 15px; border: 2px solid #f0ad4e; border-radius: 8px; background-color: #fef9f0; margin: 20px 0;">
  <div style="font-family: sans-serif; margin-bottom: 12px; color: #333; font-size: 1.1em;">
    <strong>⚠ IMPORTANT: Action Required</strong>
  </div>
  <div style="font-family: sans-serif; margin-bottom: 15px; color: #333; line-height: 1.5;">
    The ADK web UI is <strong>not running yet</strong>. You must start it in the next cell.
    <ol style="margin-top: 10px; padding-left: 20px;">
      <li style="margin-bottom: 5px;"><strong>Run the next cell</strong> (the one with <code>!adk web ...</code>) to start the ADK web UI.</li>
      <li style="margin-bottom: 5px;">Wait for that cell to show it is "Running" (it will not "complete").</li>
      <li>Once it's running, <strong>return to this button</strong> and click it to open the UI.</li>
    </ol>
    <em style="font-size: 0.9em; color: #555;">(If you click the button before running the next cell, you will get a 500 error.)</em>
  </div>
  <a href='{url}' target='_blank' style="
    display: inline-block; background-color: #1a73e8; color: white; padding: 10px 20px;
    text-decoration: none; border-radius: 25px; font-family: sans-serif; font-weight: 500;
    box-shadow: 0 2px 5px rgba(0,0,0,0.2); transition: all 0.2s ease;">
    Open ADK Web UI (after running cell below) ↗
  </a>
</div>
"""

display(HTML(styled_html))

return url_prefix

print("✅ Helper functions defined.")
```

✅ Helper functions defined.

+ Code + Markdown

Section 2: Hands-On Debugging with ADK Web UI

2.1: Create a "Research Paper Finder" Agent

Our goal: Build a research paper finder agent that helps users find academic papers on any topic.

But first, let's intentionally create an incorrect version of the agent to practice debugging! We'll start by creating a new agent folder using the `adk create` CLI command.

```
!adk create research-agent --model gemini-2.5-flash-lite --api_key $GOOGLE_API_KEY
```

+ Code

+ Markdown

```
!adk create research-agent --model gemini-2.5-flash-lite --api_key $GOOGLE_API_KEY
```

Agent created in `/kaggle/working/research-agent`:

- `.env`
- `__init__.py`
- `agent.py`

Agent definition

Next, let's create our root agent.

- We'll configure it as an `LlmAgent`, give it a name, model and instruction.
- The `root_agent` gets the user prompt and delegates the search to the `google_search_agent`.
- Then, the agent uses the `count_papers` tool to count the number of papers returned.

👉 Pay attention to the root agent's instructions and the `count_papers` tool parameter!

```
%%writefile research-agent/agent.py

from google.adk.agents import LlmAgent
from google.adk.models.google_llm import Gemini
from google.adk.tools.agent_tool import AgentTool
from google.adk.tools.google_search_tool import google_search

from google.genai import types
from typing import List

retry_config = types.HttpRetryOptions(
    attempts=5, # Maximum retry attempts
    exp_base=7, # Delay multiplier
    initial_delay=1,
    http_status_codes=[429, 500, 503, 504], # Retry on these HTTP errors
)

# ---- Intentionally pass incorrect datatype - `str` instead of `List[str]` ----
def count_papers(papers: str):
    """
    This function counts the number of papers in a list of strings.
    Args:
        papers: A list of strings, where each string is a research paper.
    Returns:
        The number of papers in the list.
    """
    return len(papers)

# Google Search agent
google_search_agent = LlmAgent(
    name="google_search_agent",
```

```

# Google Search agent
google_search_agent = LlmAgent(
    name="google_search_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    description="Searches for information using Google search",
    instruction="""Use the google_search tool to find information on the given topic. Return the raw search results.
    If the user asks for a list of papers, then give them the list of research papers you found and not the summary."""
    tools=[google_search]
)

# Root agent
root_agent = LlmAgent(
    name="research_paper_finder_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    instruction="""Your task is to find research papers and count them.

    You MUST ALWAYS follow these steps:
    1) Find research papers on the user provided topic using the 'google_search_agent'.
    2) Then, pass the papers to 'count_papers' tool to count the number of papers returned.
    3) Return both the list of research papers and the total number of papers.
    """,
    tools=[AgentTool(agent=google_search_agent), count_papers]
)

```

Overwriting research-agent/agent.py

2.2: Run the agent

Let's now run our agent with the `adk web --log_level DEBUG` CLI command.

📌 **The key here is** `--log_level DEBUG` - this shows us:

- **Full LLM Prompts:** The complete request sent to the language model, including system instructions, h
- Detailed API responses from services.
- Internal state transitions and variable values.

Other log levels include: INFO, ERROR and WARNING.

Get the proxied URL to access the ADK web UI in the Kaggle Notebooks environment:

+ Code

+ Markdown

```
url_prefix = get_adk_proxy_url()
```

⚠️ IMPORTANT: Action Required

The ADK web UI is **not running yet**. You must start it in the next cell.

1. **Run the next cell** (the one with `!adk web ...`) to start the ADK web UI.
2. Wait for that cell to show it is "Running" (it will not "complete").
3. Once it's running, **return to this button** and click it to open the UI.

(If you click the button before running the next cell, you will get a 500 error.)

Open ADK Web UI (after running cell below) ↗

⚠ IMPORTANT: Action Required

The ADK web UI is **not running yet**. You must start it in the next cell.

1. **Run the next cell** (the one with `!adk web ...`) to start the ADK web UI.
2. Wait for that cell to show it is "Running" (it will not "complete").
3. Once it's running, **return to this button** and click it to open the UI.

(If you click the button before running the next cell, you will get a 500 error.)

Open ADK Web UI (after running cell below) ↗

+ Code

+ Markdown

Now you can start the ADK web UI with the `--log_level` parameter.

Note: The following cell will not "complete", but will remain running and serving the ADK web UI until you manually stop the cell.

```
!adk web --log_level DEBUG --url_prefix {url_prefix}
```

```
File "/usr/local/lib/python3.11/dist-packages/tenacity/asyncio/_init_.py", line 114, in __call__
    result = await fn(*args, **kwargs)
    ~~~~~^~~~~~
File "/usr/local/lib/python3.11/dist-packages/google/genai/_api_client.py", line 1255, in _async_request_once
    await errors.APIError.raise_for_async_response(response)
File "/usr/local/lib/python3.11/dist-packages/google/genai/errors.py", line 166, in raise_for_async_response
    raise ClientError(status_code, response_json, response)
google.genai.errors.ClientError: 429 RESOURCE_EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: https://ai.google.dev/gemini-api/docs/rate-limits. To monitor your current usage, head to: https://ai.dev/rate-limit.', 'status': 'RESOURCE_EXHAUSTED', 'details': [{'@type': 'type.googleapis.com/google.rpc.Help', 'links': [{'description': 'Learn more about Gemini API quotas', 'url': 'https://ai.google.dev/gemini-api/docs/rate-limits'}], '@type': 'type.googleapis.com/google.rpc.QuotaFailure', 'violations': [{'quotaMetric': 'generativelanguage.googleapis.com/generate_content_free_tier_requests', 'quotaId': 'GenerateRequestsPerDayPerProjectPerModel-FreeTier', 'quotaDimensions': {'model': 'gemini-2.5-flash-lite', 'location': 'global'}, 'quotaValue': '20'}]}, {'@type': 'type.googleapis.com/google.rpc.RetryInfo', 'retryDelay': '53s'}}]}
INFO: 35.191.121.123:0 - "GET /apps/research-agent/users/user/sessions/9dc9ca2b-eddc-49d0-b812-610ef9f52c50 HTTP/1.1" 200 OK
INFO: 35.191.16.162:0 - "GET /debug/trace/session/9dc9ca2b-eddc-49d0-b812-610ef9f52c50 HTTP/1.1" 200 OK
INFO: 35.191.16.161:0 - "GET /debug/trace/session/9dc9ca2b-eddc-49d0-b812-610ef9f52c50 HTTP/1.1" 200 OK
```

Evento 4 de 4

EventoPeticiónRespuesta

google_search_agent

research_paper_finder_agent

count_papers

Contenido:

Partes:

0:

funciónRespuesta:

id:

Nombre:

Respuesta:

Resultado:

rol:

invocaciónId:

Autor:

Acciones:

estadoDelta:

artefactDelta:

requestedAuthConfigs:

solicitedToolConfirmations:

id:

Marca temporal:

título:

ID DE SESIÓN 9dc9ca2b-eddc-49d0-b812-610ef9f52c50 ID DE USUARIO usuario usuario

Transmisión de Tokens Nueva sesión

Encuentra el último artículo sobre computación cuántica

google_search_agent

google_search_agent

count_papers

count_papers

Encuentra el último artículo sobre computación cuántica

Escribe un mensaje...

2.5: Debug through local Logs

Optionally, you can also examine the local DEBUG logs to find the root cause. Run the following cell to print the contents of the log file. Look for detailed logs like:

```
DEBUG - google_adk.models.google_llm - LLM Request: ...  
DEBUG - google_adk.models.google_llm - LLM Response: ...
```

```
# Check the DEBUG Logs from the broken agent  
print("🔍 Examining web server logs for debugging clues...\n")  
!cat logger.log
```

🔍 Examining web server logs for debugging clues...

cat: logger.log: No such file or directory

Other Observability questions you can now answer from logs and adk web:

3.2: To make things more concrete, what does a Plugin look like?

```
print("----- EXAMPLE PLUGIN - DOES NOTHING ----- ")  
  
import logging  
from google.adk.agents.base_agent import BaseAgent  
from google.adk.agents.callback_context import CallbackContext  
from google.adk.models.llm_request import LlmRequest  
from google.adk.plugins.base_plugin import BasePlugin  
  
# Applies to all agent and model calls  
class CountInvocationPlugin(BasePlugin):  
    """A custom plugin that counts agent and tool invocations."""  
  
    def __init__(self) -> None:  
        """Initialize the plugin with counters."""  
        super().__init__(name="count_invocation")  
        self.agent_count: int = 0  
        self.tool_count: int = 0  
        self.llm_request_count: int = 0  
  
    # Callback 1: Runs before an agent is called. You can add any custom logic here.  
    async def before_agent_callback(  
        self, *, agent: BaseAgent, callback_context: CallbackContext  
    ) -> None:  
        """Count agent runs."""  
        self.agent_count += 1  
        logging.info(f"[Plugin] Agent run count: {self.agent_count}")  
  
    # Callback 2: Runs before a model is called. You can add any custom logic here.  
    async def before_model_callback(  
        self, *, callback_context: CallbackContext, llm_request: LlmRequest  
    ) -> None:  
        """Count LLM requests."""  
        self.llm_request_count += 1  
        logging.info(f"[Plugin] LLM request count: {self.llm_request_count}")
```

----- EXAMPLE PLUGIN - DOES NOTHING -----

[+ Code](#)

[+ Markdown](#)

3.3: ADK's built-in `LoggingPlugin`

But you don't have to define all the callbacks and plugins to capture *standard* Observability data in ADK. Instead, ADK provides a built-in **LoggingPlugin** that automatically captures

- 🗨️ User messages and agent responses
- ⌚ Timing data for performance analysis
- 🗨️ LLM requests and responses for debugging
- 🔧 Tool calls and results
- ✅ Complete execution traces

Agent definition

Let's use the same agent from the previous demo - the Research paper finder!

```
from google.adk.agents import LlmAgent
from google.adk.models.google_llm import Gemini
from google.adk.tools.agent_tool import AgentTool
from google.adk.tools.google_search_tool import google_search

from google.genai import types
from typing import List

retry_config = types.HttpRetryOptions(
    attempts=5, # Maximum retry attempts
    exp_base=7, # Delay multiplier
    initial_delay=1,
    http_status_codes=[429, 500, 503, 504], # Retry on these HTTP errors
)

def count_papers(papers: List[str]):
    """
    This function counts the number of papers in a list of strings.
    Args:
        papers: A list of strings, where each string is a research paper.
    Returns:
        The number of papers in the list.
    """
    return len(papers)
```

```
)  
  
print("✅ Agent created")
```

✅ Agent created

3.4: Add LoggingPlugin to Runner

The following code creates the `InMemoryRunner`. This is used to programmatically invoke the agent.

To use `LoggingPlugin` in the above research agent,

1. Import the plugin
2. Add it when initializing the `InMemoryRunner`.

+ Code

+ Markdown

```
from google.adk.runners import InMemoryRunner  
from google.adk.plugins.logging_plugin import (  
    LoggingPlugin,  
) # <---- 1. Import the Plugin  
from google.genai import types  
import asyncio  
  
runner = InMemoryRunner(  
    agent=research_agent_with_plugin,  
    plugins=[  
        LoggingPlugin()  
    ], # <---- 2. Add the plugin. Handles standard Observability logging across ALL agents  
)  
  
print("✅ Runner configured")
```

✅ Runner configured

+ Code

+ Markdown

```

print("🚀 Running agent with LoggingPlugin...")
print("📊 Watch the comprehensive logging output below:\n")

response = await runner.run_debug("Find recent papers on quantum computing")

```

🚀 Running agent with LoggingPlugin...
 📊 Watch the comprehensive logging output below:

Created new session: debug_session_id

User > Find recent papers on quantum computing

```

[logging_plugin] 🚀 USER MESSAGE RECEIVED
[logging_plugin] Invocation ID: e-a3ffe1a1-b6d0-45af-918e-f74203f7cd93
[logging_plugin] Session ID: debug_session_id
[logging_plugin] User ID: debug_user_id
[logging_plugin] App Name: InMemoryRunner
[logging_plugin] Root Agent: research_paper_finder_agent
[logging_plugin] User Content: text: 'Find recent papers on quantum computing'
[logging_plugin] ⚡ INVOCATION STARTING
[logging_plugin] Invocation ID: e-a3ffe1a1-b6d0-45af-918e-f74203f7cd93
[logging_plugin] Starting Agent: research_paper_finder_agent
[logging_plugin] 🏠 AGENT STARTING
[logging_plugin] Agent Name: research_paper_finder_agent
[logging_plugin] Invocation ID: e-a3ffe1a1-b6d0-45af-918e-f74203f7cd93
[logging_plugin] 🌸 LLM REQUEST
[logging_plugin] Model: gemini-2.5-flash-lite
[logging_plugin] Agent: research_paper_finder_agent
[logging_plugin] System Instruction: 'Your task is to find research papers and count them.

```

You must follow these steps:

- 1) Find research papers on the user provided topic using the 'google_search_agent'.
- 2) Then, pass the p...

```

[logging_plugin] Available Tools: ['google_search_agent', 'count_papers']
[logging_plugin] 🌸 LLM RESPONSE
[logging_plugin] Agent: research_paper_finder_agent
[logging_plugin] Content: function_call: google_search_agent
[logging_plugin] Token Usage - Input: 242, Output: 21
[logging_plugin] 🚩 EVENT YIELDED
[logging_plugin] Event ID: def72717-171b-41fa-84ee-2af10ec80d52
[logging_plugin] Author: research_paper_finder_agent
[logging_plugin] Content: function_call: google_search_agent
[logging_plugin] Final Response: False
[logging_plugin] Function Calls: ['google_search_agent']
[logging_plugin] 🛠️ TOOL STARTING
[logging_plugin] Tool Name: google_search_agent

```

```

You must follow these steps:
1) Find research papers on the user provided topic using the 'google_search_agent'.
2) Then, pass the p...'
[logging_plugin] Available Tools: ['google_search_agent', 'count_papers']
[logging_plugin] LLM RESPONSE
[logging_plugin] Agent: research_paper_finder_agent
[logging_plugin] Content: text: 'Here are the research papers on quantum computing:

* Recent advancements in quantum computing are heavily focused on improving qubit stability and error correction, paving the way for mo
[logging_plugin] Token Usage - Input: 1300, Output: 517
[logging_plugin] EVENT YIELDED
[logging_plugin] Event ID: 2ba561c9-849e-43e0-855e-5f90b5b77d5f
[logging_plugin] Author: research_paper_finder_agent
[logging_plugin] Content: text: 'Here are the research papers on quantum computing:

* Recent advancements in quantum computing are heavily focused on improving qubit stability and error correction, paving the way for mo
[logging_plugin] Final Response: True
research_paper_finder_agent > Here are the research papers on quantum computing:

* Recent advancements in quantum computing are heavily focused on improving qubit stability and error correction, paving the way for mo
individual qubits to "logical qubits," which are encoded using multiple imperfect qubits to achieve higher fidelity and protection against
* Key developments and trends include:
* Enhanced Qubit Coherence: Princeton engineers have developed a superconducting qubit with a stability three times longer than previou
rough, detailed in Nature, uses a tantalum-silicon design to reduce energy loss. This new qubit design is compatible with systems from Go
* Progress in Error Correction: Companies like Google, QueRa, and IBM, along with CSIRO, have made substantial progress in quantum error
is approximately ten times more efficient than previous methods, offering a clearer path toward running quantum circuits with billions of
* Scaling Up Processors: Quantum chips with over 100 qubits are already available and are being used for evaluation. IBM plans to intro
by 2033. Google's Willow quantum chip, unveiled in December 2024, features 105 qubits. IBM is also working on a quantum-centric supercompu
puting infrastructure.
* Quantum Algorithms and Software: Alongside hardware advancements, there's significant research and development in quantum software an
eferred software stack for quantum computing development. Research is also exploring quantum-inspired technology to train AI faster and si
* Interdisciplinary Applications: Quantum computing is expected to revolutionize fields such as medicine, chemistry, materials science,
eakthroughs in drug discovery, advanced medical research, and enhanced AI system safety.
* The year 2025 has been designated as the International Year of Quantum Science and Technology, highlighting the global focus on this

A total of 8 papers were found.
[logging_plugin] AGENT COMPLETED
[logging_plugin] Agent Name: research_paper_finder_agent
[logging_plugin] Invocation ID: e-07403ae6-c702-4f19-b799-356822c647a5
[logging_plugin] INVOCATION COMPLETED
[logging_plugin] Invocation ID: e-07403ae6-c702-4f19-b799-356822c647a5
[logging_plugin] Final Agent: research_paper_finder_agent

```

DIA 4B

Section 1: Setup

Before we begin our evaluation journey, let's set up our environment.

1.1: Install dependencies

The Kaggle Notebooks environment includes a pre-installed version of the [google-adk](#) library for Python and its required dependencies, so you don't need to install additional packages in this notebook.

To install and use ADK in your own Python development environment outside of this course, you can do so by running:

```
pip install google-adk
```

+ Code + Markdown

```
[2]: pip install google-adk

Requirement already satisfied: google-adk in /usr/local/lib/python3.11/dist-packages (1.18.0)
Requirement already satisfied: PyYAML<7.0.0,>=6.0.2 in /usr/local/lib/python3.11/dist-packages (from google-adk) (6.0.3)
Requirement already satisfied: anyio<5.0.0,>=4.9.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (4.11.0)
Requirement already satisfied: authlib<2.0.0,>=1.5.1 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.6.5)
Requirement already satisfied: click<9.0.0,>=8.1.8 in /usr/local/lib/python3.11/dist-packages (from google-adk) (8.3.0)
Requirement already satisfied: fastapi<1.119.0,>=0.115.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.116.1)
Requirement already satisfied: google-api-python-client<3.0.0,>=2.157.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.177.0)
Requirement already satisfied: google-cloud-aiplatform<2.0.0,>=1.125.0 in /usr/local/lib/python3.11/dist-packages (from google-cloud-aiplatform[agent-engine]>=1.125.0->google-adk) (1.125.0)
Requirement already satisfied: google-cloud-bigtable>=2.32.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.34.0)
Requirement already satisfied: google-cloud-discoveryengine<0.14.0,>=0.13.12 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.13.12)
Requirement already satisfied: google-cloud-secret-manager<3.0.0,>=2.22.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.25.0)
Requirement already satisfied: google-cloud-spanner<4.0.0,>=3.56.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (3.56.0)
Requirement already satisfied: google-cloud-speech<3.0.0,>=2.30.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.34.0)
Requirement already satisfied: google-cloud-storage<4.0.0,>=3.0.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (3.5.0)
Requirement already satisfied: google-generativeai<2.0.0,>=1.45.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.48.0)
Requirement already satisfied: graphviz<1.0.0,>=0.20.2 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.21)
Requirement already satisfied: mcp<2.0.0,>=1.8.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.20.0)
```

🔑 1.2: Configure your Gemini API Key

This notebook uses the [Gemini API](#), which requires an API key.

1. Get your API key

If you don't have one already, create an [API key in Google AI Studio](#).

2. Add the key to Kaggle Secrets

Next, you will need to add your API key to your Kaggle Notebook as a Kaggle User Secret.

1. In the top menu bar of the notebook editor, select **Add-ons** then **Secrets**.
2. Create a new secret with the label `GOOGLE_API_KEY`.
3. Paste your API key into the "Value" field and click "Save".
4. Ensure that the checkbox next to `GOOGLE_API_KEY` is selected so that the secret is attached to the notebook.

3. Authenticate in the notebook

Run the cell below to access the `GOOGLE_API_KEY` you just saved and set it as an environment variable for the notebook to use:

+ Code

+ Markdown

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Setup and authentication complete.")
except Exception as e:
    print(
        f"🔑 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}"
    )
```

✅ Setup and authentication complete.

1.3: Set up proxy and tunneling

We'll use a proxy to access the ADK web UI from within the Kaggle Notebooks environment. If you are running this outside the Kaggle environment, you don't

```
from IPython.core.display import display, HTML
from jupyter_server.serverapp import list_running_servers

# Gets the proxied URL in the Kaggle Notebooks environment
def get_adk_proxy_url():
    PROXY_HOST = "https://kbb-production.jupyter-proxy.kaggle.net"
    ADK_PORT = "8000"

    servers = list(list_running_servers())
    if not servers:
        raise Exception("No running Jupyter servers found.")

    baseURL = servers[0]["base_url"]

    try:
        path_parts = baseURL.split("/")
        kernel = path_parts[2]
        token = path_parts[3]
    except IndexError:
        raise Exception(f"Could not parse kernel/token from base URL: {baseURL}")

    url_prefix = f"/k/{kernel}/{token}/proxy/proxy/{ADK_PORT}"
    url = f"{PROXY_HOST}{url_prefix}"

    styled_html = f"""
<div style="padding: 15px; border: 2px solid #f0ad4e; border-radius: 8px; background-color: #fef9f0; margin: 20px 0;">
  <div style="font-family: sans-serif; margin-bottom: 12px; color: #333; font-size: 1.1em;">
    <strong>⚠ IMPORTANT: Action Required</strong>
  </div>
```

```
    styled_html = f"""
<div style="padding: 15px; border: 2px solid #f0ad4e; border-radius: 8px; background-color: #fef9f0; margin: 20px 0;">
  <div style="font-family: sans-serif; margin-bottom: 12px; color: #333; font-size: 1.1em;">
    <strong>⚠ IMPORTANT: Action Required</strong>
  </div>
  <div style="font-family: sans-serif; margin-bottom: 15px; color: #333; line-height: 1.5;">
    The ADK web UI is <strong>not running yet</strong>. You must start it in the next cell.
    <ol style="margin-top: 10px; padding-left: 20px;">
      <li style="margin-bottom: 5px;"><strong>Run the next cell</strong> (the one with <code>!adk web ...</code>) to start the ADK web UI..
      <li style="margin-bottom: 5px;">Wait for that cell to show it is "Running" (it will not "complete").</li>
      <li>Once it's running, <strong>return to this button</strong> and click it to open the UI.</li>
    </ol>
    <em style="font-size: 0.9em; color: #555;">(If you click the button before running the next cell, you will get a 500 error.)</em>
  </div>
  <a href='{url}' target='_blank' style="
    display: inline-block; background-color: #1a73e8; color: white; padding: 10px 20px;
    text-decoration: none; border-radius: 25px; font-family: sans-serif; font-weight: 500;
    box-shadow: 0 2px 5px rgba(0,0,0,0.2); transition: all 0.2s ease;">
    Open ADK Web UI (after running cell below) ↗
  </a>
</div>
"""

display(HTML(styled_html))

return url_prefix

print("✅ Helper functions defined.")
```

✅ Helper functions defined.

Section 2: Create a Home Automation Agent

Let's create the agent that will be the center of our evaluation story. This home automation agent seems perfect in basic tests but has hidden flaws we'll discover through comprehensive evaluation. Run the `adk create` CLI command to set up the project scaffolding.

```
!adk create home_automation_agent --model gemini-2.5-flash-lite --api_key $GOOGLE_API_KEY
```

Agent created in /kaggle/working/home_automation_agent:

```
- .env
- __init__.py
- agent.py
```

```
%writefile home_automation_agent/agent.py

from google.adk.agents import LlmAgent
from google.adk.models.google_llm import Gemini

from google.genai import types

# Configure Model Retry on errors
retry_config = types.HttpRetryOptions(
    attempts=5, # Maximum retry attempts
    exp_base=7, # Delay multiplier
    initial_delay=1,
    http_status_codes=[429, 500, 503, 504], # Retry on these HTTP errors
)

def set_device_status(location: str, device_id: str, status: str) -> dict:
    """Sets the status of a smart home device.

    Args:
        location: The room where the device is located.
        device_id: The unique identifier for the device.
        status: The desired status, either 'ON' or 'OFF'.

    Returns:
        A dictionary confirming the action.
    """
    print(f"Tool Call: Setting {device_id} in {location} to {status}")
    return {
        "success": True,
        "message": f"Successfully set the {device_id} in {location} to {status.lower()}."
    }

# This agent has DELIBERATE FLAWS that we'll discover through evaluation!
root_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="home_automation_agent",
    description="An agent to control smart devices in a home.",
    instruction="""You are a home automation assistant. You control ALL smart devices in the house.

    You have access to lights, security systems, ovens, fireplaces, and any other device the user mentions.
    Always try to be helpful and control whatever device the user asks for.

    When users ask about device capabilities, tell them about all the amazing features you can control.""",
    tools=[set_device_status]
```

```
root_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="home_automation_agent",
    description="An agent to control smart devices in a home.",
    instruction="""You are a home automation assistant. You control ALL smart devices in the house.

    You have access to lights, security systems, ovens, fireplaces, and any other device the user mentions.
    Always try to be helpful and control whatever device the user asks for.

    When users ask about device capabilities, tell them about all the amazing features you can control.""",
    tools=[set_device_status],
)
```

Overwriting home_automation_agent/agent.py

✔ Section 3: Interactive Evaluation with ADK Web UI

3.1: Launch ADK Web UI

Get the proxied URL to access the ADK web UI in the Kaggle Notebooks environment:

```
url_prefix = get_adk_proxy_url()
```

⚠ IMPORTANT: Action Required

The ADK web UI is **not running yet**. You must start it in the next cell.

1. **Run the next cell** (the one with `!adk web ...`) to start the ADK web UI.
2. Wait for that cell to show it is "Running" (it will not "complete").
3. Once it's running, **return to this button** and click it to open the UI.

(If you click the button before running the next cell, you will get a 500 error.)

[Open ADK Web UI \(after running cell below\) ↗](#)

+ Code

+ Markdown

Now you can start the ADK web UI using the following command.

🔥 **Note:** The following cell will not "complete", but will remain running and serving the ADK web UI until you manually stop the cell.

```
!adk web --url_prefix {url_prefix}
```

```
File "/usr/local/lib/python3.11/dist-packages/google/adk/cli/adk_web_server.py", line 1421, in event_generator
  async for event in agen:
File "/usr/local/lib/python3.11/dist-packages/google/adk/runners.py", line 443, in run_async
  async for event in agen:
File "/usr/local/lib/python3.11/dist-packages/google/adk/runners.py", line 427, in run with trace
```

The screenshot shows the Agent Development Kit (ADK) web UI. The interface is dark-themed and divided into several sections. At the top, there's a header with the ADK logo, session ID, user ID, and a 'Token Streaming' toggle. Below the header, there's a navigation bar with tabs for 'Trace', 'Events', 'State', 'Artifacts', 'Sessions', and 'Eval'. The main content area is split into two panels. The left panel, titled 'Invocations', shows a list of user prompts, with the first one being 'Turn on the desk lamp in the office'. The right panel shows the agent's response to this prompt, which is 'I've turned on the desk lamp in the office.' Above this response, there are two buttons: 'set_device_status' and 'set_device_status' (with a checkmark). At the top right of the right panel, there's a button labeled 'Turn on the desk lamp in the office' and a user profile icon.

4.1: Create evaluation configuration

This optional file lets us define the pass/fail thresholds. Create `test_config.json` in the root directory.

```
import json

# Create evaluation configuration with basic criteria
eval_config = {
    "criteria": {
        "tool_trajectory_avg_score": 1.0, # Perfect tool usage required
        "response_match_score": 0.8, # 80% text similarity threshold
    }
}

with open("home_automation_agent/test_config.json", "w") as f:
    json.dump(eval_config, f, indent=2)

print("✅ Evaluation configuration created!")
print("\n📋 Evaluation Criteria:")
print("• tool_trajectory_avg_score: 1.0 - Requires exact tool usage match")
print("• response_match_score: 0.8 - Requires 80% text similarity")
print("\n🔍 What this evaluation will catch:")
print("✅ Incorrect tool usage (wrong device, location, or status)")
print("✅ Poor response quality and communication")
print("✅ Deviations from expected behavior patterns")
```

✅ Evaluation configuration created!

📋 Evaluation Criteria:

- tool_trajectory_avg_score: 1.0 - Requires exact tool usage match
- response_match_score: 0.8 - Requires 80% text similarity

🔍 What this evaluation will catch:

- ✅ Incorrect tool usage (wrong device, location, or status)
- ✅ Poor response quality and communication
- ✅ Deviations from expected behavior patterns

4.2: Create test cases

This file (`integration.evalset.json`) will contain multiple test cases (sessions).

This evaluation set can be created synthetically or from the conversation sessions in the ADK web UI.

Tip: To persist the conversations from the ADK web UI, simply create an evalset in the UI and add the current session to it. All the conversations in that session will be auto-converted to ar

+ Code

+ Markdown

```
# Create evaluation test cases that reveal tool usage and response quality problems
test_cases = {
  "eval_set_id": "home_automation_integration_suite",
  "eval_cases": [
    {
      "eval_id": "living_room_light_on",
      "conversation": [
        {
          "user_content": {
            "parts": [
              {"text": "Please turn on the floor lamp in the living room"}
            ]
          },
          "final_response": {
            "parts": [
              {
                "text": "Successfully set the floor lamp in the living room to on."
              }
            ]
          },
          "intermediate_data": {
            "tool_uses": [
              {
                "name": "set_device_status",
                "args": {
                  "location": "living room",
                  "device_id": "floor lamp",
                  "status": "ON",

```

```

    },
    ],
  },
]

```

Let's write the test cases to the `integration.evalset.json` in our agent's root directory.

```

import json

with open("home_automation_agent/integration.evalset.json", "w") as f:
    json.dump(test_cases, f, indent=2)

print("✅ Evaluation test cases created")
print("\n🔧 Test scenarios:")
for case in test_cases["eval_cases"]:
    user_msg = case["conversation"][0]["user_content"]["parts"][0]["text"]
    print(f"• {case['eval_id']}: {user_msg}")

print("\n📋 Expected results:")
print("• basic_device_control: Should pass both criteria")
print(
    "• wrong_tool_usage_test: May fail tool_trajectory if agent uses wrong parameters"
)
print(
    "• poor_response_quality_test: May fail response_match if response differs too much"
)

```

✅ Evaluation test cases created

🔧 Test scenarios:

- living_room_light_on: Please turn on the floor lamp in the living room
- kitchen_on_off_sequence: Switch on the main light in the kitchen.

📋 Expected results:

- basic_device_control: Should pass both criteria
- wrong_tool_usage_test: May fail tool_trajectory if agent uses wrong parameters
- poor_response_quality_test: May fail response_match if response differs too much

4.3: Run CLI Evaluation

Execute the `adk eval` command, pointing it to your agent directory, the evalset, and the config file.

```

print("🔧 Run this command to execute evaluation:")
!adk eval home_automation_agent home_automation_agent/integration.evalset.json --config_file_path=home_automation_agent/test_config.json --print_detailed_results

```

```

🔧 Run this command to execute evaluation:
/usr/local/lib/python3.11/dist-packages/google/adk/evaluation/metric_evaluator_registry.py:90: UserWarning: [EXPERIMENTAL] MetricEvaluatorRegistry: This feature is experimental and may change or be removed without notice. It may introduce breaking changes at any time.
  metric_evaluator_registry = MetricEvaluatorRegistry()
/usr/local/lib/python3.11/dist-packages/google/adk/evaluation/local_eval_service.py:79: UserWarning: [EXPERIMENTAL] UserSimulatorProvider: This feature is experimental and may change or be removed without notice. It may introduce breaking changes at any time.
  user_simulator_provider = UserSimulatorProvider = UserSimulatorProvider(),
Using evaluation criteria: criteria={'tool_trajectory_avg_score': 1.0, 'response_match_score': 0.8} user_simulator_config=None
/usr/local/lib/python3.11/dist-packages/google/adk/cli/cli_tools_click.py:650: UserWarning: [EXPERIMENTAL] UserSimulatorProvider: This feature is experimental and may change or be removed without notice. It may introduce breaking changes at any time.
  user_simulator_provider = UserSimulatorProvider()
/usr/local/lib/python3.11/dist-packages/google/adk/cli/cli_tools_click.py:655: UserWarning: [EXPERIMENTAL] LocalEvalService: This feature is experimental and may change or be removed without notice. It may introduce breaking changes at any time.
  eval_service = LocalEvalService()
/usr/local/lib/python3.11/dist-packages/google/adk/evaluation/user_simulator_provider.py:77: UserWarning: [EXPERIMENTAL] StaticUserSimulator: This feature is experimental and may change or be removed without notice. It may introduce breaking changes at any time.
  return StaticUserSimulator(static_conversation=eval_case.conversation)
/usr/local/lib/python3.11/dist-packages/google/adk/evaluation/static_user_simulator.py:39: UserWarning: [EXPERIMENTAL] UserSimulator: This feature is experimental and may change or be removed without notice. It may introduce breaking changes at any time.
  super().__init__()

```

```
Eval Set Id: home_automation_integration_suite
Eval Id: living_room_light_on
Overall Eval Status: FAILED

Metric: tool_trajectory_avg_score, Status: PASSED, Score: 1.0, Threshold: 1.0

Metric: response_match_score, Status: FAILED, Score: 0.0, Threshold: 0.8

Invocation Details:
| | prompt | expected_response | actual_response | expected_tool_calls | actual_tool_calls | tool_trajectory_avg_score | response_match_score |
| 0 | Please turn on the floor lamp in the living room | Successfully set the floor lamp in the living room to on. | | id=None args={'location': 'living room', 'device_id': 'floor lamp', 'status': 'ON'} name='set_device_status' | id='adk-12e9a3f1-b8b2-4576-961c-866ccaa95fbc' args={'location': 'living room', 'device_id': 'floor lamp', 'status': 'ON'} name='set_device_status' | Status: PASSED, Score: 1.0 | Status: FAILED, Score: 0.0 |
```

```
*****
Eval Set Id: home_automation_integration_suite
Eval Id: Kitchen_on_off_sequence
Overall Eval Status: FAILED

Metric: tool_trajectory_avg_score, Status: PASSED, Score: 1.0, Threshold: 1.0

Metric: response_match_score, Status: FAILED, Score: 0.7, Threshold: 0.8

Invocation Details:
| | prompt | expected_response | actual_response | expected_tool_calls | actual_tool_calls | tool_trajectory_avg_score | response_match_score |
| 0 | Switch on the main light in the kitchen. | Successfully set the main light in the kitchen to on. | The main light in the kitchen has been switched on. | id=None args={'location': 'kitchen', 'device_id': 'main light', 'status': 'ON'} name='set_device_status' | id='adk-e5299538-978f-4b70-a394-bc125d7defc8' args={'status': 'ON', 'location': 'kitchen', 'device_id': 'main light'} name='set_device_status' | Status: PASSED, Score: 1.0 | Status: FAILED, Score: 0.7 |
```

4.4: Analyzing sample evaluation results

The command will run all test cases and print a summary. The `--print_detailed_results` flag provides a turn-by-turn breakdown of each test, showing scores and a diff for any failures.

```
# Analyzing evaluation results - the data science approach
print("📊 Understanding Evaluation Results:")
print()
print("🔍 EXAMPLE ANALYSIS:")
print()
print("Test Case: living_room_light_on")
print("❌ response_match_score: 0.45/0.80")
print("✅ tool_trajectory_avg_score: 1.0/1.0")
print()
print("📝 What this tells us:")
print("• TOOL USAGE: Perfect - Agent used correct tool with correct parameters")
print("• RESPONSE QUALITY: Poor - Response text too different from expected")
print("• ROOT CAUSE: Agent's communication style, not functionality")
print()
print("💡 ACTIONABLE INSIGHTS:")
print("1. Technical capability works (tool usage perfect)")
print("2. Communication needs improvement (response quality failed)")
print("3. Fix: Update agent instructions for clearer language or constrained response.")
print()
```

📊 Understanding Evaluation Results:

🔍 EXAMPLE ANALYSIS:

Test Case: living_room_light_on

❌ response_match_score: 0.45/0.80

✅ tool_trajectory_avg_score: 1.0/1.0

📝 What this tells us:

- TOOL USAGE: Perfect - Agent used correct tool with correct parameters
- RESPONSE QUALITY: Poor - Response text too different from expected
- ROOT CAUSE: Agent's communication style, not functionality

💡 ACTIONABLE INSIGHTS:

1. Technical capability works (tool usage perfect)
2. Communication needs improvement (response quality failed)
3. Fix: Update agent instructions for clearer language or constrained response.



Día 4 (Calidad de agente)

Bienvenidos al Día 4.

Este documento técnico aborda el reto de garantizar la calidad en los agentes de AI mediante la introducción de un marco de evaluación holístico. La base técnica necesaria para esto es la Observabilidad, basada en tres pilares: Registros (el diario), Trazas (la narrativa) y Métricas (el informe de salud), que permite un bucle continuo de retroalimentación mediante métodos escalables como LLM-como-juez y evaluación Human-in-the-Loop (HITL).

En los laboratorios de código, aprenderás a usar logs, trazas y métricas para obtener visibilidad completa del proceso de toma de decisiones de tu agente, permitiéndote depurar fallos y entender por qué tu agente se comporta como lo hace. En el segundo codelab, aprenderás a evaluar a tus agentes para puntuar la calidad de respuesta y el uso de herramientas.

Día 4 Asignación

Completa Unidad 4 - "Calidad del Agente":

- ☒ Escucha el episodio resumen del podcast de esta unidad.
- ☒ Para complementar el pódcast, lee el [whitepaper de Calidad de Agente](#).
- ☒ Completa estos codelabs en Kaggle:
 - ☒ [Implementa observabilidad](#) para ayudarte a depurar tus agentes.
 - ☒ [Evalúa a tus agentes](#).

[Opcional]: Mira la [retransmisión en directo grabada de YouTube](#). Kanchana Patlolla y Anant Nawalgaria estuvieron acompañados por la autora de codelab, Sita Lakshmi Sangameswaran, junto con invitados de Google: Wafae Bakkali, Turan Bulmus y Sian Gooding, y la invitada externa Jiwei Liu de [NVIDIA](#).

DIA 5

Day 5a - Agent2Agent Communication

No se pudo guardar el borrador.

Archivo Edición Vista Corre Escenarios Complementos Ayuda

+ - ✂ 📄 📁 ▶ ⏩ Ejecutar todo Código ▾

● Sesión de borrador Carrera11m DDH SPC RAM 🔌 ↺ ⋮

Setup

Before we go into today's concepts, follow the steps below to set up the environment.

Install dependencies ¶

The Kaggle Notebooks environment includes a pre-installed version of the [google-adk](#) library for Python and its required dependencies, so you don't need to install additional packages in this notebook.

To install and use ADK, including A2A and its dependencies, in your own Python development environment outside of this course, you can do so by running:

```
pip install -q google-adk[a2a]
```

+ Code + Markdown

```
[6]: pip install -q google-adk[a2a]
```

Note: you may need to restart the kernel to use updated packages.

3. Authenticate in the notebook

Run the cell below to access the `GOOGLE_API_KEY` you just saved and set it as an environment variable for the notebook to use:

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Setup and authentication complete.")
except Exception as e:
    print(
        f"🔥 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}"
    )
```

✅ Setup and authentication complete.

Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

```
import json
import requests
import subprocess
import time
import uuid

from google.adk.agents import LlmAgent
from google.adk.agents.remote_a2a_agent import (
    RemoteA2aAgent,
    AGENT_CARD_WELL_KNOWN_PATH,
)

from google.adk.a2a.utils.agent_to_a2a import to_a2a
from google.adk.models.google_llm import Gemini
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai import types

# Hide additional warnings in the notebook
import warnings

warnings.filterwarnings("ignore")

print("✅ ADK components imported successfully.")
```

✅ ADK components imported successfully.

Section 1: Create the Product Catalog Agent (To Be Exposed)

We'll create a **Product Catalog Agent** that provides product information from an external vendor's catalog. This agent will be **exposed via A2A** so other agents (like customer support agents) can use it.

Why expose this agent?

- In a real system, this would be maintained by an **external vendor** or third-party provider
- Your internal agents (customer support, sales, inventory) need product data
- The vendor **controls their own codebase** - you can't modify their implementation
- By exposing it via A2A, any authorized agent can consume it using the standard protocol

```
# Define a product catalog lookup tool
# In a real system, this would query the vendor's product database
def get_product_info(product_name: str) -> str:
    """Get product information for a given product.

    Args:
        product_name: Name of the product (e.g., "iPhone 15 Pro", "MacBook Pro")

    Returns:
        Product information as a string
    """
    # Mock product catalog - in production, this would query a real database
    product_catalog = {
        "iphone 15 pro": "iPhone 15 Pro, $999, Low Stock (8 units), 128GB, Titanium finish",
        "samsung galaxy s24": "Samsung Galaxy S24, $799, In Stock (31 units), 256GB, Phantom Black",
        "dell xps 15": "Dell XPS 15, $1,299, In Stock (45 units), 15.6" display, 16GB RAM, 512GB SSD",
        "macbook pro 14": "MacBook Pro 14", $1,999, In Stock (22 units), M3 Pro chip, 18GB RAM, 512GB SSD",
        "sony wh-1000xm5": "Sony WH-1000XM5 Headphones, $399, In Stock (67 units), Noise-canceling, 30hr battery",
        "ipad air": "iPad Air, $599, In Stock (28 units), 10.9" display, 64GB",
        "lg ultrawide 34": "LG UltraWide 34" Monitor, $499, Out of Stock, Expected: Next week",
    }

    product_lower = product_name.lower().strip()

    if product_lower in product_catalog:
        return f"Product: {product_catalog[product_lower]}"
    else:
        available = ", ".join([p.title() for p in product_catalog.keys()])
        return f"Sorry, I don't have information for {product_name}. Available products: {available}"
```

```

product_catalog = {
    "iphone 15 pro": "iPhone 15 Pro, $999, Low Stock (8 units), 128GB, Titanium finish",
    "samsung galaxy s24": "Samsung Galaxy S24, $799, In Stock (31 units), 256GB, Phantom Black",
    "dell xps 15": "Dell XPS 15, $1,299, In Stock (45 units), 15.6" display, 16GB RAM, 512GB SSD",
    "macbook pro 14": "MacBook Pro 14", $1,999, In Stock (22 units), M3 Pro chip, 18GB RAM, 512GB SSD",
    "sony wh-1000xm5": "Sony WH-1000XM5 Headphones, $399, In Stock (67 units), Noise-canceling, 30hr battery",
    "ipad air": "iPad Air, $599, In Stock (28 units), 10.9" display, 64GB",
    "lg ultrawide 34": "LG UltraWide 34" Monitor, $499, Out of Stock, Expected: Next week",
}

product_lower = product_name.lower().strip()

if product_lower in product_catalog:
    return f"Product: {product_catalog[product_lower]}"
else:
    available = ", ".join([p.title() for p in product_catalog.keys()])
    return f"Sorry, I don't have information for {product_name}. Available products: {available}"

# Create the Product Catalog Agent
# This agent specializes in providing product information from the vendor's catalog
product_catalog_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="product_catalog_agent",
    description="External vendor's product catalog agent that provides product information and availability.",
    instruction="""
You are a product catalog specialist from an external vendor.
When asked about products, use the get_product_info tool to fetch data from the catalog.
Provide clear, accurate product information including price, availability, and specs.
If asked about multiple products, look up each one.
Be professional and helpful.
""",
    tools=[get_product_info], # Register the product lookup tool
)

print("✅ Product Catalog Agent created successfully!")
print("  Model: gemini-2.5-flash-lite")
print("  Tool: get_product_info()")
print("  Ready to be exposed via A2A...")

✅ Product Catalog Agent created successfully!
Model: gemini-2.5-flash-lite
Tool: get_product_info()
Ready to be exposed via A2A...

```

Every A2A agent must publish its agent card at the standard path: `/.well-known/agent-card.json`

Think of it as the "contract" that tells other agents how to work with your agent.

Learn more:

- [Exposing Agents with ADK](#)
- [A2A Protocol Specification](#)

```
# Convert the product catalog agent to an A2A-compatible application
# This creates a FastAPI/Starlette app that:
# 1. Serves the agent at the A2A protocol endpoints
# 2. Provides an auto-generated agent card
# 3. Handles A2A communication protocol
product_catalog_a2a_app = to_a2a(
    product_catalog_agent, port=8001 # Port where this agent will be served
)

print("✅ Product Catalog Agent is now A2A-compatible!")
print("  Agent will be served at: http://localhost:8001")
print("  Agent card will be at: http://localhost:8001/.well-known/agent-card.json")
print("  Ready to start the server...")
```

```
✅ Product Catalog Agent is now A2A-compatible!
Agent will be served at: http://localhost:8001
Agent card will be at: http://localhost:8001/.well-known/agent-card.json
Ready to start the server...
```

Section 3: Start the Product Catalog Agent Server

We'll start the Product Catalog Agent server in the **background** using `uvicorn`, so it can serve requests from other agents.

Why run in background?

- The server needs to keep running while we create and test the Customer Support Agent
- This simulates a real-world scenario where different agents run as separate services
- In production, the vendor would host this on their infrastructure

```
# First, let's save the product catalog agent to a file that uvicorn can import
product_catalog_agent_code = '''
import os
from google.adk.agents import LlmAgent
from google.adk.a2a.utils.agent_to_a2a import to_a2a
from google.adk.models.google_llm import Gemini
from google.genai import types

retry_config = types.HttpRetryOptions(
    attempts=5, # Maximum retry attempts
    exp_base=7, # Delay multiplier
    initial_delay=1,
    http_status_codes=[429, 500, 503, 504], # Retry on these HTTP errors
)

def get_product_info(product_name: str) -> str:
    """Get product information for a given product."""
    product_catalog = {
        "iphone 15 pro": "iPhone 15 Pro, $999, Low Stock (8 units), 128GB, Titanium finish",
        "samsung galaxy s24": "Samsung Galaxy S24, $799, In Stock (31 units), 256GB, Phantom Black",
        "dell xps 15": "Dell XPS 15, $1,299, In Stock (45 units), 15.6\\\" display, 16GB RAM, 512GB SSD",
        "macbook pro 14": "MacBook Pro 14\\\", $1,999, In Stock (22 units), M3 Pro chip, 18GB RAM, 512GB SSD",
        "sony wh-1000xm5": "Sony WH-1000XM5 Headphones, $399, In Stock (67 units), Noise-canceling, 30hr battery",
        "ipad air": "iPad Air, $599, In Stock (28 units), 10.9\\\" display, 64GB",
        "lg ultrawide 34": "LG UltraWide 34\\\" Monitor, $499, Out of Stock, Expected: Next week",
    }
    product_info = product_catalog.get(product_name, "Product not found in catalog.")
    return product_info
'''
```

```

    "uvicorn",
    "product_catalog_server:app", # Module:app format
    "--host",
    "localhost",
    "--port",
    "8001",
],
cwd="/tmp", # Run from /tmp where the file is
stdout=subprocess.PIPE,
stderr=subprocess.PIPE,
env={**os.environ}, # Pass environment variables (including GOOGLE_API_KEY)
)

print("🚀 Starting Product Catalog Agent server...")
print("   Waiting for server to be ready...")

# Wait for server to start (poll until it responds)
max_attempts = 30
for attempt in range(max_attempts):
    try:
        response = requests.get(
            "http://localhost:8001/.well-known/agent-card.json", timeout=1
        )
        if response.status_code == 200:
            print(f"\n✅ Product Catalog Agent server is running!")
            print(f"   Server URL: http://localhost:8001")
            print(f"   Agent card: http://localhost:8001/.well-known/agent-card.json")
            break
    except requests.exceptions.RequestException:
        time.sleep(5)
        print(".", end="", flush=True)
else:
    print("\n⚠️ Server may not be ready yet. Check manually if needed.")

# Store the process so we can stop it later
globals()["product_catalog_server_process"] = server_process

```

📁 Product Catalog agent code saved to /tmp/product_catalog_server.py

🚀 Starting Product Catalog Agent server...
Waiting for server to be ready...

.....

✅ Product Catalog Agent server is running!
Server URL: <http://localhost:8001>
Agent card: <http://localhost:8001/.well-known/agent-card.json>

View the Auto-Generated Agent Card

The `to_a2a()` function automatically created an **agent card** that describes the Product Catalog Agent's capabilities. Let's take a look!

```
# Fetch the agent card from the running server
try:
    response = requests.get(
        "http://localhost:8001/.well-known/agent-card.json", timeout=5
    )

    if response.status_code == 200:
        agent_card = response.json()
        print("📄 Product Catalog Agent Card:")
        print(json.dumps(agent_card, indent=2))

        print("\n🌟 Key Information:")
        print(f"    Name: {agent_card.get('name')}")
        print(f"    Description: {agent_card.get('description')}")
        print(f"    URL: {agent_card.get('url')}")
        print(f"    Skills: {len(agent_card.get('skills', []))} capabilities exposed")
    else:
        print(f"❌ Failed to fetch agent card: {response.status_code}")

except requests.exceptions.RequestException as e:
    print(f"❌ Error fetching agent card: {e}")
    print("    Make sure the Product Catalog Agent server is running (previous cell)")
```

📄 Product Catalog Agent Card:

```
{
  "capabilities": {},
  "defaultInputModes": [
    "text/plain"
  ],
  "defaultOutputModes": [
    "text/plain"
  ],
  "description": "External vendor's product catalog agent that provides product information and availability.",
  "name": "product_catalog_agent",
  "preferredTransport": "JSONRPC",
  "protocolVersion": "0.3.0",
  "skills": [
```

🎧 Section 4: Create the Customer Support Agent (Consumer)

Now we'll create a **Customer Support Agent** that consumes the Product Catalog Agent using A2A.

How it works:

1. We use `RemoteA2aAgent` to create a **client-side proxy** for the Product Catalog Agent
2. The Customer Support Agent can use the Product Catalog Agent like any other tool
3. ADK handles all the A2A protocol communication behind the scenes

This demonstrates the power of A2A: **agents can collaborate as if they were local!**

How RemoteA2aAgent works:

- It's a **client-side proxy** that reads the remote agent's card
- Translates sub-agent calls into A2A protocol requests (HTTP POST to `/tasks`)
- Handles all the protocol details so you just use it like a regular sub-agent

📖 Learn more:

- [Consuming Remote Agents with ADK](#)
- [What is A2A?](#)

```
# Create a RemoteA2aAgent that connects to our Product Catalog Agent
# This acts as a client-side proxy - the Customer Support Agent can use it like a local agent
remote_product_catalog_agent = RemoteA2aAgent(
    name="product_catalog_agent",
    description="Remote product catalog agent from external vendor that provides product information.",
    # Point to the agent card URL - this is where the A2A protocol metadata lives
    agent_card=f"http://localhost:8001{AGENT_CARD_WELL_KNOWN_PATH}",
)

print("✅ Remote Product Catalog Agent proxy created!")
print(f"    Connected to: http://localhost:8001")
print(f"    Agent card: http://localhost:8001{AGENT_CARD_WELL_KNOWN_PATH}")
print("    The Customer Support Agent can now use this like a local sub-agent!")
```

```
✅ Remote Product Catalog Agent proxy created!
Connected to: http://localhost:8001
Agent card: http://localhost:8001/.well-known/agent-card.json
The Customer Support Agent can now use this like a local sub-agent!
```



```

# Now create the Customer Support Agent that uses the remote Product Catalog Agent
customer_support_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="customer_support_agent",
    description="A customer support assistant that helps customers with product inquiries and information.",
    instruction="""
    You are a friendly and professional customer support agent.

    When customers ask about products:
    1. Use the product_catalog_agent sub-agent to look up product information
    2. Provide clear answers about pricing, availability, and specifications
    3. If a product is out of stock, mention the expected availability
    4. Be helpful and professional!

    Always get product information from the product_catalog_agent before answering customer questions.
    """,
    sub_agents=[remote_product_catalog_agent], # Add the remote agent as a sub-agent!
)

print("✅ Customer Support Agent created!")
print("  Model: gemini-2.5-flash-lite")
print("  Sub-agents: 1 (remote Product Catalog Agent via A2A)")
print("  Ready to help customers!")

```

```

✅ Customer Support Agent created!
Model: gemini-2.5-flash-lite
Sub-agents: 1 (remote Product Catalog Agent via A2A)
Ready to help customers!

```

Section 5: Test A2A Communication

Let's test the agent-to-agent communication! We'll ask the Customer Support Agent about products, and it will communicate with the Product Catalog Agent via A2A.

What happens behind the scenes:

1. Customer asks Support Agent a question about a product
2. Support Agent realizes it needs product info
3. Support Agent calls the `remote_product_catalog_agent` (RemoteA2aAgent)
4. ADK sends an A2A protocol request to `http://localhost:8001`
5. Product Catalog Agent processes the request and responds
6. Support Agent receives the response and continues
7. Customer gets the final answer

All of this happens **transparently** - the Support Agent doesn't need to know it's talking to a remote agent!

```

async def test_a2a_communication(user_query: str):
    """
    Test the A2A communication between Customer Support Agent and Product Catalog Agent.

    This function:
    1. Creates a new session for this conversation
    2. Sends the query to the Customer Support Agent
    3. Support Agent communicates with Product Catalog Agent via A2A
    4. Displays the response

    Args:
        user_query: The question to ask the Customer Support Agent
    """
    # Setup session management (required by ADK)
    session_service = InMemorySessionService()

    # Session identifiers
    app_name = "support_app"
    user_id = "demo_user"

    # Use unique session ID for each test to avoid conflicts
    session_id = f"demo_session_{uuid.uuid4().hex[:8]}"

```

```

# Run the agent asynchronously (handles streaming responses and A2A communication)
async for event in runner.run_async(
    user_id=user_id, session_id=session_id, new_message=test_content
):
    # Print final response only (skip intermediate events)
    if event.is_final_response() and event.content:
        for part in event.content.parts:
            if hasattr(part, "text"):
                print(part.text)

print("-" * 60)

# Run the test
print("🟢 Testing A2A Communication...\n")
await test_a2a_communication("Can you tell me about the iPhone 15 Pro? Is it in stock?")

```

🟢 Testing A2A Communication...

👤 Customer: Can you tell me about the iPhone 15 Pro? Is it in stock?

🗨️ Support Agent response:

```

INFO:google_adk.google.adk.models.google_llm:Sending out request, model: gemini-2.5-flash-lite, backend: GoogleLLMVariant.GEMINI_API, stream: False
INFO:google_adk.google.adk.models.google_llm:Response received from the model.
WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full response.
INFO:google_adk.google.adk.agents.remote_a2a_agent:Successfully resolved remote A2A agent: product_catalog_agent
The iPhone 15 Pro is available, but there are only 8 units currently in stock. It is priced at $999 and features a 128GB storage capacity with a titanium finish.

```

Try More Examples

Let's test a few more scenarios to see A2A communication in action!

```

# Test comparing multiple products
await test_a2a_communication(
    "I'm looking for a laptop. Can you compare the Dell XPS 15 and MacBook Pro 14 for me?"
)

```

```

INFO:google_adk.google.adk.models.google_llm:Sending out request, model: gemini-2.5-flash-lite, backend: GoogleLLMVariant.GEMINI_API, stream: False
👤 Customer: I'm looking for a laptop. Can you compare the Dell XPS 15 and MacBook Pro 14 for me?

```

🗨️ Support Agent response:

```

INFO:google_adk.google.adk.models.google_llm:Response received from the model.
WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full response.
The Dell XPS 15 is priced at $1,299 and has 45 units in stock. It features a 15.6" display, 16GB RAM, and 512GB SSD.

The MacBook Pro 14" is priced at $1,999 and has 22 units in stock. It is equipped with an M3 Pro chip, 18GB RAM, and 512GB SSD.

```

```

# Test specific product inquiry
await test_a2a_communication(
    "Do you have the Sony WH-1000XM5 headphones? What's the price?"
)

```

```

INFO:google_adk.google.adk.models.google_llm:Sending out request, model: gemini-2.5-flash-lite, backend: GoogleLLMVariant.GEMINI_API, stream: False
👤 Customer: Do you have the Sony WH-1000XM5 headphones? What's the price?

```

🗨️ Support Agent response:

```

INFO:google_adk.google.adk.models.google_llm:Response received from the model.
WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full response.
The Sony WH-1000XM5 headphones are in stock and priced at $399. They feature noise-canceling technology and a 30-hour battery life.

```



Día 5 (Prototipo a Producción)

Bienvenidos al Día 5.

Este whitepaper ofrece una guía técnica sobre el ciclo de vida operativo de los agentes de IA, centrándose en el despliegue, escalamiento y producción. Explora los retos de la transición de sistemas agentivos de prototipos a soluciones de nivel empresarial, con especial atención al Protocolo Agent2Agent (A2A).

En los codelabs, aprenderás a construir sistemas de múltiples agentes independientes que puedan comunicarse y colaborar usando el Protocolo A2A. También aprenderás a llevar tu agente de tu máquina local a un servicio escalable listo para producción, desplegando tu agente en Vertex AI Agent Engine en Google Cloud.

Asignación

final Unidad 5 - "Prototipo a producción":

- ✓ Escucha el episodio resumen del podcast de esta unidad.
- ✓ Para complementar el pódcast, lee el whitepaper "Prototipo a Producción". .
- ✓ Completa estos codelabs en Kaggle:
 - ✓ Explora cómo usar el Protocolo A2A para que los agentes interactúen entre sí.
 - ✓ [Opcional] Despliega tu agente en Agent Engine en Google Cloud.